

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO
ĐỒ ÁN CHUYÊN NGÀNH

**Phân tích mối liên hệ về tình trạng giao
thông tại các nút giao thông quan trọng
ở TP.HCM**

Ngành: Khoa học máy tính

Hội đồng:

GVHD1: PGS.TS. Trần Minh Quang

GVHD2: ThS. Đỗ Thanh Thái

—o0o—

SVTH 1: Nguyễn Lê Quốc Khánh (2211522)

SVTH 3: Nguyễn Minh Khang (2211452)

SVTH 2: Nguyễn Hữu Khánh (2211521)

TP. HỒ CHÍ MINH, THÁNG 12/2025

Lời cam đoan

Chúng tôi xin cam đoan rằng toàn bộ nội dung trong báo cáo này là kết quả của quá trình nghiên cứu, tìm hiểu và làm việc nghiêm túc của nhóm. Các nội dung trích dẫn từ tài liệu, công trình nghiên cứu, nguồn dữ liệu và các hình ảnh minh họa đều được ghi rõ nguồn hoặc đã được xử lý phù hợp với mục đích học thuật.

Báo cáo không sao chép hoặc sử dụng bất kỳ nội dung nào từ các công trình khác mà không trích dẫn theo đúng quy định. Các phân tích, đánh giá, biểu đồ và kết luận được trình bày trong báo cáo được xây dựng dựa trên những dữ liệu mà nhóm đã thu thập, xử lý và phân tích trong quá trình thực hiện đề tài.

Chúng tôi hoàn toàn chịu trách nhiệm trước nhà trường và giảng viên hướng dẫn về tính trung thực, tính nguyên bản cũng như độ chính xác của toàn bộ nội dung trong báo cáo này. Nếu phát hiện bất kỳ sai phạm nào liên quan đến tính trung thực học thuật, chúng tôi xin chịu mọi hình thức kỷ luật theo quy định.

Tác giả

Nguyễn Lê Quốc Khánh

Nguyễn Hữu Khánh

Nguyễn Minh Khang

Lời cảm ơn

Trước hết, nhóm chúng tôi xin được gửi lời cảm ơn sâu sắc đến thầy Quang và thầy Thái - những giảng viên đã tận tình hướng dẫn, định hướng và hỗ trợ chúng tôi trong suốt quá trình thực hiện đề tài. Sự chỉ dẫn tận tâm của quý thầy, từ việc định hình hướng nghiên cứu, góp ý kỹ thuật đến việc hoàn thiện nội dung báo cáo, đã giúp nhóm mở rộng kiến thức, tiếp cận đúng phương pháp và phát triển dự án một cách có hệ thống.

Bên cạnh đó, nhóm xin trân trọng cảm ơn các thầy cô trong Khoa đã cung cấp nền tảng kiến thức vững chắc thông qua chương trình học và các buổi giảng dạy, giúp nhóm có đủ năng lực để thực hiện dự án nghiên cứu này. Đồng thời, chúng tôi xin được cảm ơn những người bạn đã hỗ trợ nhóm trên con đường hoàn thành dự án.

Nhóm cũng xin gửi lời cảm ơn đến các thành viên đã luôn hợp tác, hỗ trợ và đồng hành trong suốt quá trình triển khai. Tinh thần làm việc nghiêm túc, sự trao đổi kỹ thuật liên tục và ý thức trách nhiệm của từng thành viên là yếu tố quan trọng giúp nhóm hoàn thành đúng tiến độ và mục tiêu đề ra.

Chúng tôi cũng ghi nhận sự hỗ trợ gián tiếp từ các nguồn tài nguyên học thuật, công cụ phần mềm, nền tảng dữ liệu mở và các công cụ phân tích đã được sử dụng trong quá trình thực nghiệm. Những nguồn tài nguyên này đã giúp nhóm mở rộng khả năng nghiên cứu và triển khai các ý tưởng một cách hiệu quả hơn.

Một lần nữa, nhóm xin gửi lời cảm ơn chân thành đến tất cả những sự hỗ trợ quý báu nói trên.

Tác giả

Nguyễn Lê Quốc Khánh

Nguyễn Hữu Khánh

Nguyễn Minh Khang

Tóm tắt

Tình trạng giao thông đô thị là một trong những vấn đề trọng yếu tại các thành phố lớn. Đặc biệt tại thành phố Hồ Chí Minh, lưu lượng phương tiện giao thông liên tục tăng, gây ra áp lực lớn lên mạng lưới giao thông hiện hữu. Báo cáo này trình bày quá trình nghiên cứu, thu thập và phân tích dữ liệu nhằm đánh giá tình trạng giao thông trong đô thị, đồng thời khám phá mối quan hệ giữa các đoạn đường và khu vực trong mạng lưới giao thông.

Trong dự án này, dữ liệu giao thông được thu thập từ các nguồn số hóa hiện có, trải qua quá trình tiền xử lý nhằm chuẩn hóa, làm sạch và cấu trúc hóa dữ liệu để phục vụ cho phân tích. Sau đó, nhóm sẽ tiến hành khảo sát nhiều phương pháp phân tích và mô hình hóa khác nhau, từ các kỹ thuật phân tích truyền thống cho đến các mô hình học sâu hiện đại, nhằm tìm ra hướng tiếp cận phù hợp cho việc nhận diện xu hướng, đánh giá mức độ ùn tắc và phát hiện mối liên hệ giữa các đoạn đường.

Hệ thống được đề xuất hướng tới việc tích hợp kết quả phân tích vào một nền tảng trực tuyến về giao thông hiện có, giúp người dùng và cơ quan quản lý dễ dàng theo dõi tình trạng giao thông, xem các biểu đồ trực quan và đánh giá các chỉ số quan trọng theo thời gian và không gian.

Kết quả nghiên cứu không chỉ cung cấp một quy trình phân tích mang tính tổng quát, có khả năng mở rộng sang nhiều loại dữ liệu khác nhau, mà còn đưa ra các gợi ý về hướng tiếp cận mô hình hóa, giúp hỗ trợ cho các nghiên cứu chuyên sâu hơn trong tương lai. Dự án góp phần minh chứng tiềm năng của dữ liệu số và các kỹ thuật phân tích hiện đại trong việc nâng cao hiệu quả quản lý giao thông đô thị.



Mục lục



Danh sách hình vẽ



Danh sách bảng

1 Giới thiệu

1.1 Động cơ của đề tài

Trong bối cảnh toàn cầu hóa và đô thị hóa diễn ra mạnh mẽ, Thành phố Hồ Chí Minh (TP.HCM) với vai trò là trung tâm kinh tế lớn nhất Việt Nam đang đối mặt với nhiều thách thức về quản lý đô thị, trong đó vấn đề giao thông là một trong những bài toán nan giải nhất. Theo báo cáo từ Sở Giao thông Vận tải TP.HCM năm 2023, trung bình mỗi ngày thành phố ghi nhận hơn 1.000 vụ ùn tắc giao thông cục bộ với khoảng 1,5 triệu ô tô và hơn 8 triệu xe máy lưu thông trong giờ cao điểm. Tình trạng này không chỉ gây áp lực khổng lồ lên hạ tầng giao thông vốn đã quá tải mà còn tạo ra những hệ lụy nghiêm trọng về kinh tế - xã hội.

Về mặt kinh tế, ùn tắc giao thông làm tăng chi phí logistics, gây chậm trễ trong vận chuyển hàng hóa và giảm năng suất lao động do người lao động mất nhiều thời gian di chuyển. Về mặt xã hội, nó ảnh hưởng trực tiếp đến chất lượng cuộc sống của người dân thông qua việc tăng thời gian đi lại, gây căng thẳng tâm lý, và làm gia tăng ô nhiễm không khí do phương tiện phải hoạt động với hiệu suất thấp trong trạng thái dừng - chạy liên tục.

Một đặc điểm quan trọng nhưng thường bị bỏ qua trong bài toán giao thông đô thị là **hiện tượng lan truyền tắc nghẽn** (cascading congestion). Khi một nút giao thông quan trọng bị tắc nghẽn, nó không chỉ gây ảnh hưởng cục bộ mà còn tạo ra hiệu ứng dây chuyền lan rộng đến các nút giao thông lân cận và xa hơn trong mạng lưới. Hiện tượng này diễn ra do các tuyến đường trong đô thị có mối liên hệ chặt chẽ với nhau: dòng xe từ nút A sẽ di chuyển đến nút B, rồi tiếp tục đến nút C, tạo thành một chuỗi phụ thuộc phức tạp. Việc hiểu rõ và định lượng được các mối liên hệ này là then chốt để có thể dự báo, cảnh báo sớm và đưa ra giải pháp điều phối giao thông hiệu quả.

Tuy nhiên, qua quan sát thực tế tại TP.HCM, nhóm nghiên cứu nhận thấy rằng phần lớn hoạt động điều phối và giảm ùn tắc giao thông hiện nay vẫn mang tính **phản ứng thụ động** hơn là **dự báo chủ động**. Các cơ quan chức năng thường chỉ vào cuộc xử lý khi đã nhận được tin báo về tình trạng mất trật tự hoặc dựa vào kinh nghiệm về những đoạn đường thường xuyên xảy ra tắc nghẽn. Điều này cho thấy mặc dù đã có các hệ thống giám sát và thu thập dữ liệu giao thông như UTraffic, nhưng dữ liệu này chưa được khai thác và phân tích một cách tối ưu để chuyển hóa thành các thông tin hữu ích phục vụ công tác dự báo và ra quyết định.

Chính từ những nhận định trên, nhóm nghiên cứu quyết định hướng đến giải pháp **phân tích mối liên hệ về tình trạng giao thông giữa các nút giao thông quan trọng tại TP.HCM**.

Với việc tích hợp các phương pháp phân tích hiện đại như Graph Neural Networks (GNN), phân tích chuỗi thời gian, và các kỹ thuật Big Data, đề tài hướng đến mục tiêu xây dựng một hệ thống có khả năng:

- Phát hiện và định lượng mối tương quan giữa các nút giao thông trong mạng lưới đô thị.
- Xác định các mối quan hệ nhân quả (causal relationships) để biết nút giao nào là nguồn gốc gây ra tắc nghẽn lan truyền.
- Cảnh báo sớm khi phát hiện nguy cơ lan truyền tắc nghẽn từ một nút sang các nút khác.
- Cung cấp công cụ trực quan hóa giúp các nhà quản lý giao thông có cái nhìn toàn cảnh về mạng lưới giao thông.

Đây là một hướng tiếp cận mang tính chiến lược và lâu dài, góp phần chuyển đổi công tác quản lý giao thông từ mô hình phản ứng sang mô hình chủ động dựa trên dữ liệu và trí tuệ nhân tạo, từ đó cải thiện chất lượng cuộc sống của người dân TP.HCM.

1.2 Mục tiêu

Mục tiêu tổng quát của đề tài là nghiên cứu và phát triển giải pháp phân tích mối liên hệ về tình trạng giao thông tại các nút giao thông quan trọng ở TP.HCM dựa trên dữ liệu thực tế, từ đó cung cấp cơ sở khoa học cho công tác dự báo, cảnh báo sớm và ra quyết định trong quản lý giao thông đô thị. Cụ thể, đề tài hướng đến các mục tiêu sau:

- **Khảo sát và đánh giá các giải pháp hiện có:** Tổng hợp, phân tích các nghiên cứu và giải pháp phân tích, dự báo giao thông đang được áp dụng trong và ngoài nước. Từ đó xác định phương pháp phù hợp với điều kiện thực tế tại TP.HCM, đặc biệt là bối cảnh giao thông hỗn hợp với mật độ xe máy cao.
- **Phát triển mô hình phân tích mối quan hệ không gian - thời gian:** Xây dựng mô hình có khả năng học và định lượng mối tương quan giữa các nút giao thông trong mạng lưới đô thị. Mô hình cần phát hiện được các cặp nút có độ ảnh hưởng cao lẫn nhau, xác định độ trễ lan truyền (propagation delay), và nhận diện các nút giao thông đóng vai trò quan trọng (hub nodes) trong việc gây ra hoặc kiểm soát tắc nghẽn.
- **Xây dựng hệ thống phân tích và cảnh báo:** Phát triển một hệ thống demo tích hợp vào UTraffic với các chức năng:
 - Thu thập và xử lý dữ liệu giao thông thời gian thực.
 - Phân tích và trực quan hóa mối liên hệ giữa các nút giao thông trên bản đồ.
 - Cảnh báo sớm khi phát hiện nguy cơ lan truyền tắc nghẽn.
 - Cung cấp báo cáo phân tích hỗ trợ ra quyết định.
- **Đánh giá hiệu quả trên dữ liệu thực tế:** Kiểm chứng tính khả thi, độ chính xác và hiệu năng của giải pháp đề xuất thông qua việc thử nghiệm trên dữ liệu giao thông thực tế của TP.HCM thu thập từ hệ thống UTraffic.

Với những mục tiêu trên, đề án không chỉ giải quyết vấn đề cụ thể về phân tích giao thông tại TP.HCM mà còn đóng góp vào việc phát triển hệ thống quản lý giao thông thông minh (Intelligent Transportation System - ITS) trong tương lai.

1.3 Phạm vi

Để đảm bảo tính khả thi và hiệu quả trong thời gian thực hiện đồ án, nghiên cứu được giới hạn trong các phạm vi sau:

1.3.1 Phạm vi không gian

Đề tài tập trung vào **các nút giao thông trọng điểm tại khu vực trung tâm TP.HCM**, bao gồm các quận có mật độ giao thông cao và thường xuyên xảy ra tắc nghẽn như Quận 1, Quận 3, Quận 5, Quận 10, và một số tuyến đường huyết mạch nối liền các khu vực này. Cụ thể:

- Số lượng nút giao thông nghiên cứu: 15-25 nút giao thông quan trọng.
- Tiêu chí lựa chọn: Các nút có lưu lượng phương tiện cao, thường xuyên xảy ra ùn tắc, hoặc có vị trí chiến lược (cổng vào/ra khu vực trung tâm, giao lộ lớn).
- Các tuyến đường chính được xem xét bao gồm: đường Võ Văn Kiệt, đường Cách Mạng Tháng Tám, đường Điện Biên Phủ, đường Nguyễn Thị Minh Khai, và các trục đường nối liền chúng.

1.3.2 Phạm vi thời gian

- **Dữ liệu lịch sử:** Phân tích dữ liệu trong khoảng thời gian từ 6 tháng đến 1 năm gần nhất để đảm bảo phản ánh chính xác xu hướng và đặc điểm hiện tại của tình trạng giao thông, đồng thời có đủ dữ liệu để huấn luyện mô hình học máy.
- **Phân tích theo khung giờ:** Tập trung vào các khung giờ cao điểm (7h-9h sáng, 17h-19h chiều) và so sánh với giờ thấp điểm để đánh giá sự khác biệt về mức độ tương quan.
- **Độ phân giải thời gian:** Dữ liệu được thu thập và phân tích với độ phân giải 5-10 phút để cân bằng giữa độ chi tiết và khả năng xử lý.

1.3.3 Phạm vi kỹ thuật

- **Phương pháp phân tích:** Sử dụng các kỹ thuật Big Data (Apache Spark), Deep Learning (Graph Neural Networks, LSTM), và các phương pháp thống kê (Granger Causality Test, correlation analysis).
- **Loại dữ liệu:** Dữ liệu về tốc độ trung bình, mật độ phương tiện, thời gian di chuyển tại các nút giao thông. Không bao gồm dữ liệu cá nhân hoặc dữ liệu nhận dạng phương tiện.

- **Nền tảng tích hợp:** Hệ thống được phát triển như một module mở rộng của UTraffic, tương thích với kiến trúc hiện có.

1.3.4 Giới hạn của đề tài

Đề tài **không** bao gồm:

- Dự báo dài hạn (trên 2 giờ) về tình trạng giao thông.
- Phân tích ảnh hưởng của các yếu tố thời tiết cực đoan, sự kiện đặc biệt (lễ hội, biểu tình).
- Đề xuất giải pháp điều chỉnh hạ tầng vật lý (xây dựng đường mới, cầu vượt).
- Tối ưu hóa chu kỳ đèn tín hiệu (có thể là hướng phát triển tương lai dựa trên kết quả phân tích).

1.3.5 Đối tượng thực hiện và thụ hưởng

- **Đối tượng thực hiện:** Sinh viên ngành Công nghệ thông tin, chuyên ngành Big Data và Deep Learning.
- **Đối tượng thụ hưởng:**
 - Cơ quan quản lý giao thông đô thị (Sở GTVT TP.HCM).
 - Các đơn vị cung cấp dịch vụ giám sát, điều khiển giao thông.
 - Nhà quy hoạch và nghiên cứu về hạ tầng giao thông.
 - Người dân TP.HCM thông qua việc cải thiện tình trạng giao thông.

Việc mở rộng nghiên cứu sang các khu vực khác trong TP.HCM hoặc áp dụng cho các đô thị khác tại Việt Nam sẽ được thảo luận trong phần hướng phát triển tương lai.

1.4 Ý nghĩa

1.4.1 Ý nghĩa thực tiễn

Kết quả nghiên cứu của đề án mang lại những giá trị ứng dụng thực tế quan trọng đối với công tác quản lý và vận hành mạng lưới giao thông đô thị tại TP.HCM:

- **Hỗ trợ điều phối giao thông theo mạng lưới (Network-wide Management):** Thay vì điều tiết cục bộ tại từng nút giao độc lập, giải pháp giúp các cơ quan chức năng có "bức tranh toàn cảnh" về mối liên hệ giữa các nút giao thông. Từ đó nhận diện được nút giao nào là nguyên nhân gốc rễ (root cause) gây ra sự ùn tắc lan truyền sang các khu vực lân cận, để có phương án xử lý ưu tiên và hiệu quả hơn.
- **Cảnh báo sớm và dự báo chủ động:** Hệ thống có khả năng phát hiện các dấu hiệu ban đầu của tắc nghẽn tại một nút và dự đoán khả năng lan truyền sang các nút khác dựa trên mối quan hệ đã được học. Điều này cho phép các cơ quan chức năng can thiệp sớm, giảm thiểu thời gian và mức độ ùn tắc.
- **Cơ sở cho việc đồng bộ hóa tín hiệu đèn (Signal Synchronization):** Việc định lượng được mức độ ảnh hưởng và độ trễ của dòng giao thông giữa các nút là tham số đầu vào quan trọng để thiết lập các làn sóng xanh (Green Wave) hoặc tối ưu hóa chu kỳ đèn tín hiệu tại các giao lộ liên tiếp, giúp dòng xe di chuyển mượt mà hơn.
- **Tối ưu hóa đầu tư hạ tầng giám sát:** Hệ thống giúp xác định các cặp nút giao thông có độ tương quan cao và các nút có tầm quan trọng chiến lược (hub nodes). Thông tin này hỗ trợ quy hoạch lắp đặt thêm cảm biến, camera hoặc nâng cấp hạ tầng tại các vị trí trọng yếu, tối ưu hóa chi phí đầu tư.
- **Cung cấp thông tin hữu ích cho người tham gia giao thông:** Người dùng không chỉ biết điểm nào đang kẹt mà còn hiểu được xu hướng lan truyền của tắc nghẽn, từ đó chủ động chọn lộ trình tránh các trục đường đang chịu ảnh hưởng dây chuyền, tiết kiệm thời gian di chuyển.
- **Nâng cao hiệu quả vận hành hệ thống UTraffic:** Đề tài giúp chuyển hóa dữ liệu thô từ UTraffic thành các tri thức về cấu trúc vận hành của mạng lưới giao thông, tăng giá trị sử dụng của hệ thống hiện có mà không cần đầu tư lớn vào phần cứng mới.

1.4.2 Ý nghĩa khoa học

Về mặt nghiên cứu, đề tài đóng góp những giá trị học thuật trong việc áp dụng Trí tuệ nhân tạo và Big Data vào bài toán Giao thông thông minh (Intelligent Transportation

Systems - ITS):

- **Mô hình hóa sự phụ thuộc không gian - thời gian (Spatio-Temporal Dependency):** Đề án tập trung nghiên cứu và định lượng mối quan hệ phức tạp giữa các nút giao thông trong mạng lưới đô thị. Thay vì chỉ xét khoảng cách địa lý, nghiên cứu đề xuất phương pháp học các mối liên hệ ẩn (latent correlations) dựa trên dữ liệu dòng chảy thực tế, phản ánh đúng bản chất động của hệ thống giao thông.
- **Ứng dụng Graph Neural Networks (GNN) cho giao thông Việt Nam:** Đóng góp vào việc kiểm chứng hiệu quả của các kiến trúc học sâu trên dữ liệu đồ thị (graph-based data) đối với đặc thù giao thông hỗn hợp tại Việt Nam - nơi mà các quy luật di chuyển thường phức tạp và khó nắm bắt hơn so với các nước phát triển do sự đa dạng về loại phương tiện (xe máy, ô tô, xe buýt) và văn hóa tham gia giao thông.
- **Phân tích quan hệ nhân quả trong mạng lưới giao thông:** Áp dụng các phương pháp thống kê tiên tiến như Granger Causality Test để không chỉ phát hiện tương quan mà còn xác định quan hệ nhân - quả giữa các nút giao thông. Đây là hướng nghiên cứu quan trọng nhưng chưa được khai thác nhiều trong các công trình về giao thông tại Việt Nam.
- **Xây dựng dataset giao thông TP.HCM:** Đề tài góp phần xây dựng và công bố một dataset về giao thông đô thị TP.HCM có gắn nhãn về mối liên hệ giữa các nút giao thông, phục vụ cho cộng đồng nghiên cứu trong và ngoài nước.
- **Cơ sở cho hệ thống giao thông thích nghi (Adaptive Traffic Systems):** Việc phân tích thành công mối liên hệ động (dynamic relationships) giữa các nút giao thông sẽ là tiền đề quan trọng để phát triển các hệ thống AI có khả năng tự thích nghi với sự thay đổi cấu trúc mạng lưới giao thông theo thời gian thực, hướng tới mô hình giao thông tự trị (autonomous traffic management).
- **Kết hợp đa phương pháp (Multi-method Approach):** Đề tài tích hợp nhiều phương pháp từ các lĩnh vực khác nhau: Graph Theory, Deep Learning, Time Series Analysis, Statistical Causality, tạo ra một framework toàn diện để giải quyết bài toán phức tạp về giao thông đô thị.

1.5 Cấu trúc báo cáo

Nội dung của đề án được tổ chức và trình bày theo 9 chương chính, cụ thể như sau:

- **Chương 1 - Giới thiệu:** Trình bày bối cảnh thực hiện đề tài, xác định vấn đề cần giải quyết, mục tiêu, phạm vi nghiên cứu và ý nghĩa thực tiễn của việc phân tích mối liên hệ giao thông.
- **Chương 2 - Kiến thức và công nghệ nền tảng:** Cung cấp cơ sở lý thuyết về các mô hình học sâu trên đồ thị (GCN, GAT) và chuỗi thời gian (RNN, LSTM, GRU). Chương này cũng đi sâu vào kiến trúc T-GCN, các cải tiến hiện đại như Dynamic Graph Learning, cùng với hệ sinh thái công nghệ được sử dụng (Python, PyTorch, kiến trúc Kappa với Kafka/Flink, Docker và Kubernetes).
- **Chương 3 - Công trình liên quan:** Khảo sát các nghiên cứu về mô hình lan truyền, các phương pháp Deep Learning với ma trận kề tĩnh và động. Đồng thời, chương này phân tích các hệ thống thực tế như Google Maps, TomTom, UTraffic để xác định khoảng trống nghiên cứu (Gap Analysis) mà đề tài hướng tới giải quyết.
- **Chương 4 - Phương pháp giải quyết:** Đây là chương trọng tâm mô tả chi tiết quy trình khoa học dữ liệu và thuật toán. Nội dung bao gồm:
 - Quy trình thu thập và tiền xử lý dữ liệu từ TomTom API.
 - Trích xuất đặc trưng không gian - thời gian và phân tích tương quan (DTW, Wavelet).
 - Kiểm định quan hệ nhân quả (Granger Causality) để xây dựng đồ thị nhân quả.
 - Xây dựng mô hình dự báo Dynamic Graph Convolutional Network và chiến lược huấn luyện.
 - Thiết kế hệ thống cảnh báo, phát hiện lan truyền tắc nghẽn và quy trình triển khai (Deployment).
- **Chương 5 - Hệ thống đề xuất:** Tập trung vào khía cạnh kỹ thuật phần mềm, mô tả mục đích hệ thống, nhóm người dùng mục tiêu, và đặc tả chi tiết các yêu cầu chức năng, phi chức năng cũng như các yêu cầu khắt khe về dữ liệu đầu vào/đầu ra.
- **Chương 6 - Phân tích & Thiết kế hệ thống:** Trình bày mô hình hóa tương tác người dùng, quy trình nghiệp vụ, và thiết kế chi tiết kiến trúc giải pháp công nghệ (Frontend, Backend) để hiện thực hóa các yêu cầu đã đặt ra.

- **Chương 7 - Hiện thực và kiểm thử:** Mô tả quá trình hiện thực mã nguồn, môi trường cài đặt và các kết quả kiểm thử hệ thống để đảm bảo tính đúng đắn của các chức năng.
- **Chương 8 - Đánh giá hệ thống:** Đánh giá tổng thể hiệu quả của hệ thống sau khi triển khai thực tế hoặc trên môi trường thử nghiệm.
- **Chương 9 - Tổng kết:** Tóm tắt các kết quả đạt được, đối chiếu với mục tiêu ban đầu, nêu rõ tính mới và đóng góp của đề tài, đồng thời đề xuất các hướng phát triển trong tương lai.

Ngoài ra, báo cáo còn bao gồm danh mục tài liệu tham khảo và các phụ lục liên quan đến mã nguồn và kết quả thực nghiệm chi tiết.

2 Kiến thức và công nghệ nền tảng

2.1 Tổng quan về phân tích tương quan giao thông đô thị

2.1.1 Đặc điểm giao thông đô thị và các thách thức

Giao thông đô thị tại các thành phố lớn thường mang tính phi tuyến, đa biến và chịu ảnh hưởng đồng thời của nhiều yếu tố tự nhiên – xã hội. Sự gia tăng mạnh mẽ của phương tiện cá nhân, hạ tầng hạn chế, cùng với tính khó đoán trong hành vi tham gia giao thông khiến việc phân tích và dự báo trở thành một bài toán phức tạp. Hơn nữa, trạng thái giao thông không chỉ bị chi phối tại chỗ mà còn lan truyền theo mạng lưới đường, dẫn đến hiện tượng "tắc nghẽn dây chuyền" (spillover congestion).

Đối với các hệ thống quản lý giao thông thông minh (ITS), việc hiểu rõ bản chất tương tác không gian–thời gian giữa các nút giao là cơ sở để thiết kế mô hình dự báo có độ chính xác cao, hỗ trợ lập kế hoạch và điều tiết giao thông hiệu quả.

2.1.1.a Đặc điểm giao thông tại TP.HCM

TP.HCM là đô thị có cơ cấu phương tiện đặc thù với tỷ lệ xe máy chiếm ưu thế tuyệt đối. Đặc điểm này hình thành một môi trường giao thông không đồng nhất, giàu biến động và khó chuẩn hóa. Một số đặc điểm quan trọng của giao thông TP.HCM bao gồm:

- **Xe máy chiếm hơn 90% tổng phương tiện:** tạo nên dòng di chuyển linh hoạt, không tuân thủ nghiêm ngặt làn đường.
- **Mật độ sở hữu xe máy cao (> 350 xe/1000 dân):** gây áp lực lớn lên hạ tầng đô thị.
- **Dòng xe hỗn hợp:** ô tô, xe tải, xe buýt, xe máy di chuyển xen kẽ, làm gia tăng độ bất định của lưu lượng.
- **Tác động mạnh từ môi trường xã hội:** thời tiết, sự kiện, ùn tắc cục bộ có thể làm biến đổi trạng thái giao thông toàn khu vực.

Những yếu tố này đòi hỏi mô hình phân tích phải đủ linh hoạt để phản ánh đúng tính động, tính phi tuyến và tính bất định của hệ thống giao thông đô thị.

2.1.1.b Hiện tượng lan truyền tắc nghẽn

Tắc nghẽn giao thông không chỉ xảy ra tại một điểm mà có khả năng lan truyền đến các đoạn đường lân cận. Một đoạn đường quá tải sẽ làm giảm tốc độ, tăng mật độ phương tiện, và gây ảnh hưởng lên khả năng lưu thông đầu vào – đầu ra của các nút láng giềng. Hiện tượng này có thể được mô hình hóa như một quá trình lan truyền:

$$\frac{dc(t)}{dt} = -\mu c(t) + \beta(k-1)c(t)[1-r(t)-c(t)] \quad (1)$$

Trong đó:

- $c(t)$: tỷ lệ đoạn đường đang tắc tại thời điểm t ,
- β : tốc độ lan truyền tắc nghẽn,
- μ : tốc độ hồi phục,
- k : bậc trung bình của nút trong mạng lưới,
- $r(t)$: tỷ lệ đoạn đường đã hồi phục.

Mô hình này cho phép mô tả cách tắc nghẽn di chuyển theo mạng lưới, từ đó hỗ trợ dự báo các hiện tượng "hiệu ứng domino" trong giao thông đô thị.

2.1.2 Phân tích tương quan không gian–thời gian

Hệ thống giao thông biểu hiện đồng thời hai loại phụ thuộc quan trọng:

1. **Phụ thuộc theo không gian**: trạng thái của một đoạn đường nằm dưới ảnh hưởng của các đoạn đường lân cận.
2. **Phụ thuộc theo thời gian**: trạng thái hiện tại liên quan mật thiết đến trạng thái của chính nó ở các thời điểm trước.

Sự kết hợp giữa hai chiều phụ thuộc này tạo nên tính chất "không gian–thời gian" (spatiotemporal), đóng vai trò then chốt trong các mô hình học sâu hiện đại.

2.1.2.a Tương quan không gian (Spatial Dependence)

Tương quan không gian phản ánh mức độ ảnh hưởng lẫn nhau giữa các nút giao hoặc đoạn đường. Khi một khu vực xảy ra ùn tắc, áp lực sẽ lan truyền theo các tuyến liên thông. Hệ số tương quan giữa hai đoạn đường i và j được xác định như sau:

$$r_{ij} = \frac{\sum_{d,t} (y_{itd} - \bar{y}_i) w_{ij} (y_{jtd} - \bar{y}_j)}{\sqrt{\sum_{d,t} (y_{itd} - \bar{y}_i)^2 \sum_{d,t} (y_{jtd} - \bar{y}_j)^2}} \quad (2)$$

Trong đó, trọng số w_{ij} biểu thị sự gần gũi không gian dựa trên:

- khoảng cách địa lý,
- mức độ kết nối trực tiếp giữa hai đoạn đường,
- thời gian di chuyển trung bình,
- hoặc cấu trúc của mạng lưới giao thông.

2.1.2.b Tương quan thời gian (Temporal Dependence)

Lưu lượng giao thông biến đổi theo các quy luật thời gian rõ rệt. Hai đặc điểm nổi bật bao gồm:

- **Tính chu kỳ:** Giao thông thường lặp lại theo các khung giờ cao điểm – thấp điểm hàng ngày (sáng, trưa, chiều) hoặc các mô hình theo ngày trong tuần.
- **Tính phụ thuộc thời điểm:** Lưu lượng tại thời điểm t chịu ảnh hưởng mạnh từ trạng thái của chính nó tại các thời điểm $t - 1, t - 2, \dots$

2.1.2.c Mô hình hóa tương quan động

Mối quan hệ giữa các đoạn đường không phải là bất biến. Chúng biến đổi theo thời gian do:

- thay đổi dòng xe,
- thời tiết, mưa, ngập,
- sự kiện đột xuất,
- tai nạn hoặc phong tỏa đường.

Do đó, ma trận quan hệ giữa các nút cần được mô hình hóa dưới dạng động:

$$A_t = f_{corr}(X_t, \theta) \quad (3)$$

Trong đó:

- A_t : ma trận kề đại diện cho cấu trúc không gian tại thời điểm t ,
- X_t : đặc trưng giao thông (tốc độ, lưu lượng, mật độ, độ trễ...),
- θ : tham số mô hình học được.

2.2 Lý thuyết đồ thị và Graph Neural Networks

2.2.1 Biểu diễn mạng giao thông dưới dạng đồ thị

2.2.1.a Định nghĩa đồ thị giao thông

Một mạng giao thông đô thị có thể được mô hình hoá dưới dạng đồ thị có hướng $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A})$, trong đó:

- $\mathcal{V} = \{v_1, v_2, \dots, v_N\}$ là tập N nút, mỗi nút đại diện cho một đoạn đường, giao lộ hoặc điểm đo cảm biến.
- $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ là tập các cạnh có hướng, phản ánh mối quan hệ kết nối hoặc luồng giao thông từ đoạn đường này sang đoạn đường khác.
- $\mathcal{A} \in \mathbb{R}^{N \times N}$ là ma trận kề, mã hoá mức độ liên kết giữa các nút.

Đối với bài toán dự báo giao thông, cách xây dựng ma trận kề $\mathcal{A}$ đóng vai trò đặc biệt quan trọng vì nó quyết định mô hình học các tương tác không gian như thế nào.

2.2.1.b Đặc trưng nút và cạnh

Ma trận đặc trưng của các nút tại thời điểm t được biểu diễn bởi $\mathbf{X}^{(t)} \in \mathbb{R}^{N \times F}$, trong đó:

- N là số lượng nút trong mạng.
- F là số chiều của không gian đặc trưng, ví dụ: tốc độ trung bình, mật độ xe, lưu lượng qua trạm, thời gian chờ,...
- $\mathbf{X}_{i,:}^{(t)}$ là vector đặc trưng của nút i tại thời điểm t .

Trong một số mô hình nâng cao, đặc trưng cạnh E_{ij} cũng có thể được bổ sung, ví dụ: chiều dài đoạn đường, số làn xe, tốc độ tối đa, hoặc dữ liệu cảm biến IoT đặt dọc tuyến.

2.2.2 Graph Convolutional Networks (GCN)

2.2.2.a Phép tích chập trên đồ thị

GCN được thiết kế để mở rộng phép tích chập từ không gian Euclidean (ảnh, âm thanh) sang cấu trúc đồ thị phi Euclidean. Công thức phổ biến của GCN (theo Kipf & Welling) là:

$$\mathbf{H}^{(l+1)} = \sigma \left(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} \mathbf{H}^{(l)} \mathbf{W}^{(l)} \right), \quad (4)$$

trong đó:

- $\tilde{A} = A + I_N$ bổ sung self-loop giúp mỗi nút sử dụng thêm thông tin của chính nó,
- $\tilde{D}$ là ma trận bậc tương ứng với $\tilde{A}$,
- $\mathbf{W}^{(l)}$ là trọng số của lớp thứ l ,
- σ là hàm kích hoạt phi tuyến.

GCN cho phép khuếch tán thông tin giữa các nút theo cấu trúc không gian của mạng giao thông, đóng vai trò nền tảng cho việc mô hình hóa tương quan không gian.

2.3 Mô hình Transformer và BERT

2.3.1 Kiến trúc Transformer

2.3.1.a Cơ chế Self-Attention

Transformer là kiến trúc học sâu dựa trên cơ chế attention, được đề xuất bởi Vaswani và cộng sự năm 2017. Khác với RNN/LSTM xử lý tuần tự, Transformer có thể xử lý toàn bộ chuỗi đầu vào song song thông qua cơ chế self-attention.

Cho một chuỗi đầu vào $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_T]$, self-attention tính toán như sau:

$$\mathbf{Q} = \mathbf{XW}^Q, \quad \mathbf{K} = \mathbf{XW}^K, \quad \mathbf{V} = \mathbf{XW}^V \quad (5)$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^T}{\sqrt{d_k}}\right) \mathbf{V} \quad (6)$$

Trong đó:

- $\mathbf{Q}$ (Query), $\mathbf{K}$ (Key), $\mathbf{V}$ (Value) là các phép chiếu tuyến tính của đầu vào
- d_k là chiều của không gian key, dùng để chuẩn hóa
- Softmax tạo ra phân phối xác suất thể hiện mức độ quan trọng của mỗi vị trí

Cơ chế này cho phép mỗi vị trí trong chuỗi "chú ý" đến tất cả các vị trí khác, nắm bắt được mối quan hệ phụ thuộc dài hạn mà không gặp vấn đề vanishing gradient như RNN.

2.3.1.b Multi-Head Attention

Để tăng khả năng biểu diễn, Transformer sử dụng nhiều attention heads song song:

$$\text{head}_i = \text{Attention}(\mathbf{QW}_i^Q, \mathbf{KW}_i^K, \mathbf{VW}_i^V) \quad (7)$$

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \mathbf{W}^O \quad (8)$$

Mỗi head học một khía cạnh khác nhau của mối quan hệ giữa các vị trí trong chuỗi.

2.3.1.c Positional Encoding

Vì Transformer xử lý song song, nó không có thông tin về thứ tự trong chuỗi. Do đó, positional encoding được thêm vào:

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (9)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right) \quad (10)$$

Điều này giúp mô hình phân biệt được vị trí tương đối và tuyệt đối của các phần tử trong chuỗi.

2.3.2 BERT: Bidirectional Encoder Representations from Transformers

2.3.2.a Kiến trúc BERT

BERT là mô hình pre-training dựa trên Transformer encoder, được giới thiệu bởi Devlin và cộng sự (Google AI) năm 2018. Khác với các mô hình ngôn ngữ truyền thống chỉ đọc văn bản theo một chiều (trái sang phải hoặc phải sang trái), BERT đọc theo cả hai chiều (bidirectional) để hiểu ngữ cảnh đầy đủ.

Cấu trúc cơ bản:

- BERT-Base: 12 lớp Transformer, 768 hidden units, 12 attention heads, 110M parameters
- BERT-Large: 24 lớp Transformer, 1024 hidden units, 16 attention heads, 340M parameters

Đầu vào của BERT là một chuỗi token được mã hóa thành:

$$\mathbf{E} = \mathbf{E}_{token} + \mathbf{E}_{segment} + \mathbf{E}_{position} \quad (11)$$

Trong đó:

- $\mathbf{E}_{token}$: embedding của token
- $\mathbf{E}_{segment}$: embedding phân biệt các câu khác nhau
- $\mathbf{E}_{position}$: positional encoding

2.3.2.b Chiến lược Pre-training của BERT

BERT sử dụng hai nhiệm vụ pre-training:

1. Masked Language Model (MLM):

Ngẫu nhiên che (mask) 15% tokens trong chuỗi đầu vào và huấn luyện mô hình dự đoán các token bị che:

$$\mathcal{L}_{MLM} = - \sum_{i \in \mathcal{M}} \log P(x_i | \mathbf{x}_{\setminus \mathcal{M}}) \quad (12)$$

Trong đó $\mathcal{M}$ là tập các vị trí bị mask.

2. Next Sentence Prediction (NSP):

Dự đoán xem câu B có phải là câu tiếp theo của câu A hay không:

$$\mathcal{L}_{NSP} = - \log P(IsNext|[CLS], \text{SentA}, \text{SentB}) \quad (13)$$

Token đặc biệt [CLS] được thêm vào đầu mỗi chuỗi và đại diện cho toàn bộ chuỗi trong các tác vụ phân loại.

2.3.2.c Fine-tuning BERT

Sau pre-training, BERT có thể được fine-tune cho các tác vụ downstream cụ thể bằng cách:

- Thêm một lớp output phù hợp (classification, regression, sequence labeling...)
- Huấn luyện end-to-end trên dữ liệu của tác vụ đích
- Sử dụng learning rate nhỏ để tránh "catastrophic forgetting"

2.3.3 Ưu điểm của BERT trong phân tích chuỗi thời gian

1. **Học biểu diễn ngữ cảnh hai chiều:** BERT xem xét cả thông tin quá khứ và tương lai, phù hợp với việc phân tích mối quan hệ phức tạp trong dữ liệu giao thông.
2. **Khả năng xử lý song song:** Khác với RNN/LSTM xử lý tuần tự, BERT có thể xử lý toàn bộ chuỗi đầu vào đồng thời, tăng tốc độ huấn luyện và inference.

3. **Nắm bắt phụ thuộc dài hạn:** Self-attention cho phép mô hình kết nối trực tiếp các vị trí xa nhau trong chuỗi, vượt trội hơn LSTM trong việc học các mẫu chu kỳ dài.
4. **Pre-training và transfer learning:** BERT có thể được pre-train trên dữ liệu lớn và fine-tune trên tác vụ cụ thể với dữ liệu hạn chế.
5. **Khả năng tích hợp với Graph:** BERT có thể kết hợp với GNN để tạo nên các mô hình hybrid mạnh mẽ cho dữ liệu đồ thị có tính thời gian.

2.3.4 Áp dụng BERT cho dữ liệu giao thông

2.3.4.a Tokenization cho chuỗi thời gian

Trong bài toán giao thông, mỗi "token" có thể là:

- Một bước thời gian (time step) với vector đặc trưng tương ứng
- Một cửa sổ thời gian được tổng hợp (sliding window)
- Một đoạn đường tại một thời điểm cụ thể

Vector đặc trưng cho mỗi token có thể bao gồm: tốc độ, mật độ, lưu lượng, thời tiết, ngày trong tuần, v.v.

2.3.4.b Spatial-Temporal BERT

Để áp dụng BERT vào dữ liệu giao thông có tính không gian-thời gian, ta cần:

1. Encoding không gian:

$$\mathbf{E}_{spatial}(v_i) = \text{Learnable_Embedding}(i) \quad \text{hoặc} \quad \text{GCN}(\mathbf{A}, \mathbf{X}_i) \quad (14)$$

2. Encoding thời gian:

$$\mathbf{E}_{temporal}(t) = \mathbf{E}_{position}(t) + \mathbf{E}_{hour}(t) + \mathbf{E}_{day}(t) + \mathbf{E}_{week}(t) \quad (15)$$

3. Kết hợp:

$$\mathbf{E}(v_i, t) = \mathbf{X}_{i,t} + \mathbf{E}_{spatial}(v_i) + \mathbf{E}_{temporal}(t) \quad (16)$$

2.3.4.c BERT-GNN Hybrid Model

Mô hình kết hợp có thể được thiết kế theo các cách:

Phương án 1: GNN trước, BERT sau

$$\mathbf{H}_t^{spatial} = \text{GCN}(\mathbf{A}, \mathbf{X}_t) \quad (17)$$

$$\mathbf{H}^{temporal} = \text{BERT}([\mathbf{H}_1^{spatial}, \dots, \mathbf{H}_T^{spatial}]) \quad (18)$$

Phương án 2: BERT trước, GNN sau

$$\mathbf{H}_i^{temporal} = \text{BERT}([\mathbf{X}_{i,1}, \dots, \mathbf{X}_{i,T}]) \quad (19)$$

$$\mathbf{H}^{output} = \text{GCN}(\mathbf{A}, [\mathbf{H}_1^{temporal}, \dots, \mathbf{H}_N^{temporal}]) \quad (20)$$

Phương án 3: Xen kẽ GNN và BERT

$$\mathbf{H}^{(l+1)} = \text{BERT}(\text{GCN}(\mathbf{H}^{(l)})) \quad (21)$$

2.3.5 So sánh BERT với các phương pháp khác

Bảng 1: So sánh BERT với RNN/LSTM/GRU

Tiêu chí	RNN/LSTM/GRU	BERT
Xử lý tuần tự	Có	Không (song song)
Tốc độ huấn luyện	Chậm	Nhanh
Phụ thuộc dài hạn	Khó (LSTM/GRU tốt hơn RNN)	Rất tốt
Ngữ cảnh hai chiều	Không (trừ Bi-LSTM)	Có
Số tham số	Ít đến trung bình	Nhiều
Yêu cầu dữ liệu	Thấp đến trung bình	Cao (pre-training)
Transfer learning	Khó	Dễ dàng
Khả năng diễn giải	Tương đối tốt	Khó hơn

2.4 Mô hình đề xuất: Traffic-BERT

2.4.1 Kiến trúc tổng thể

Dựa trên nền tảng BERT và các đặc điểm của dữ liệu giao thông, nghiên cứu này đề xuất mô hình **Traffic-BERT** gồm các thành phần chính:

1. **Spatial Encoder (GCN):** Trích xuất đặc trưng không gian từ cấu trúc mạng lưới
2. **Temporal Encoder (BERT):** Học mối quan hệ thời gian hai chi

2.5 Công nghệ nền tảng và framework

2.5.1 Python và hệ sinh thái Deep Learning

2.5.1.a Python trong phân tích dữ liệu và AI

Python đã trở thành ngôn ngữ lập trình tiêu chuẩn cho các ứng dụng Machine Learning và Deep Learning nhờ tính đơn giản, dễ học và hệ sinh thái thư viện phong phú. Đối với bài toán dự báo giao thông, Python cung cấp đầy đủ công cụ từ xử lý dữ liệu, huấn luyện mô hình đến triển khai sản phẩm.

Các ưu điểm chính bao gồm cú pháp rõ ràng giúp code dễ đọc và bảo trì, thư viện phong phú như NumPy, Pandas, Scikit-learn, PyTorch, TensorFlow, cộng đồng lớn hỗ trợ tốt, và khả năng tích hợp với nhiều hệ thống và database khác nhau.

2.5.1.b PyTorch và PyTorch Geometric

PyTorch Core

PyTorch là framework deep learning được phát triển bởi Meta AI, nổi bật với cơ chế dynamic computational graph cho phép debug và thử nghiệm linh hoạt. PyTorch cung cấp các thành phần cốt lõi như tensor operations với hỗ trợ GPU, module system để xây dựng mạng neural network, các thuật toán tối ưu (Adam, SGD, RMSprop), và cơ chế automatic differentiation cho backpropagation.

Kiến trúc của PyTorch được thiết kế theo nguyên tắc modular, cho phép research scientists dễ dàng thử nghiệm các ý tưởng mới trong khi vẫn đảm bảo hiệu năng cao khi triển khai production.

PyTorch Geometric (PyG)

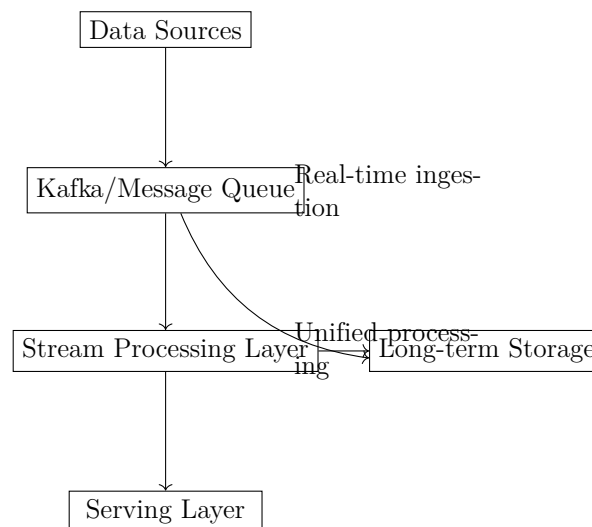
PyTorch Geometric là thư viện mở rộng chuyên biệt cho Graph Neural Networks, cung cấp các cấu trúc dữ liệu tối ưu cho đồ thị, message passing framework để truyền thông điệp giữa các nút, các lớp GNN được cài đặt sẵn (GCNConv, GATConv, SAGEConv, GraphConv), cơ chế xử lý batch cho nhiều đồ thị đồng thời, và các tiện ích như sampling, pooling, normalization.

PyG đặc biệt phù hợp cho bài toán giao thông vì có khả năng biểu diễn tự nhiên mạng lưới đường, hiệu năng cao nhờ sparse matrix operations, dễ dàng thử nghiệm các kiến trúc GNN khác nhau, và hỗ trợ GPU acceleration cho đồ thị lớn.

2.5.2 Kiến trúc Kappa cho xử lý dữ liệu streaming

2.5.2.a Tổng quan về Kappa Architecture

Kiến trúc Kappa là một mô hình xử lý dữ liệu hiện đại được đề xuất bởi Jay Kreps (LinkedIn) như một giải pháp đơn giản hóa so với Lambda Architecture truyền thống. Khác với Lambda Architecture có hai đường xử lý riêng biệt (batch và stream), Kappa Architecture chỉ sử dụng một đường xử lý streaming duy nhất, giảm độ phức tạp và chi phí bảo trì.



Hình 1: Kiến trúc Kappa

Các thành phần chính:

- (a) **Message Queue (Kafka):** Nhận và lưu trữ tất cả dữ liệu đầu vào dưới dạng event log bất biến. Kafka đóng vai trò là single source of truth, cho phép replay dữ liệu khi cần thiết.
- (b) **Stream Processing Layer:** Xử lý dữ liệu theo thời gian thực, thực hiện transformations, aggregations, và tính toán. Tất cả logic xử lý đều nằm trong lớp này.
- (c) **Serving Layer:** Lưu trữ kết quả đã được xử lý, phục vụ các truy vấn từ applications và users.
- (d) **Long-term Storage:** Lưu trữ dữ liệu lịch sử cho mục đích phân tích, audit, và reprocessing.

2.5.2.b Apache Kafka làm backbone

Apache Kafka là distributed streaming platform đóng vai trò trung tâm trong Kappa Architecture. Kafka được thiết kế như một commit log phân tán, nơi mọi message

được lưu trữ theo thứ tự và có thể được đọc lại nhiều lần.

Các khái niệm cốt lõi:

- **Topics:** Danh mục logic để phân loại messages. Ví dụ: traffic_speed, traffic_volume, traffic_incidents.
- **Partitions:** Mỗi topic được chia thành nhiều partitions để tăng khả năng song song hóa và throughput.
- **Producers:** Các services ghi dữ liệu vào Kafka topics.
- **Consumers:** Các services đọc và xử lý dữ liệu từ topics.
- **Consumer Groups:** Nhóm consumers làm việc cùng nhau để xử lý song song dữ liệu từ một topic.

Ưu điểm của Kafka trong hệ thống giao thông:

- **High Throughput:** Xử lý hàng triệu messages/giây từ các cảm biến giao thông
- **Durability:** Messages được replicate và persist, đảm bảo không mất dữ liệu
- **Scalability:** Dễ dàng scale horizontal bằng cách thêm brokers và partitions
- **Replay Capability:** Có thể đọc lại dữ liệu lịch sử để recompute hoặc fix bugs
- **Low Latency:** Độ trễ chỉ vài milliseconds, phù hợp cho real-time applications

2.5.2.c Apache Flink cho Stream Processing

Apache Flink là distributed stream processing framework được thiết kế đặc biệt cho xử lý streaming với độ trễ thấp và khả năng xử lý stateful phức tạp.

Đặc điểm nổi bật:

- (a) **True Streaming:** Khác với Spark Streaming (micro-batch), Flink xử lý từng event một, đạt độ trễ thấp hơn.
- (b) **Event Time Processing:** Xử lý dựa trên thời gian event xảy ra thực tế, không phải thời gian xử lý. Điều này quan trọng khi dữ liệu đến không đúng thứ tự.
- (c) **Stateful Computations:** Flink quản lý state một cách hiệu quả, cho phép thực hiện các phép tính phức tạp như windowing, aggregations, pattern detection.

- (d) **Exactly-Once Semantics**: Đảm bảo mỗi event được xử lý đúng một lần, ngay cả khi có failures.
- (e) **Fault Tolerance**: Cơ chế checkpointing tự động để recovery khi có lỗi.

Windowing trong Flink:

Flink hỗ trợ nhiều loại windows để tổng hợp dữ liệu streaming:

- **Tumbling Windows**: Các window không chồng lấp, kích thước cố định (ví dụ: 5 phút)
- **Sliding Windows**: Các window chồng lấp, phù hợp cho moving averages
- **Session Windows**: Window dựa trên khoảng idle time giữa các events
- **Global Windows**: Window toàn cục với custom triggers

2.5.2.d So sánh Kappa với Lambda Architecture

Bảng 2: So sánh Kappa và Lambda Architecture

Tiêu chí	Lambda Architecture	Kappa Architecture
Batch Layer	Có (MapReduce/Spark)	Không có
Speed Layer	Có (Stream Processing)	Stream processing duy nhất
Độ phức tạp	Cao (2 codebases riêng)	Thấp (1 codebase)
Độ trễ	Cao hơn do batch	Thấp, real-time
Consistency	Eventual consistency	Easier to maintain
Reprocessing	Cần chạy lại batch job	Replay từ Kafka
Maintenance	Khó (sync 2 systems)	Dễ hơn
Use case phù hợp	Cần cả batch và stream	Streaming-first applications

2.5.2.e Ứng dụng Kappa trong hệ thống giao thông

Trong bài toán phân tích giao thông, Kappa Architecture phù hợp vì:

- (a) **Tính real-time**: Dữ liệu giao thông cần được xử lý ngay lập tức để cảnh báo kịp thời.
- (b) **Data replay**: Khi cần retrain model hoặc fix bugs, có thể replay dữ liệu từ Kafka mà không cần maintain batch pipeline riêng.
- (c) **Unified logic**: Cùng một đoạn code xử lý được sử dụng cho cả real-time và historical analysis.
- (d) **Event-driven**: Giao thông là event-driven system (xe đi qua cảm biến, tắc nghẽn xảy ra, tai nạn), phù hợp với stream processing.

Pipeline xử lý trong hệ thống:

- (a) Cảm biến giao thông (camera, loop detectors, GPS) publish data vào Kafka topics
- (b) Flink consumers đọc từ Kafka, thực hiện:
 - Data cleaning và validation
 - Feature extraction (tốc độ, volume, occupancy)
 - Windowed aggregations (5-min, 15-min averages)
 - Anomaly detection
 - Correlation analysis
- (c) Kết quả được ghi vào:
 - Redis (serving layer cho low-latency queries)
 - PostgreSQL (structured data và spatial queries)
 - Time-series database (historical analysis)
- (d) ML models subscribe processed data để thực hiện predictions
- (e) Results được publish lại vào Kafka để downstream services consume

2.5.3 TomTom Historical Traffic Stats API

TomTom cung cấp dữ liệu giao thông lịch sử với độ bao phủ cao, bao gồm tốc độ trung bình, mức độ tắc nghẽn và chỉ số độ trễ theo từng đoạn đường. Dữ liệu được tổng hợp từ nhiều nguồn GPS và cập nhật theo khoảng thời gian từ 1–5 phút.

2.5.3.1 Ưu điểm

- Dữ liệu lịch sử dài hạn, hỗ trợ phân tích xu hướng theo giờ, ngày và tuần.
- Độ bao phủ rộng giúp bù đắp cho các khu vực thiếu cảm biến tại địa phương.
- Dễ dàng tích hợp vào các mô hình học chuỗi thời gian và mô hình mạng lưới.

2.5.3.2 Hạn chế

- Không phản ánh nguyên nhân của tắc nghẽn, chỉ mô tả hiện tượng.
- Chi phí sử dụng có thể cao nếu mở rộng cho toàn thành phố.

2.5.3.3 Vai trò trong đề tài Dữ liệu lịch sử từ TomTom giúp:

- Huấn luyện mô hình dự báo giao thông.
- Phân tích lan truyền tắc nghẽn giữa các đoạn đường.
- Xây dựng dữ liệu đầu vào cho mô hình học sâu theo dạng đồ thị.

2.5.4 Cơ sở dữ liệu và lưu trữ

2.5.4.a PostgreSQL với PostGIS

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở mạnh mẽ, nổi tiếng với tính ổn định, tuân thủ chuẩn ACID, và khả năng mở rộng thông qua extensions. PostGIS là extension quan trọng bổ sung khả năng xử lý dữ liệu không gian địa lý (spatial data).

Tính năng quan trọng cho giao thông:

- **Spatial Indexing:** R-tree và GiST index giúp tăng tốc độ truy vấn không gian lên hàng nghìn lần
- **Geometric Operations:** Tính toán khoảng cách, giao điểm, buffer zones, containment
- **Topology:** Phân tích mạng lưới đường với topology rules

- **Routing Extensions:** pgRouting hỗ trợ tìm đường đi ngắn nhất, TSP, isochrones
- **Geographic Coordinate System:** Hỗ trợ nhiều hệ tọa độ, transformations

PostGIS cho phép lưu trữ và truy vấn hiệu quả các thực thể giao thông như nodes, edges, polygons (khu vực), và thực hiện các phân tích không gian phức tạp như tìm các nút trong bán kính, tính toán shortest path, phân tích clustering.

2.5.4.b TimescaleDB

TimescaleDB là extension của PostgreSQL được tối ưu hóa đặc biệt cho time-series data. Nó kế thừa toàn bộ tính năng của PostgreSQL nhưng thêm các khả năng chuyên biệt cho dữ liệu chuỗi thời gian.

Tính năng chính:

- **Automatic Partitioning:** Tự động chia dữ liệu thành chunks theo thời gian
- **Compression:** Nén dữ liệu lịch sử với tỷ lệ cao (10-20x), tiết kiệm storage
- **Continuous Aggregates:** Materialized views tự động cập nhật cho aggregations
- **Retention Policies:** Tự động xóa dữ liệu cũ theo policy định sẵn
- **Time-based Queries:** Tối ưu hóa queries theo time range

Đối với dữ liệu giao thông với millions of readings mỗi ngày, TimescaleDB cung cấp performance tốt hơn traditional RDBMS khi truy vấn theo time windows, đồng thời vẫn giữ được khả năng JOIN với spatial data trong PostGIS.

2.5.4.c Redis

Redis là in-memory data store hoạt động như cache layer và message broker với hiệu năng cực cao (sub-millisecond latency).

Vai trò trong hệ thống giao thông:

(a) **Caching Layer:**

- Cache kết quả dự đoán cho các queries phổ biến
- Cache ma trận tương quan giữa các nodes
- Session storage cho web applications

(b) **Real-time Analytics:**

- Counters cho traffic volume
- Leaderboards (busiest intersections)
- HyperLogLog cho cardinality estimation

(c) **Pub/Sub Messaging:**

- Broadcast traffic alerts đến multiple clients
- Real-time notifications cho congestion events
- WebSocket message distribution

(d) **Rate Limiting:**

- Throttle API requests
- Control sensor data ingestion rate

Redis hỗ trợ nhiều data structures (strings, hashes, lists, sets, sorted sets, streams) phù hợp cho các use cases khác nhau trong hệ thống giao thông.

2.5.4.d Chiến lược lưu trữ phân tầng

Hệ thống giao thông cần chiến lược lưu trữ đa tầng để cân bằng giữa performance, cost, và retention requirements:

Bảng 3: Chiến lược lưu trữ phân tầng

Tầng	Technology	Data Age	Use Case
Hot	Redis	< 1 hour	Real-time queries
Warm	TimescaleDB	1 hour - 30 days	Recent analysis
Cold	PostgreSQL + S3	> 30 days	Historical analysis
Archive	S3 Glacier	> 1 year	Compliance, audit

2.5.5 API và Web Development Stack

2.5.5.a FastAPI

FastAPI là modern web framework cho Python, được thiết kế để xây dựng APIs với hiệu năng cao và developer experience tốt.

Ưu điểm chính:

- **High Performance:** Tốc độ ngang ngửa NodeJS và Go nhờ Starlette và Pydantic
- **Type Safety:** Sử dụng Python type hints cho automatic validation
- **Async Support:** Native async/await cho concurrent requests
- **Auto Documentation:** Tự động generate OpenAPI (Swagger) và ReDoc

- **Dependency Injection:** Clean code architecture với DI system
- **WebSocket Support:** Real-time communication cho traffic updates

FastAPI đặc biệt phù hợp cho các API cần xử lý nhiều concurrent connections từ sensors và clients, đồng thời cần response time thấp cho real-time applications.

2.5.5.b React và Modern Frontend

React là thư viện JavaScript phổ biến nhất để xây dựng user interfaces, đặc biệt là single-page applications (SPA). React sử dụng component-based architecture, cho phép tái sử dụng code và quản lý state hiệu quả.

Các thư viện quan trọng trong hệ thống giao thông:

(a) **Mapping Libraries:**

- **Leaflet / React-Leaflet:** Open-source, lightweight, mobile-friendly
- **Mapbox GL JS:** High-performance vector maps với 3D support
- **Deck.gl:** WebGL-based visualization cho big data

(b) **Data Visualization:**

- **D3.js:** Low-level, flexible cho custom visualizations
- **Recharts:** Composable chart library built on D3
- **Plotly.js:** Interactive scientific charts
- **ECharts:** Rich visualization library từ Apache

(c) **State Management:**

- **Redux Toolkit:** Centralized state management
- **React Query:** Server state management và caching
- **Zustand:** Lightweight alternative cho simple cases

(d) **Real-time Communication:**

- **Socket.IO:** Bidirectional event-based communication
- **WebSocket API:** Native browser WebSocket support

UI Component Libraries:

Material-UI, Ant Design, và Chakra UI cung cấp pre-built components tuân theo design systems, giúp đẩy nhanh development và đảm bảo consistency trong UI/UX.

2.5.6 Containerization và Orchestration

2.5.6.a Docker

Docker là platform containerization cho phép đóng gói applications và dependencies thành containers độc lập, nhất quán trên mọi môi trường từ development đến production.

Lợi ích của Docker:

- **Consistency:** "Works on my machine" problem được giải quyết
- **Isolation:** Mỗi service chạy trong môi trường riêng
- **Portability:** Deploy trên bất kỳ platform nào hỗ trợ Docker
- **Resource Efficiency:** Lightweight hơn VMs, chia sẻ kernel
- **Versioning:** Image versioning cho rollback dễ dàng

Docker Compose quản lý multi-container applications, định nghĩa toàn bộ stack (API, databases, cache, message queue) trong một file YAML, cho phép start/stop toàn bộ hệ thống với một command.

2.5.6.b Kubernetes

Kubernetes (K8s) là container orchestration platform mã nguồn mở, tự động hóa việc deploy, scale, và manage containerized applications trong môi trường production.

Khái niệm cốt lõi:

- **Pods:** Đơn vị nhỏ nhất, chứa một hoặc nhiều containers
- **ReplicaSets:** Đảm bảo số lượng pod replicas mong muốn
- **Deployments:** Quản lý rolling updates và rollbacks
- **Services:** Expose pods qua stable network endpoint
- **Ingress:** HTTP routing và load balancing
- **ConfigMaps và Secrets:** Quản lý configuration và sensitive data
- **Persistent Volumes:** Storage abstraction cho stateful apps
- **Namespaces:** Logical isolation giữa các môi trường

Tính năng quan trọng cho production:

- (a) **Auto-scaling:** Horizontal Pod Autoscaler (HPA) tự động scale dựa trên CPU, memory, hoặc custom metrics
- (b) **Self-healing:** Tự động restart failed containers, replace và reschedule pods khi nodes die
- (c) **Service Discovery và Load Balancing:** Tự động distribute traffic đến healthy pods
- (d) **Rolling Updates:** Zero-downtime deployments với progressive rollout
- (e) **Resource Management:** CPU và memory requests/limits để optimize cluster utilization

Với hệ thống giao thông cần high availability và khả năng scale dynamic theo traffic load, Kubernetes cung cấp infrastructure layer mạnh mẽ để đảm bảo reliability và performance.

2.5.7 Monitoring và Observability

2.5.7.a Prometheus và Grafana

Prometheus là monitoring system và time-series database, thu thập metrics từ các services thông qua pull model. Prometheus lưu trữ metrics với timestamps và labels, hỗ trợ PromQL query language mạnh mẽ cho aggregations và alerting.

Grafana là visualization platform, kết nối với Prometheus và nhiều data sources khác để tạo dashboards tương tác. Grafana cung cấp real-time monitoring, alerting rules, và khả năng tạo custom visualizations.

Metrics quan trọng trong hệ thống giao thông:

- System metrics: CPU, memory, disk I/O, network
- Application metrics: API latency, throughput, error rates
- Business metrics: Prediction accuracy, correlation detection rate
- Kafka metrics: Consumer lag, message rate, partition distribution
- Flink metrics: Backpressure, checkpoint duration, task utilization

2.5.7.b ELK Stack

ELK (Elasticsearch, Logstash, Kibana) stack cung cấp giải pháp log management và analysis:

- **Elasticsearch:** Distributed search và analytics engine

- **Logstash:** Data processing pipeline cho log ingestion
- **Kibana:** Visualization platform cho logs và metrics

Log aggregation giúp troubleshoot issues, analyze user behavior, audit system activities, và detect anomalies trong hệ thống phân tán.

2.6 Kết chương

Chương 2 đã trình bày một cách toàn diện nền tảng lý thuyết và công nghệ cần thiết để xây dựng hệ thống phân tích mối liên hệ giao thông đô thị. Những kiến thức này tạo thành bộ công cụ quan trọng phục vụ cho các chương tiếp theo của đồ án.

Về mặt lý thuyết, chúng ta đã nắm được:

- Đặc điểm phức tạp của giao thông đô thị TP.HCM với cơ cấu phương tiện đặc thù, tỷ lệ xe máy chiếm ưu thế tuyệt đối (>90%), và hiện tượng lan truyền tắc nghẽn theo mạng lưới đường. Việc hiểu rõ bản chất này giúp thiết kế mô hình phù hợp với đặc thù địa phương.
- Cơ sở toán học vững chắc về biểu diễn mạng giao thông dưới dạng đồ thị $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{A})$, các phép biến đổi trên cấu trúc đồ thị, và cách mô hình hóa tương quan không gian-thời gian.
- Các kiến trúc Graph Neural Networks tiên tiến như GCN và GAT, cho phép học các mối quan hệ phức tạp giữa các nút giao thông mà không cần định nghĩa thủ công. Cơ chế message passing và attention mechanism là nền tảng để mô hình tự động phát hiện các mối tương quan ẩn.
- Các mô hình chuỗi thời gian từ RNN cơ bản đến LSTM và GRU, giúp nắm bắt phụ thuộc temporal với khả năng học long-term dependencies. Việc hiểu rõ ưu nhược điểm của từng loại giúp lựa chọn kiến trúc phù hợp cho bài toán dự báo đa bước (multi-step prediction).

Về mặt công nghệ, chương này đã xác định được bộ khung kỹ thuật (Technology Stack) hiện đại và mạnh mẽ để hiện thực hóa hệ thống:

- **Hệ sinh thái Python & PyTorch**: Cung cấp môi trường phát triển linh hoạt, đặc biệt là thư viện PyTorch Geometric chuyên dụng cho các mô hình GNN, giúp đơn giản hóa việc cài đặt các thuật toán phức tạp trên dữ liệu đồ thị.
- **Kiến trúc Kappa & Apache Kafka**: Đảm bảo khả năng xử lý dữ liệu streaming thời gian thực với độ trễ thấp và độ tin cậy cao, phù hợp với đặc thù dữ liệu liên tục từ các cảm biến giao thông.
- **Cơ sở dữ liệu không gian - thời gian**: Sự kết hợp giữa PostgreSQL (PostGIS) và TimescaleDB giải quyết bài toán lưu trữ và truy vấn hiệu quả cho dữ liệu vừa có thuộc tính vị trí địa lý, vừa có tính chất chuỗi thời gian.

- **Hạ tầng triển khai:** Việc sử dụng Docker và Kubernetes đảm bảo hệ thống có khả năng mở rộng (scalability), dễ dàng triển khai và vận hành ổn định trong môi trường thực tế.

Sự kết hợp chặt chẽ giữa các nền tảng lý thuyết vững chắc và các công nghệ tiên tiến này là tiền đề quan trọng để bước sang Chương 3 – nơi chúng ta sẽ khảo sát các công trình liên quan để định vị rõ hơn đóng góp của đề tài, và Chương 4 – nơi mô hình hệ thống cụ thể sẽ được đề xuất.

3 Công trình liên quan

Chương này trình bày tổng quan một cách có hệ thống về các nghiên cứu khoa học và hệ thống thực tế liên quan trực tiếp đến bài toán phân tích và dự báo giao thông đô thị. Nội dung được triển khai theo ba hướng chính: (1) tổng hợp các mô hình nghiên cứu từ truyền thống đến hiện đại; (2) phân tích các hệ thống lớn đang được vận hành trong thực tiễn; và (3) xác định các khoảng trống khoa học (gap) mà đề tài hướng đến giải quyết. Cách tiếp cận toàn diện này giúp đặt đề tài vào đúng bối cảnh phát triển của lĩnh vực, đồng thời làm rõ tính mới và đóng góp kỳ vọng của nghiên cứu.

3.1 Các nghiên cứu liên quan

3.1.1 Mô hình hóa sự lan truyền và tắc nghẽn giao thông

Các nghiên cứu nền tảng trong lĩnh vực giao thông trước thời kỳ học sâu chủ yếu dựa vào mô hình vật lý và thống kê nhằm mô tả bản chất lan truyền của dòng chảy giao thông. Những mô hình này giải thích cách tắc nghẽn hình thành, khuếch tán theo thời gian và ảnh hưởng lên các khu vực lân cận.

3.1.1.a Mô hình lan truyền sóng (Wave Propagation Models)

Jiang và cộng sự nghiên cứu dữ liệu cảm biến tại Bắc Kinh và Thâm Quyển, qua đó xác định các *jam centers*. Nhóm tác giả chỉ ra rằng tắc nghẽn lan truyền theo dạng sóng với vận tốc xấp xỉ 30 ± 10 km/h và cường độ suy giảm tuyến tính theo khoảng cách. Nghiên cứu cung cấp cơ sở thực nghiệm quan trọng giúp mô hình hóa tính chất không gian của tắc nghẽn.

3.1.1.b Mô hình lây lan (Contagion Process Models)

Saberi và cộng sự tiếp cận theo hướng mô phỏng tắc nghẽn giao thông như một dạng bệnh dịch lây lan. Sử dụng mô hình SIR (Susceptible–Infected–Recovered), họ mô tả trạng thái “nhiễm tắc nghẽn” và “hồi phục” của các đoạn đường theo thời gian. Mô hình này hiệu quả ở cấp độ vĩ mô nhưng thiếu khả năng mô tả đặc trưng vi mô tại từng nút giao thông.

3.1.1.c Xác định nút thắt cổ chai (Bottleneck Detection)

Li và cộng sự đề xuất phương pháp dựa trên lý thuyết đồ thị và chuỗi Markov nhằm xác định các nút thắt cổ chai dựa trên “chi phí tắc nghẽn”. Chi phí này phản ánh

không chỉ mức độ tắc tại một điểm mà còn khả năng lan truyền sang các đoạn khác. Đây là tư tưởng quan trọng được kế thừa trong đề tài khi đánh giá tác động liên vùng của các nút giao thông.

3.1.2 Các mô hình Deep Learning với Ma trận kề tĩnh (Static Graph)

Sự phát triển của Graph Convolutional Networks (GCN) mở ra phương pháp khai thác cấu trúc topo của mạng lưới giao thông để phục vụ dự báo.

3.1.2.a T-GCN

Mô hình T-GCN kết hợp GCN và GRU để xử lý đồng thời thông tin không gian–thời gian.

- **Ưu điểm:** T-GCN vượt trội hơn các mô hình truyền thống như ARIMA hay SVR nhờ học trực tiếp đặc trưng topo của mạng lưới.
- **Hạn chế:** Ma trận kề A được xây dựng tĩnh dựa trên khoảng cách hoặc kết nối vật lý, không phản ánh tính động của mối quan hệ giao thông theo thời gian.

3.1.2.b ST-GCN, DCRNN và các biến thể

Các mô hình ST-GCN và DCRNN xử lý tốt hơn cơ chế khuếch tán, nhưng vẫn dựa trên đồ thị tĩnh đã được xác định trước. Điều này làm giảm hiệu quả trong bối cảnh giao thông phức tạp khi các mối quan hệ có thể thay đổi theo giờ cao điểm, sự cố, hoặc các yếu tố ngẫu nhiên khác.

3.1.3 Các mô hình Deep Learning với Ma trận kề động (Dynamic Graph Learning)

Nhằm khắc phục hạn chế của đồ thị tĩnh, các nghiên cứu gần đây tập trung vào xây dựng ma trận kề động, cập nhật theo từng thời điểm dựa trên dữ liệu thực nghiệm.

3.1.3.a DTC-STGCN

Xu và cộng sự đề xuất mô hình DTC-STGCN, trong đó ma trận kề được tính toán động dựa trên tương quan giữa các nút qua từng thời điểm. Kết quả cho thấy độ chính xác dự báo tăng đáng kể nhờ mô hình hóa đúng bản chất thay đổi của mạng lưới giao thông.

3.1.3.b Attention và Graph Attention Networks (GAT)

Cơ chế Attention giúp mô hình tự động học trọng số giữa các nút mà không cần ma trận kề định sẵn. Điều này đặc biệt phù hợp với bối cảnh TP.HCM, nơi tồn tại nhiều *kết nối ngữ nghĩa* (semantic connections) không thể hiện bằng kết nối vật lý, ví dụ các nút giao thông xa nhau nhưng cùng nằm trên một luồng di chuyển chủ đạo.

3.2 Các hệ thống và nền tảng thực tế

3.2.1 Google Maps và Waze

Hai hệ thống này sử dụng dữ liệu khổng lồ từ cộng đồng, áp dụng các mô hình AI độ chính xác cao để dự báo thời gian thực. Tuy nhiên, chúng hoạt động như *hộp đen*:

- Không cung cấp khả năng phân tích nguyên nhân tắc nghẽn.
- Không hỗ trợ đánh giá mức độ lan truyền giữa các nút.
- Không phù hợp cho mục tiêu tối ưu hoá quản lý giao thông cấp thành phố.

3.2.2 TomTom, Mapbox và các nền tảng bản đồ thương mại khác

Bên cạnh Google Maps và Waze, còn có nhiều nền tảng bản đồ thương mại và dịch vụ dữ liệu giao thông chuyên sâu đóng vai trò quan trọng trong hệ sinh thái giao thông thông minh, tiêu biểu là TomTom, Mapbox, HERE, và INRIX. Các nền tảng này cung cấp tập hợp dịch vụ, API và dữ liệu có thể hỗ trợ cả ứng dụng điều hướng thương mại lẫn phân tích chuyên sâu cho nhà quản lý giao thông.

3.2.2.1 TomTom

- **Tính năng:** Cung cấp routing, traffic flow, traffic incidents, historical traffic, và map tiles. Hệ thống có dữ liệu thời gian thực và dữ liệu lịch sử được chuẩn hóa cho mục đích phân tích.
- **Nguồn dữ liệu:** Kết hợp dữ liệu cảm biến, probe data (điểm đo từ phương tiện), và hợp tác với nhà cung cấp địa phương.
- **Ưu điểm:** Chất lượng dữ liệu dành cho routing và dự báo thời gian đến (ETA) tốt; có các API chuyên nghiệp, SLA cho doanh nghiệp.
- **Hạn chế:** Dữ liệu thương mại (giấy phép) — chi phí có thể cao cho quy mô toàn thành phố; hạn chế về minh bạch (khó truy xuất nguồn gốc chi tiết từng điểm dữ liệu).

3.2.2.2 Mapbox

- **Tính năng:** Cung cấp công cụ hiển thị bản đồ (map rendering), vector tiles, routing, traffic overlays, và SDK cho frontend (web/mobile). Hỗ trợ tùy biến bản đồ cao, phù hợp cho dashboard và trực quan hoá.
- **Nguồn dữ liệu:** Kết hợp OpenStreetMap, dữ liệu đối tác, và các dịch vụ traffic provider để hiển thị trạng thái giao thông.
- **Ưu điểm:** Khả năng tùy biến giao diện bản đồ mạnh — phù hợp cho hệ thống giám sát và dashboard; dễ tích hợp với stack React/Deck.gl để hiển thị dữ liệu lớn.
- **Hạn chế:** Mapbox chủ yếu tập trung vào visualization và SDK — phần dữ liệu traffic chi tiết và phân tích nâng cao thường phụ thuộc vào nguồn bên thứ ba hoặc phải trả phí.

3.2.2.3 HERE Technologies

- **Tính năng:** Dịch vụ bản đồ, routing, traffic flow, incidents, map-matching, và rich telemetry. HERE có các API dành cho doanh nghiệp và hỗ trợ phân tích lịch sử/real-time.
- **Ưu điểm:** Strong enterprise features, hợp đồng dữ liệu cho thành phố, hỗ trợ map-matching chính xác — hữu ích khi kết hợp dữ liệu GPS/OBD.
- **Hạn chế:** Tương tự TomTom — chi phí và tính thương mại của dữ liệu cần cân nhắc.

3.2.2.4 INRIX

- **Tính năng:** Chuyên về dữ liệu lưu lượng và phân tích tắc nghẽn; cung cấp chỉ số congestion, travel time index, historical trend reports.
- **Ưu điểm:** Dữ liệu phân tích chuyên sâu, thuận lợi cho hoạch định chính sách và nghiên cứu dài hạn.
- **Hạn chế:** Chi phí bản quyền, không phải giải pháp “plug-and-play” cho các ứng dụng nhỏ.

So sánh ngắn gọn và vai trò trong hệ thống quản lý giao thông đô thị

- **Ứng dụng cho điều phối thành phố:** TomTom, HERE, và INRIX cung cấp dữ liệu và API đủ mạnh để phục vụ phân tích và báo cáo cấp thành phố (ví dụ: phân tích xu hướng, báo cáo tắc nghẽn theo chu kỳ). Tuy nhiên các dữ

liệu này thường cần mua bản quyền và tích hợp bổ sung để phục vụ mục tiêu “giải thích lan truyền” (causal/spatio-temporal influence).

- **Ứng dụng cho trực quan hoá và giao diện:** Mapbox là lựa chọn thuận tiện để xây dựng dashboard tương tác và trực quan hoá dữ liệu thời gian thực (kết hợp với React, Deck.gl, hoặc Kepler.gl).
- **Tính mở và tích hợp với hệ thống nội bộ:** Nếu mục tiêu là nghiên cứu khoa học và mô hình hoá chi tiết (ví dụ: xây dựng ma trận kề động, chạy lại mô hình trên dữ liệu lịch sử), cần kết hợp dữ liệu thương mại này với nguồn nội địa (camera, loop detectors, GPS probe) và lưu trữ trong TimescaleDB/PostGIS. Dữ liệu thương mại nên được xem như *một nguồn bổ trợ* chứ không thay thế hoàn toàn dữ liệu cảm biến tại chỗ.
- **Chi phí và pháp lý:** Các dịch vụ thương mại yêu cầu hợp đồng bản quyền và có thể giới hạn quyền sử dụng dữ liệu cho mục đích nghiên cứu/tái phân phối — yếu tố cần cân nhắc khi thiết kế kiến trúc dữ liệu mở cho thành phố.

3.2.3 UTraffic

UTraffic là nền tảng giám sát giao thông thời gian thực thông qua camera và cảm biến. Hệ thống đang thực hiện tốt chức năng giám sát, nhưng vẫn còn khoảng trống lớn trong:

- phân tích sâu (deep analysis),
- dự báo thời gian dài,
- và mô hình hóa lan truyền tắc nghẽn.

3.3 Khoảng trống nghiên cứu (Gap Analysis)

Dựa trên tổng quan các công trình và hệ thống thực tế, có thể xác định ba khoảng trống chính:

- (a) **Hạn chế của ma trận kề tĩnh:** Các mô hình hiện tại như T-GCN vẫn sử dụng đồ thị tĩnh, không phản ánh đúng mối quan hệ động giữa các nút giao thông trong các đô thị phức tạp như TP.HCM.
- (b) **Thiếu nghiên cứu cho giao thông hỗn hợp:** Phần lớn công trình quốc tế nghiên cứu trên dữ liệu ô tô hoặc taxi, vốn có luồng di chuyển tổ chức. TP.HCM lại có đặc thù xe máy chiếm đa số, tạo ra tính hỗn loạn (chaotic) cao.
- (c) **Nhu cầu chuyển từ dự báo sang giải thích:** Các mô hình hiện tại chủ yếu báo trước tắc nghẽn, nhưng chưa giải thích được cơ chế lan truyền và mức độ ảnh hưởng giữa các nút – điều cần thiết cho hoạch định chiến lược.

Tóm tắt

Chương này đã phân tích toàn diện các công trình nghiên cứu và hệ thống thực tế trong lĩnh vực giao thông. Những khoảng trống được xác định là cơ sở quan trọng để đề tài lựa chọn hướng tiếp cận: **cải tiến T-GCN bằng cơ chế học ma trận kê động (Dynamic Graph Learning)**, từ đó nâng cao khả năng dự báo và phân tích lan truyền tắc nghẽn trên dữ liệu giao thông thực tế tại TP.HCM.

4 Phương pháp giải quyết

Chương này trình bày chi tiết phương pháp luận và các bước thực hiện để giải quyết bài toán phân tích mối liên hệ giao thông giữa các nút quan trọng tại TP.HCM. Phương pháp được thiết kế dựa trên nền tảng lý thuyết đã trình bày ở Chương 2 và các khoảng trống nghiên cứu được xác định ở Chương 3, đồng thời tận dụng dữ liệu lịch sử từ TomTom Historical Traffic Stats API.

4.1 Tổng quan phương pháp tiếp cận

4.1.1 Định hướng giải pháp

Để giải quyết bài toán phân tích mối liên hệ và lan truyền tắc nghẽn giữa các nút giao thông, đề tài đề xuất một pipeline xử lý toàn diện kết hợp nhiều kỹ thuật từ Big Data, Deep Learning, và phân tích thống kê. Giải pháp tập trung vào ba mục tiêu chính:

- (a) **Phát hiện tương quan không gian-thời gian:** Xác định các cặp nút giao thông có mức độ ảnh hưởng lẫn nhau cao, định lượng độ mạnh của mỗi tương quan và xác định độ trễ lan truyền.
- (b) **Xác định quan hệ nhân quả:** Phân biệt mối tương quan đơn thuần và quan hệ nhân-quả thực sự, xác định nút nào là nguyên nhân gây ra tắc nghẽn lan truyền.
- (c) **Dự báo lan truyền tắc nghẽn:** Xây dựng mô hình có khả năng dự báo khi một nút giao thông bị tắc nghẽn sẽ lan truyền đến các nút nào và trong bao lâu.

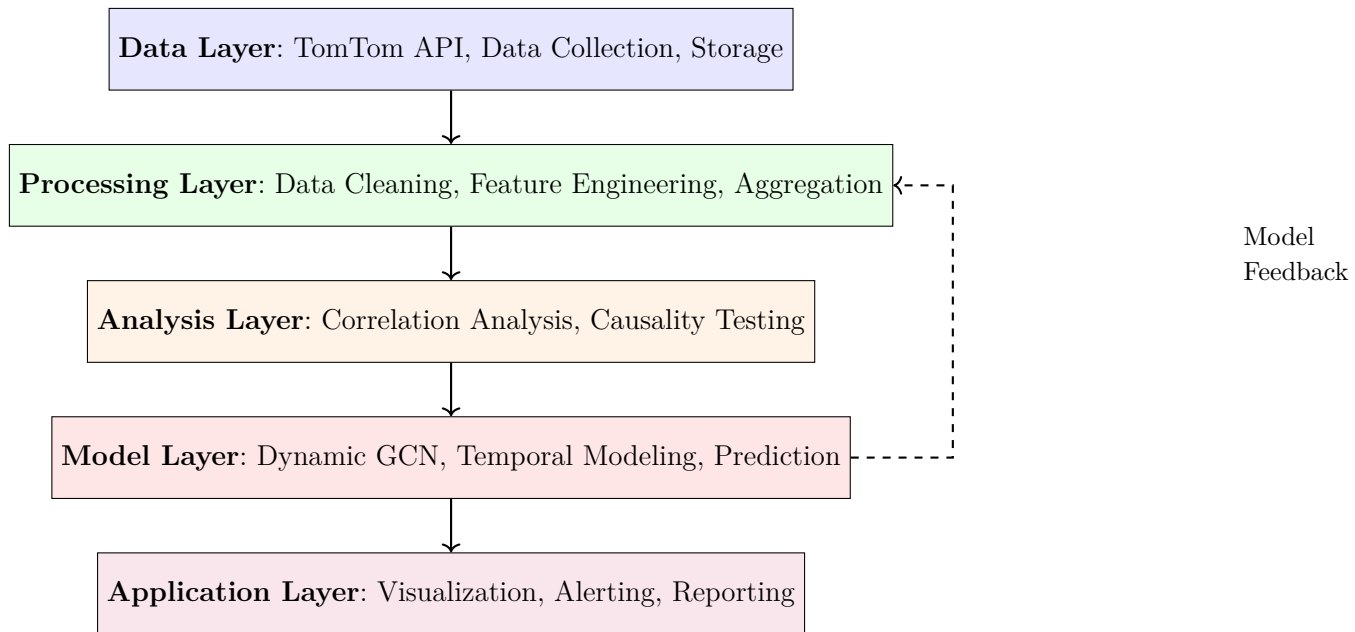
4.1.2 Kiến trúc tổng thể

Hệ thống được thiết kế theo kiến trúc phân tầng, mỗi tầng đảm nhiệm một nhóm chức năng cụ thể:

4.1.3 Quy trình xử lý chính

Pipeline xử lý dữ liệu và phân tích được thực hiện theo trình tự sau:

- (a) **Thu thập dữ liệu:** Lấy dữ liệu lịch sử từ TomTom API cho các nút giao thông được chọn
- (b) **Tiền xử lý:** Làm sạch, chuẩn hóa, và xử lý missing values
- (c) **Trích xuất đặc trưng:** Tính toán các đặc trưng giao thông và thời gian
- (d) **Phân tích tương quan:** Xác định mối tương quan không gian-thời gian
- (e) **Kiểm định nhân quả:** Áp dụng Granger Causality Test



Hình 2: Kiến trúc tổng thể hệ thống

- (f) **Xây dựng đồ thị động:** Tạo ma trận kề thay đổi theo thời gian
- (g) **Huấn luyện mô hình:** Phát triển mô hình dự báo dựa trên GNN
- (h) **Đánh giá và triển khai:** Kiểm thử và tích hợp vào hệ thống

4.2 Thu thập và chuẩn bị dữ liệu

4.2.1 Nguồn dữ liệu: TomTom Historical Traffic Stats

4.2.1.a Lý do lựa chọn TomTom API

Dữ liệu giao thông lịch sử từ TomTom được lựa chọn làm nguồn dữ liệu chính cho đề tài vì các lý do sau:

- **Độ phủ toàn diện:** TomTom cung cấp dữ liệu cho hầu hết các tuyến đường chính tại TP.HCM, bao gồm cả các khu vực trung tâm và vùng ven.
- **Chất lượng dữ liệu:** Dữ liệu được tổng hợp từ nhiều nguồn (GPS probe data, cảm biến, crowdsourcing) và đã qua xử lý, lọc nhiễu bởi TomTom.
- **Dữ liệu lịch sử dài hạn:** API cung cấp dữ liệu lịch sử từ nhiều tháng đến nhiều năm, đủ để huấn luyện mô hình học sâu.
- **Độ phân giải thời gian phù hợp:** Dữ liệu có độ phân giải 5 phút, cân bằng giữa chi tiết và khả năng xử lý.
- **Các chỉ số phong phú:** Bao gồm tốc độ thực tế, tốc độ tự do, thời gian di chuyển, độ tin cậy, giúp phân tích đa chiều.

4.2.1.b Cấu trúc dữ liệu TomTom

TomTom Historical Traffic Stats API cung cấp dữ liệu theo cấu trúc sau:

- **Road Segments:** Mỗi đoạn đường được định danh bởi segment ID duy nhất
- **Temporal Resolution:** Dữ liệu theo khung thời gian 5 phút
- **Speed Metrics:**
 - `currentSpeed`: Tốc độ trung bình thực tế (km/h)
 - `freeFlowSpeed`: Tốc độ tự do khi không có tắc nghẽn (km/h)
 - `confidence`: Độ tin cậy của số liệu (0-1)
- **Congestion Indicators:**
 - `travelTime`: Thời gian di chuyển thực tế (giây)
 - `freeFlowTravelTime`: Thời gian di chuyển lý tưởng (giây)
- **Metadata:** Thông tin đoạn đường (tên đường, tọa độ, chiều dài)

4.2.2 Lựa chọn nút giao thông nghiên cứu

4.2.2.a Tiêu chí lựa chọn

Việc lựa chọn các nút giao thông để nghiên cứu dựa trên các tiêu chí sau:

(a) **Tầm quan trọng chiến lược:**

- Nút nằm trên các tuyến huyết mạch nối các quận trung tâm
- Cổng vào/ra khu vực trung tâm TP.HCM
- Giao lộ có lưu lượng cao và thường xuyên ùn tắc

(b) **Độ phủ dữ liệu:**

- Đảm bảo có dữ liệu TomTom liên tục cho đoạn đường
- Tỷ lệ missing data dưới 10%
- Độ tin cậy trung bình (confidence) trên 0.7

(c) **Tính đại diện:**

- Phân bố đều trong khu vực nghiên cứu
- Bao gồm nhiều loại đường: đại lộ, đường chính, đường nhánh
- Đa dạng về mật độ giao thông và đặc điểm vận hành

(d) **Khả năng kết nối:**

- Có mối liên hệ trực tiếp hoặc gián tiếp với các nút khác
- Nằm trong cùng một mạng lưới liên thông

4.2.2.b Danh sách nút giao thông được chọn

Dựa trên các tiêu chí trên, đề tài lựa chọn 20 nút giao thông trọng điểm tại TP.HCM, bao gồm:

4.2.3 Quy trình thu thập dữ liệu

4.2.3.a Thiết kế data pipeline

Pipeline thu thập dữ liệu được thiết kế để đảm bảo tính ổn định, hiệu quả và khả năng mở rộng:

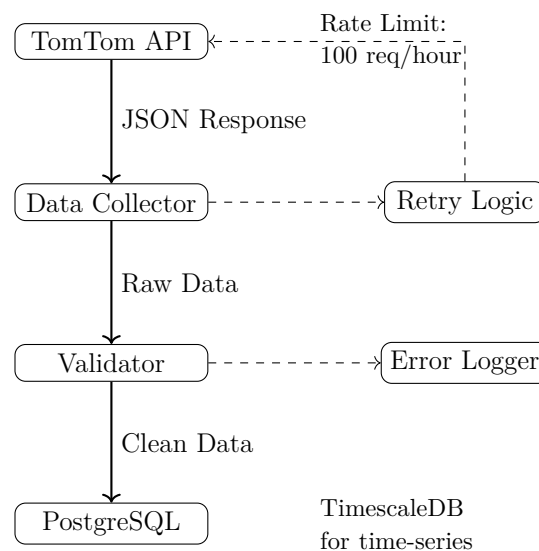
4.2.3.b Các bước thu thập

(a) **Xác định time range:**

- Thu thập dữ liệu từ 01-8-2024 đến 31-8-2024

Bảng 4: Các nút giao thông được lựa chọn

STT	Tên nút/đường	Khu vực
1	Ngã ba Cách Mạng Tháng 8 - Điện Biên Phủ	Quận 3
2	Ngã tư Hàng Xanh	Quận Bình Thạnh
3	Cầu Sài Gòn	Quận 1 - Quận 4
4	Đường Võ Văn Kiệt (đoạn Q1)	Quận 1
5	Ngã tư Bảy Hiền	Quận Tân Bình
6	Xa lộ Hà Nội (đoạn Q9)	Quận 9
7	Đại lộ Võ Văn Kiệt (đoạn Q5)	Quận 5
8	Nguyễn Thị Minh Khai	Quận 1, Quận 3
9	Cầu Kênh Tẻ	Quận 1 - Quận 7
10	Đường 3 Tháng 2	Quận 10, Quận 11
...	...	...



Hình 3: Pipeline thu thập dữ liệu từ TomTom API

- Độ phân giải: 5 phút/lần đo
- Tập trung vào giờ cao điểm: 6h-9h, 17h-20h

(b) **API Request Strategy:**

- Batch requests để tối ưu số lần gọi API
- Implement exponential backoff khi gặp rate limit
- Cache responses để tránh duplicate requests

(c) **Data Validation:**

- Kiểm tra completeness: đảm bảo đủ các trường bắt buộc
- Kiểm tra consistency: tốc độ không âm, trong giới hạn hợp lý
- Kiểm tra temporal continuity: không có gaps lớn giữa các measurements

(d) **Error Handling:**

- Log tất cả errors để phân tích sau

- Retry failed requests với backoff strategy
- Alert khi error rate vượt threshold

4.2.4 Tiền xử lý dữ liệu

4.2.4.a Xử lý missing values

Dữ liệu giao thông thực tế thường có missing values do nhiều nguyên nhân: lỗi cảm biến, mất kết nối, hoặc không đủ probe data. Chiến lược xử lý:

(a) **Phân loại missing patterns:**

- *MCAR (Missing Completely At Random)*: Xảy ra ngẫu nhiên
- *MAR (Missing At Random)*: Phụ thuộc vào biến quan sát được
- *MNAR (Missing Not At Random)*: Phụ thuộc vào giá trị bị missing

(b) **Phương pháp imputation:**

- **Linear Interpolation**: Cho gaps nhỏ (< 3 time steps)

$$v_t = v_{t-1} + \frac{v_{t+k} - v_{t-1}}{k + 1}$$

- **Seasonal Decomposition**: Cho gaps trung bình

$$v_t = \text{trend}_t + \text{seasonal}_t + \text{residual}_t$$

- **KNN Imputation**: Dựa vào các nút láng giềng tương tự

$$v_t^i = \frac{1}{k} \sum_{j \in \text{KNN}(i)} v_t^j \cdot w_{ij}$$

- **Deep Learning Imputation**: Sử dụng LSTM autoencoder cho gaps lớn

(c) **Quality threshold:**

- Loại bỏ nút có $> 20\%$ missing data
- Đánh dấu imputed values để đánh giá độ tin cậy

4.2.4.b Outlier detection và xử lý

Outliers trong dữ liệu giao thông có thể do tai nạn, sự kiện đặc biệt, hoặc lỗi đo:

(a) **Statistical Methods:**

- **Z-score:** Phát hiện điểm bất thường dựa trên độ lệch chuẩn

$$z_i = \frac{x_i - \mu}{\sigma}, \quad |z_i| > 3 \Rightarrow \text{outlier}$$

- **IQR (Interquartile Range):**

$$\text{outlier if } x < Q_1 - 1.5 \cdot \text{IQR} \text{ or } x > Q_3 + 1.5 \cdot \text{IQR}$$

(b) **Temporal Context:**

- So sánh với cùng thời điểm các ngày trước
- Xem xét xu hướng và mùa vụ
- Kiểm tra consistency với nút láng giềng

(c) **Handling Strategy:**

- **Keep:** Nếu có bằng chứng là sự kiện thực (tai nạn, sự kiện)
- **Cap:** Giới hạn trong khoảng hợp lý
- **Remove:** Nếu xác định là lỗi đo
- **Flag:** Đánh dấu để phân tích riêng

4.2.4.c Normalization và scaling

Chuẩn hóa dữ liệu là bước quan trọng để mô hình học hiệu quả:

(a) **Min-Max Scaling:**

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Sử dụng cho features có phân phối đồng đều, giữ nguyên shape.

(b) **Z-score Standardization:**

$$x_{\text{std}} = \frac{x - \mu}{\sigma}$$

Phù hợp khi features có phân phối gần Gaussian.

(c) **Robust Scaling:**

$$x_{\text{robust}} = \frac{x - \text{median}}{\text{IQR}}$$

Ít bị ảnh hưởng bởi outliers.

Lựa chọn phương pháp dựa trên:

- Phân phối của từng feature
- Yêu cầu của model (GNN, LSTM thường cần standardization)



- Presence of outliers

4.3 Trích xuất và phân tích đặc trưng

4.3.1 Đặc trưng giao thông cơ bản

4.3.1.a Speed-based features

Từ dữ liệu TomTom, trích xuất các đặc trưng liên quan đến tốc độ:

(a) **Relative Speed:**

$$v_{\text{rel}}(t) = \frac{v_{\text{current}}(t)}{v_{\text{freeflow}}}$$

Chỉ số này phản ánh mức độ tắc nghẽn, với giá trị gần 1 nghĩa là lưu thông tốt.

(b) **Speed Reduction Ratio:**

$$r_{\text{reduction}}(t) = 1 - v_{\text{rel}}(t) = \frac{v_{\text{freeflow}} - v_{\text{current}}(t)}{v_{\text{freeflow}}}$$

Tỷ lệ giảm tốc độ so với lý tưởng.

(c) **Congestion Index:**

$$CI(t) = \frac{t_{\text{travel}}(t)}{t_{\text{freeflow}}} - 1$$

Chỉ số tắc nghẽn dựa trên tỷ lệ thời gian di chuyển tăng thêm.

(d) **Speed Variance:**

$$\sigma_v^2 = \frac{1}{n} \sum_{i=1}^n (v_i - \bar{v})^2$$

Độ biến động tốc độ phản ánh tính không ổn định của giao thông.

4.3.1.b Temporal features

Các đặc trưng theo thời gian giúp mô hình nhận biết patterns chu kỳ:

(a) **Time of Day Features:**

- Hour of day: $h \in [0, 23]$
- Minute of hour: $m \in [0, 59]$
- Cyclic encoding để giữ tính liên tục:

$$h_{\text{sin}} = \sin\left(\frac{2\pi h}{24}\right), \quad h_{\text{cos}} = \cos\left(\frac{2\pi h}{24}\right)$$

(b) **Day of Week Features:**

- Weekday vs Weekend (binary)
- Day of week: $d \in [0, 6]$
- Cyclic encoding tương tự

(c) **Peak/Off-peak Indicator:**

$$\text{isPeak}(t) = \begin{cases} 1, & \text{if } t \in [6, 9] \cup [16, 19] \\ 0, & \text{otherwise} \end{cases}$$

4.3.2 Đặc trưng không gian

4.3.2.a Spatial proximity features

Tính toán các đặc trưng về mối quan hệ không gian giữa các nút:

(a) **Euclidean Distance:**

$$d_{\text{euclid}}(i, j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

(b) **Road Network Distance:**

$$d_{\text{network}}(i, j) = \text{shortest_path_length}(i, j)$$

Khoảng cách thực tế theo mạng đường, tính bằng thuật toán Dijkstra.

(c) **Travel Time Distance:**

$$d_{\text{time}}(i, j) = \mathbb{E}[t_{\text{travel}}(i \rightarrow j)]$$

Khoảng cách dựa trên thời gian di chuyển trung bình.

(d) **Connectivity Features:**

- Direct connection: Binary indicator
- Number of intermediate nodes
- Betweenness centrality: Đo lường tầm quan trọng của nút trong mạng

4.3.2.b Neighborhood aggregation

Tổng hợp thông tin từ các nút láng giềng:

(a) **Spatial Average:**

$$\bar{v}_i^{\text{spatial}}(t) = \frac{1}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} v_j(t)$$

(b) **Weighted Spatial Average:**

$$\bar{v}_i^{\text{weighted}}(t) = \frac{\sum_{j \in \mathcal{N}_i} w_{ij} \cdot v_j(t)}{\sum_{j \in \mathcal{N}_i} w_{ij}}$$

với trọng số $w_{ij} = \exp(-d_{ij}/\sigma)$ dựa trên khoảng cách.

(c) **Spatial Variance:**

$$\sigma_i^{\text{spatial}}(t) = \sqrt{\frac{1}{|\mathcal{N}_i|} \sum_{j \in \mathcal{N}_i} (v_j(t) - \bar{v}_i^{\text{spatial}}(t))^2}$$

4.3.3 Đặc trưng động học (Dynamic Features)

4.3.3.a Temporal derivatives

Các đạo hàm theo thời gian phản ánh xu hướng thay đổi của giao thông:

(a) **First-order Derivative (Velocity):**

$$\frac{dv}{dt} \approx \frac{v(t) - v(t-1)}{\Delta t}$$

Tốc độ thay đổi của traffic speed, cho biết giao thông đang cải thiện hay xấu đi.

(b) **Second-order Derivative (Acceleration):**

$$\frac{d^2v}{dt^2} \approx \frac{v(t) - 2v(t-1) + v(t-2)}{\Delta t^2}$$

Gia tốc thay đổi, hữu ích để phát hiện điểm chuyển pha (từ thông sang tắc).

(c) **Rate of Change:**

$$r_{\text{change}}(t) = \frac{v(t) - v(t-k)}{v(t-k)} \times 100\%$$

Phần trăm thay đổi so với k bước thời gian trước.

4.3.3.b Trend and momentum features

(a) **Moving Average:**

$$\text{MA}_k(t) = \frac{1}{k} \sum_{i=0}^{k-1} v(t-i)$$

Làm mượt nhiễu và phát hiện xu hướng.

(b) **Exponential Moving Average:**

$$\text{EMA}_\alpha(t) = \alpha \cdot v(t) + (1 - \alpha) \cdot \text{EMA}_\alpha(t - 1)$$

Cho trọng số cao hơn cho dữ liệu gần đây.

(c) **Momentum:**

$$M(t) = v(t) - v(t - k)$$

Đo lường sức mạnh của xu hướng.

(d) **Volatility:**

$$\text{Vol}(t) = \sqrt{\frac{1}{k} \sum_{i=0}^{k-1} (v(t - i) - \text{MA}_k(t))^2}$$

Độ dao động trong window thời gian.

4.3.4 Đặc trưng tương quan sơ bộ

Trước khi áp dụng các phương pháp phức tạp, tính toán tương quan ban đầu:

(a) **Pearson Correlation:**

$$\rho_{ij} = \frac{\text{Cov}(X_i, X_j)}{\sigma_{X_i} \sigma_{X_j}}$$

Đo lường tương quan tuyến tính giữa hai nút.

(b) **Time-lagged Correlation:**

$$\rho_{ij}(\tau) = \text{Corr}(X_i(t), X_j(t + \tau))$$

Tương quan với độ trễ τ , giúp xác định delay trong lan truyền.

(c) **Cross-correlation Function:**

$$\text{CCF}_{ij}(\tau) = \frac{\mathbb{E}[(X_i(t) - \mu_i)(X_j(t + \tau) - \mu_j)]}{\sigma_i \sigma_j}$$

4.4 Phân tích tương quan không gian-thời gian

4.4.1 Phương pháp phân tích tương quan

4.4.1.a Dynamic Time Warping (DTW)

DTW đo lường độ tương đồng giữa hai chuỗi thời gian có thể bị lệch về thời gian:

(a) **Định nghĩa DTW Distance:**

Cho hai chuỗi $X = (x_1, x_2, \dots, x_n)$ và $Y = (y_1, y_2, \dots, y_m)$, DTW tìm alignment path tối ưu:

$$\text{DTW}(X, Y) = \min_{\pi} \sqrt{\sum_{(i,j) \in \pi} d(x_i, y_j)^2}$$

(b) **Thuật toán quy hoạch động:**

$$D(i, j) = d(x_i, y_j) + \min\{D(i-1, j), D(i, j-1), D(i-1, j-1)\}$$

(c) **Ứng dụng trong giao thông:**

- Phát hiện patterns tương tự giữa các nút mặc dù có time shift
- Xác định propagation delay giữa các nút
- Clustering các nút có behavior tương đồng

4.4.1.b Wavelet Coherence Analysis

Wavelet coherence phân tích tương quan trong cả miền thời gian và tần số:

(a) **Continuous Wavelet Transform:**

$$W_x(a, b) = \frac{1}{\sqrt{|a|}} \int_{-\infty}^{\infty} x(t) \psi^* \left(\frac{t-b}{a} \right) dt$$

với a là scale parameter, b là translation parameter.

(b) **Wavelet Coherence:**

$$R^2(a, b) = \frac{|S(W_{xy}(a, b))|^2}{S(|W_x(a, b)|^2) \cdot S(|W_y(a, b)|^2)}$$

với S là smoothing operator.

(c) **Phase Difference:**

$$\phi_{xy}(a, b) = \arctan \left(\frac{\Im\{W_{xy}(a, b)\}}{\Re\{W_{xy}(a, b)\}} \right)$$

Cho biết nút nào dẫn trước (leading) và nút nào theo sau (lagging).

4.4.2 Xây dựng ma trận tương quan

4.4.2.a Static correlation matrix

Ma trận tương quan tĩnh được tính trên toàn bộ dữ liệu:

(a) **Pearson Correlation Matrix:**

$$C_{ij} = \frac{\sum_{t=1}^T (x_i(t) - \bar{x}_i)(x_j(t) - \bar{x}_j)}{\sqrt{\sum_{t=1}^T (x_i(t) - \bar{x}_i)^2} \sqrt{\sum_{t=1}^T (x_j(t) - \bar{x}_j)^2}}$$

(b) **Distance Correlation:**

$$\text{dCor}(X, Y) = \frac{\text{dCov}(X, Y)}{\sqrt{\text{dCov}(X, X) \cdot \text{dCov}(Y, Y)}}$$

Distance correlation có thể phát hiện cả quan hệ phi tuyến.

(c) **Mutual Information:**

$$I(X; Y) = \sum_{x \in X} \sum_{y \in Y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}$$

Đo lường mức độ phụ thuộc tổng quát, không giới hạn tuyến tính.

4.4.2.b Time-varying correlation matrix

Ma trận tương quan động thay đổi theo thời gian:

(a) **Rolling Window Correlation:**

$$C_{ij}(t) = \text{Corr}(X_i[t - w : t], X_j[t - w : t])$$

với w là window size.

(b) **Exponentially Weighted Correlation:**

$$C_{ij}(t) = \frac{\sum_{k=0}^t \lambda^k (x_i(t - k) - \bar{x}_i)(x_j(t - k) - \bar{x}_j)}{\sqrt{\sum_{k=0}^t \lambda^k (x_i(t - k) - \bar{x}_i)^2} \sqrt{\sum_{k=0}^t \lambda^k (x_j(t - k) - \bar{x}_j)^2}}$$

(c) **State-dependent Correlation:**

$$C_{ij}^{(s)} = \text{Corr}(X_i|S = s, X_j|S = s)$$

Tương quan phụ thuộc vào trạng thái giao thông (normal, congested, incident).

4.4.3 Xác định độ trễ lan truyền

4.4.3.a Cross-correlation analysis

(a) Tính CCF cho nhiều lags:

$$\text{CCF}_{ij}(\tau) = \text{Corr}(X_i(t), X_j(t + \tau)), \quad \tau \in [-\tau_{\max}, \tau_{\max}]$$

(b) Xác định optimal lag:

$$\tau_{ij}^* = \arg \max_{\tau} |\text{CCF}_{ij}(\tau)|$$

(c) Interpretation:

- Nếu $\tau_{ij}^* > 0$: nút i dẫn trước nút j (influence direction $i \rightarrow j$)
- Nếu $\tau_{ij}^* < 0$: nút j dẫn trước nút i (influence direction $j \rightarrow i$)
- $|\tau_{ij}^*|$ là độ trễ lan truyền (propagation delay)

4.4.3.b Transfer Entropy

Transfer Entropy đo lường lượng thông tin truyền từ nút này sang nút khác:

$$TE_{i \rightarrow j} = \sum p(x_j^{t+1}, x_j^{(k)}, x_i^{(l)}) \log \frac{p(x_j^{t+1} | x_j^{(k)}, x_i^{(l)})}{p(x_j^{t+1} | x_j^{(k)})} \quad (22)$$

với:

- $x_j^{(k)} = (x_j^t, x_j^{t-1}, \dots, x_j^{t-k+1})$: lịch sử của nút j
- $x_i^{(l)} = (x_i^t, x_i^{t-1}, \dots, x_i^{t-l+1})$: lịch sử của nút i

Transfer Entropy cao nghĩa là nút i mang thông tin dự báo hữu ích cho nút j .

4.5 Kiểm định quan hệ nhân quả

4.5.1 Granger Causality Test

4.5.1.a Cơ sở lý thuyết

Granger Causality dựa trên ý tưởng: nếu X gây ra Y , thì giá trị quá khứ của X sẽ giúp dự báo tốt hơn giá trị tương lai của Y , so với chỉ dùng giá trị quá khứ của Y .

(a) **Unrestricted Model:**

$$Y_t = \alpha_0 + \sum_{i=1}^p \alpha_i Y_{t-i} + \sum_{j=1}^q \beta_j X_{t-j} + \epsilon_t$$

(b) **Restricted Model:**

$$Y_t = \alpha_0 + \sum_{i=1}^p \alpha_i Y_{t-i} + \eta_t$$

(c) **Test Statistic:**

$$F = \frac{(\text{RSS}_R - \text{RSS}_U)/q}{\text{RSS}_U/(T - p - q - 1)}$$

với RSS là residual sum of squares.

(d) **Null Hypothesis:** $H_0 : \beta_1 = \beta_2 = \dots = \beta_q = 0$ (X không Granger-cause Y)

(e) **Decision:** Nếu $p\text{-value} < \alpha$ (thường 0.05), reject H_0 , kết luận X Granger-causes Y.

4.5.1.b Pairwise Granger Causality

Thực hiện kiểm định cho tất cả các cặp nút:

(a) **Test** $i \rightarrow j$: Kiểm tra liệu nút i có Granger-cause nút j không

(b) **Test** $j \rightarrow i$: Kiểm tra chiều ngược lại

(c) **Kết quả có thể:**

- Unidirectional: $i \rightarrow j$ hoặc $j \rightarrow i$
- Bidirectional: $i \leftrightarrow j$ (ảnh hưởng lẫn nhau)
- No causality: không có quan hệ nhân quả

4.5.1.c Lag selection

Việc chọn số lag p và q quan trọng cho độ chính xác:

(a) **Akaike Information Criterion (AIC):**

$$AIC(p) = 2k - 2 \ln(\hat{L})$$

với k là số tham số, $\hat{L}$ là likelihood.

(b) **Bayesian Information Criterion (BIC):**

$$BIC(p) = k \ln(n) - 2 \ln(\hat{L})$$

(c) **Strategy:**

- Test với nhiều giá trị lag khác nhau
- Chọn lag có AIC hoặc BIC nhỏ nhất
- Xem xét domain knowledge (propagation thường trong 15-30 phút)

4.5.2 Conditional Granger Causality

Trong mạng lưới phức tạp, cần kiểm soát ảnh hưởng của các nút khác:

(a) **Model với conditioning:**

$$Y_t = \alpha_0 + \sum_{i=1}^p \alpha_i Y_{t-i} + \sum_{j=1}^q \beta_j X_{t-j} + \sum_{k=1}^r \gamma_k Z_{t-k} + \epsilon_t$$

với Z là các nút điều kiện.

(b) **Ứng dụng:**

- Phát hiện spurious causality do confounding variables
- Xác định direct vs indirect influence
- Tìm mediating nodes trong propagation path

4.5.3 Xây dựng Causal Graph

4.5.3.a Từ kết quả kiểm định đến đồ thị

(a) **Adjacency Matrix Construction:**

$$A_{ij}^{\text{causal}} = \begin{cases} 1, & \text{if } i \text{ Granger-causes } j \text{ (significant)} \\ 0, & \text{otherwise} \end{cases}$$

(b) **Weighted Causal Graph:**

$$W_{ij}^{\text{causal}} = F\text{-statistic}_{i \rightarrow j} \text{ or } 1 - p\text{-value}_{i \rightarrow j}$$

(c) **Thresholding:**

- Giữ chỉ các edges có p -value < 0.05
- Hoặc giữ top-k strongest causal links
- Hoặc dùng FDR (False Discovery Rate) correction cho multiple testing

4.5.3.b Graph properties analysis

Phân tích các tính chất của causal graph:

(a) **In-degree và Out-degree:**

$$d_j^{\text{in}} = \sum_{i=1}^N A_{ij}^{\text{causal}}, \quad d_i^{\text{out}} = \sum_{j=1}^N A_{ij}^{\text{causal}}$$

- High out-degree: nút là "source" của tác nhân (influential node)
- High in-degree: nút dễ bị ảnh hưởng (vulnerable node)

(b) **Betweenness Centrality:**

$$BC(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

với σ_{st} là số shortest paths từ s đến t , $\sigma_{st}(v)$ là số paths đi qua v .

Nút có BC cao là "bottleneck" trong propagation network.

(c) **PageRank:**

$$PR(i) = \frac{1-d}{N} + d \sum_{j \in \text{In}(i)} \frac{PR(j)}{d_j^{\text{out}}}$$

Đánh giá tầm quan trọng của nút trong mạng causal.

4.6 Xây dựng mô hình dự báo

4.6.1 Kiến trúc Dynamic Graph Convolutional Network

4.6.1.a Motivation

Các mô hình T-GCN truyền thống sử dụng ma trận kề tĩnh A , không phản ánh:

- Sự thay đổi của mối quan hệ theo thời gian
- Ảnh hưởng của trạng thái giao thông hiện tại lên cấu trúc mạng
- Các mối liên hệ tiềm ẩn không thể hiện qua kết nối vật lý

Do đó, đề tài đề xuất Dynamic GCN với ma trận kề được học và cập nhật theo thời gian.

4.6.1.b Dynamic Adjacency Matrix Learning

Adaptive Graph Learning:

(a) **Node Embedding:**

Mỗi nút i được biểu diễn bởi một embedding vector học được:

$$\mathbf{e}_i \in \mathbb{R}^d$$

(b) **Similarity-based Adjacency:**

$$A_{ij}^{(t)} = \frac{\exp(\text{sim}(\mathbf{h}_i^{(t)}, \mathbf{h}_j^{(t)}))}{\sum_{k=1}^N \exp(\text{sim}(\mathbf{h}_i^{(t)}, \mathbf{h}_k^{(t)}))}$$

với $\text{sim}(\cdot, \cdot)$ có thể là:

- Cosine similarity: $\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^T \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}$
- Scaled dot-product: $\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^T \mathbf{v}}{\sqrt{d}}$
- Bilinear: $\text{sim}(\mathbf{u}, \mathbf{v}) = \mathbf{u}^T \mathbf{W} \mathbf{v}$

(c) **Hybrid Adjacency Matrix:**

Kết hợp cả thông tin cấu trúc và dữ liệu:

$$A^{(t)} = \alpha A^{\text{static}} + (1 - \alpha) A^{\text{dynamic}}(t)$$

với:

- A^{static} : ma trận kề dựa trên cấu trúc đường (prior knowledge)

- $A^{\text{dynamic}}(t)$: ma trận học được từ dữ liệu tại thời điểm t
- $\alpha \in [0, 1]$: hyperparameter cân bằng

Attention-based Graph Construction:

Sử dụng cơ chế attention để học trọng số động:

$$\alpha_{ij}^{(t)} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i^{(t)} \parallel \mathbf{W}\mathbf{h}_j^{(t)}]))}{\sum_{k \in \mathcal{N}_i} \exp(\text{LeakyReLU}(\mathbf{a}^T[\mathbf{W}\mathbf{h}_i^{(t)} \parallel \mathbf{W}\mathbf{h}_k^{(t)}]))} \quad (23)$$

với $\parallel$ là concatenation operation.

4.6.1.c Temporal Modeling Component

GRU Cell với Dynamic Graph:

Tại mỗi time step t :

$$\mathbf{X}'_t = \text{GraphConv}(A^{(t)}, \mathbf{X}_t) \quad (24)$$

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{X}'_t, \mathbf{h}_{t-1}] + \mathbf{b}_r) \quad (25)$$

$$\mathbf{u}_t = \sigma(\mathbf{W}_u[\mathbf{X}'_t, \mathbf{h}_{t-1}] + \mathbf{b}_u) \quad (26)$$

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{X}'_t, \mathbf{r}_t \odot \mathbf{h}_{t-1}] + \mathbf{b}_h) \quad (27)$$

$$\mathbf{h}_t = \mathbf{u}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{u}_t) \odot \tilde{\mathbf{h}}_t \quad (28)$$

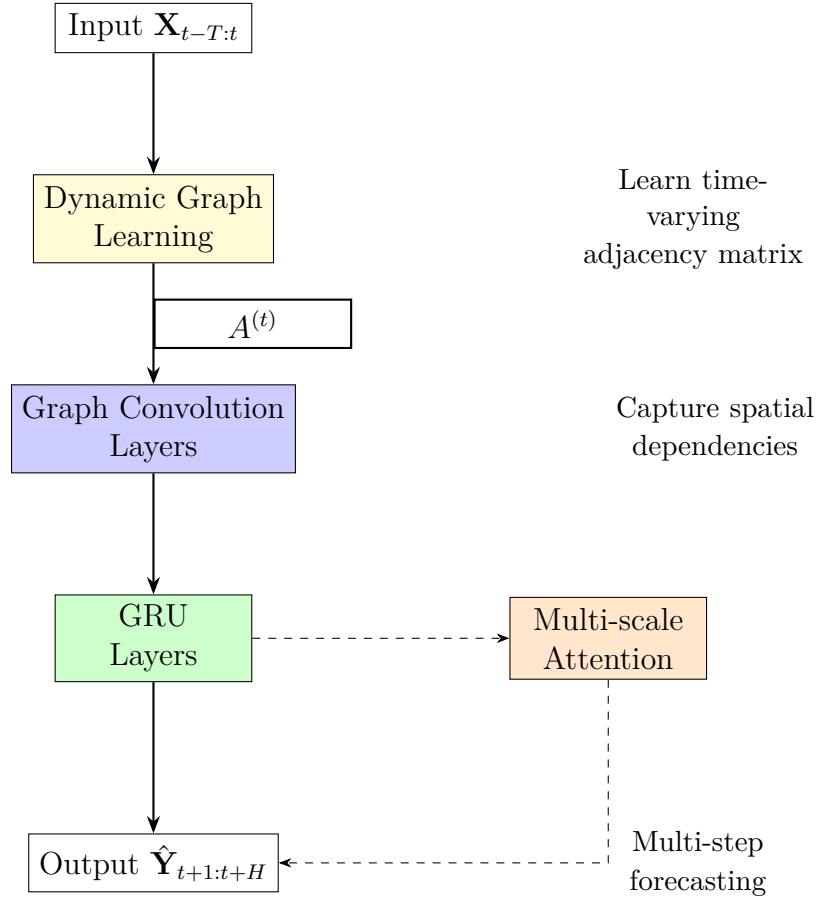
Multi-scale Temporal Attention:

Để nắm bắt dependencies ở nhiều time scales:

$$\mathbf{h}_t^{\text{multi}} = \text{Concat}[\mathbf{h}_t^{\text{short}}, \mathbf{h}_t^{\text{medium}}, \mathbf{h}_t^{\text{long}}] \quad (29)$$

với:

- $\mathbf{h}_t^{\text{short}}$: GRU trên window 12 steps (1 giờ)
- $\mathbf{h}_t^{\text{medium}}$: GRU trên window 36 steps (3 giờ)
- $\mathbf{h}_t^{\text{long}}$: GRU trên window 144 steps (12 giờ)



Hình 4: Kiến trúc mô hình *Dynamic GCN* với học đồ thị động và mô hình hóa thời gian đa tầng

4.6.1.d Kiến trúc tổng thể

4.6.2 Training Strategy

4.6.2.a Loss Function

Hàm mất mát kết hợp nhiều objectives:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{pred}} + \lambda_1 \mathcal{L}_{\text{graph}} + \lambda_2 \mathcal{L}_{\text{reg}} \quad (30)$$

1. Prediction Loss:

$$\mathcal{L}_{\text{pred}} = \frac{1}{NH} \sum_{i=1}^N \sum_{h=1}^H \|\mathbf{y}_i^{(t+h)} - \hat{\mathbf{y}}_i^{(t+h)}\|_2^2$$

Hoặc sử dụng Huber loss để robust hơn với outliers:

$$\mathcal{L}_{\text{Huber}}(\delta) = \begin{cases} \frac{1}{2}(y - \hat{y})^2, & |y - \hat{y}| \leq \delta \\ \delta|y - \hat{y}| - \frac{1}{2}\delta^2, & |y - \hat{y}| > \delta \end{cases}$$

2. Graph Regularization Loss:

Để đảm bảo ma trận kề học được có tính chất tốt:

$$\mathcal{L}_{\text{graph}} = \|A^{(t)} - A^{(t-1)}\|_F^2 + \lambda_{\text{sparse}} \|A^{(t)}\|_1$$

- Term 1: Temporal smoothness - ma trận kề không thay đổi quá đột ngột
- Term 2: Sparsity - khuyến khích mạng thưa, chỉ giữ connections quan trọng

3. Regularization Loss:

$$\mathcal{L}_{\text{reg}} = \|\theta\|_2^2$$

L2 regularization cho tất cả parameters để tránh overfitting.

4.6.2.b Optimization Algorithm

Adam Optimizer với Learning Rate Scheduling:

(a) **Base optimizer:** Adam với parameters:

- Initial learning rate: $\eta_0 = 0.001$
- $\beta_1 = 0.9$, $\beta_2 = 0.999$
- $\epsilon = 10^{-8}$

(b) **Learning rate schedule:**

$$\eta_t = \eta_0 \cdot \min\left(1, \frac{\text{warmup_steps}}{t}\right) \cdot 0.5^{\lfloor t/\text{decay_steps} \rfloor}$$

- Warmup period: 1000 steps để ổn định training
- Decay: giảm 50

(c) **Gradient clipping:**

$$\mathbf{g} \leftarrow \min\left(1, \frac{\text{max_norm}}{\|\mathbf{g}\|}\right) \mathbf{g}$$

với max_norm = 5.0 để tránh exploding gradients.

4.6.2.c Training Procedure

(a) **Data Splitting:**

- Training set: 70% (dữ liệu cũ nhất)
- Validation set: 15% (dữ liệu giữa)
- Test set: 15% (dữ liệu mới nhất)
- Temporal split - không shuffle để giữ tính thời gian

(b) **Training Loop:**

- Batch size: 32 sequences
- Input window: 12 time steps (1 giờ)
- Prediction horizon: 12 time steps (1 giờ)
- Early stopping: patience = 20 epochs
- Model checkpoint: lưu best model theo validation loss

(c) **Data Augmentation:**

- Add Gaussian noise: $\mathbf{X}' = \mathbf{X} + \mathcal{N}(0, 0.01)$
- Time masking: randomly mask một số time steps
- Node masking: randomly mask features của một số nodes

4.6.3 Model Ensemble

Để tăng độ robust, sử dụng ensemble của nhiều models:

(a) **Multiple Seeds:** Train 5 models với random seeds khác nhau

(b) **Ensemble Prediction:**

$$\hat{\mathbf{y}}_{\text{ensemble}} = \frac{1}{K} \sum_{k=1}^K w_k \hat{\mathbf{y}}_k$$

với weights w_k có thể là:

- Uniform: $w_k = 1/K$
- Performance-based: $w_k \propto 1/\text{MAE}_k^{\text{val}}$

(c) **Uncertainty Quantification:**

$$\sigma_{\text{ensemble}}^2 = \frac{1}{K} \sum_{k=1}^K (\hat{\mathbf{y}}_k - \hat{\mathbf{y}}_{\text{ensemble}})^2$$

Variance của predictions cho biết độ tin cậy của dự báo.

4.7 Hệ thống cảnh báo và phát hiện lan truyền

4.7.1 Thuật toán phát hiện bất thường

4.7.1.a Threshold-based Detection

Định nghĩa trạng thái tắc nghẽn:

$$\text{Congestion Level} = \begin{cases} \text{Free Flow,} & \text{if } CI(t) < 0.2 \\ \text{Moderate,} & \text{if } 0.2 \leq CI(t) < 0.5 \\ \text{Heavy,} & \text{if } 0.5 \leq CI(t) < 0.8 \\ \text{Severe,} & \text{if } CI(t) \geq 0.8 \end{cases}$$

với $CI(t) = \frac{t_{\text{travel}}(t)}{t_{\text{freelane}}} - 1$ là Congestion Index.

Multi-level Thresholds:

- **Speed-based:**

$$v_{\text{rel}}(t) < \tau_v \Rightarrow \text{Congested}$$

với $\tau_v = 0.6$ (tốc độ dưới 60% tốc độ tự do)

- **Trend-based:**

$$\frac{dv}{dt} < -\tau_{\Delta v} \Rightarrow \text{Deteriorating}$$

Phát hiện xu hướng giảm tốc độ nhanh

4.7.1.b Statistical Anomaly Detection

Z-score Method:

$$z_i(t) = \frac{v_i(t) - \mu_i}{\sigma_i}$$

$$\text{Anomaly if } |z_i(t)| > \theta_z$$

với $\theta_z = 3$ (3-sigma rule).

Isolation Forest:

Sử dụng ensemble of random trees để phát hiện điểm bất thường:

- Anomalies có average path length ngắn hơn

- Anomaly score:

$$s(x, n) = 2^{-\frac{E(h(x))}{c(n)}}$$

với $h(x)$ là path length, $c(n)$ là normalization factor

4.7.2 Mô hình lan truyền

4.7.2.a Cascade Prediction Model

Khi phát hiện tắc nghẽn tại nút i ở thời điểm t_0 , dự đoán xác suất lan truyền:

$$P(j \text{ congested at } t | i \text{ congested at } t_0) = f(A_{ij}, \tau_{ij}, \mathbf{h}_j(t)) \quad (31)$$

với:

- A_{ij} : causal influence strength từ i đến j
- τ_{ij} : propagation delay
- $\mathbf{h}_j(t)$: current state của nút j

Logistic Regression Model:

$$P(y_j = 1 | \mathbf{x}) = \frac{1}{1 + \exp(-(\beta_0 + \beta_1 A_{ij} + \beta_2 v_j + \beta_3 d_{ij}))}$$

Neural Network Classifier:

Input features:

- Causal influence A_{ij}
- Current speed $v_j(t)$
- Distance d_{ij}
- Time of day
- Historical congestion patterns

Output: Probability of congestion at different time horizons ($t + 15\text{min}$, $t + 30\text{min}$, $t + 60\text{min}$).

4.7.2.b Graph-based Propagation Simulation

Sử dụng causal graph để mô phỏng lan truyền:

Algorithm 1 Congestion Propagation Simulation

Require: Causal graph G , initial congested node i_0 , time horizon T

Ensure: Set of affected nodes at each time step

```
1: Congested[0]  $\leftarrow \{i_0\}$ 
2: for  $t = 1$  to  $T$  do
3:   Congested[ $t$ ]  $\leftarrow$  Congested[ $t - 1$ ]
4:   for all  $i \in$  Congested[ $t - 1$ ] do
5:     for all  $j$  such that  $A_{ij} > 0$  do
6:       if  $t - t_0 \geq \tau_{ij}$  and  $P_{\text{spread}}(i \rightarrow j) > \theta$  then
7:         Congested[ $t$ ]  $\leftarrow$  Congested[ $t$ ]  $\cup \{j\}$ 
8:       end if
9:     end for
10:  end for
11: end for
12: return Congested[1 :  $T$ ]
```

4.7.3 Hệ thống cảnh báo đa cấp

4.7.3.a Alert Severity Levels

Bảng 5: *Mức độ cảnh báo*

Level	Condition	Action
Info	Moderate congestion, localized	Log and monitor
Warning	Heavy congestion, risk of spreading	Alert operators
Critical	Severe congestion, cascading	Immediate intervention
Emergency	Multiple critical nodes, network failure	Emergency protocol

4.7.3.b Alert Content

Mỗi alert bao gồm:

- **Source Node:** Nút gốc bị tắc nghẽn
- **Affected Nodes:** Danh sách nút có nguy cơ bị ảnh hưởng
- **Estimated Impact Time:** Thời điểm dự kiến tắc nghẽn lan đến từng nút
- **Severity Score:** Mức độ nghiêm trọng tổng hợp
- **Recommended Actions:** Các biện pháp đề xuất
- **Confidence Level:** Độ tin cậy của dự báo

4.7.3.c Alert Routing

- **Dashboard:** Real-time visualization cho control center
- **Mobile App:** Push notifications cho traffic officers



- **API:** Integration với các hệ thống khác (VMS, traffic lights)
- **Email/SMS:** Notifications cho stakeholders

4.8 Đánh giá và metrics

4.8.1 Metrics cho dự báo

4.8.1.a Point Prediction Metrics

1. Mean Absolute Error (MAE):

$$\text{MAE} = \frac{1}{N \cdot H} \sum_{i=1}^N \sum_{h=1}^H |y_i^{(t+h)} - \hat{y}_i^{(t+h)}|$$

Đo lường sai số trung bình, ít bị ảnh hưởng bởi outliers.

2. Root Mean Squared Error (RMSE):

$$\text{RMSE} = \sqrt{\frac{1}{N \cdot H} \sum_{i=1}^N \sum_{h=1}^H (y_i^{(t+h)} - \hat{y}_i^{(t+h)})^2}$$

Penalize lỗi lớn mạnh hơn MAE.

3. Mean Absolute Percentage Error (MAPE):

$$\text{MAPE} = \frac{100\%}{N \cdot H} \sum_{i=1}^N \sum_{h=1}^H \left| \frac{y_i^{(t+h)} - \hat{y}_i^{(t+h)}}{y_i^{(t+h)}} \right|$$

Đo lường sai số tương đối, dễ interpret.

4. R^2 Score:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Tỷ lệ variance được giải thích bởi mô hình.

4.8.1.b Distribution Prediction Metrics

Cho probabilistic predictions:

1. Continuous Ranked Probability Score (CRPS):

$$\text{CRPS}(F, y) = \int_{-\infty}^{\infty} (F(x) - \mathbb{1}_{\{x \geq y\}})^2 dx$$

với F là cumulative distribution function của prediction.

2. Prediction Interval Coverage Probability (PICP):

$$\text{PICP} = \frac{1}{N} \sum_{i=1}^N \mathbb{1}\{y_i \in [L_i, U_i]\}$$

Tỷ lệ true values nằm trong prediction interval $[L_i, U_i]$.

4.8.2 Metrics cho phát hiện tương quan

4.8.2.a Correlation Detection Accuracy

So sánh với ground truth (nếu có) hoặc expert knowledge:

1. Precision và Recall:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

với:

- TP: Correctly identified correlations
- FP: Falsely identified correlations
- FN: Missed correlations

2. Area Under ROC Curve (AUC-ROC):

Đánh giá khả năng phân biệt giữa correlated và uncorrelated pairs.

4.8.2.b Causal Discovery Accuracy

1. Structural Hamming Distance (SHD):

$$\text{SHD}(G_{\text{true}}, G_{\text{pred}}) = |E_{\text{true}} \triangle E_{\text{pred}}| + |E_{\text{reversed}}|$$

Đếm số edges sai khác (missing, extra, reversed).

2. F1 Score for Causal Edges:

Tính precision, recall trên directed edges.

4.8.3 Metrics cho cảnh báo

4.8.3.a Alert Quality Metrics

1. True Positive Rate (Detection Rate):

$$\text{TPR} = \frac{\# \text{ correct alerts}}{\# \text{ actual events}}$$

2. False Positive Rate:

$$\text{FPR} = \frac{\# \text{ false alerts}}{\# \text{ non-events}}$$

3. Lead Time:

$$\text{Lead Time} = t_{\text{alert}} - t_{\text{event}}$$

Thời gian cảnh báo trước khi sự kiện xảy ra (càng lớn càng tốt, nhưng không quá lớn để tránh false alarms).

4. Precision at k:

$$\text{P@k} = \frac{\# \text{ relevant nodes in top-k predictions}}{k}$$

Đo lường accuracy của ranked list các nút có nguy cơ.

4.8.4 Baseline Models

So sánh với các baseline để chứng minh hiệu quả:

(a) Historical Average (HA):

$$\hat{y}_i(t) = \frac{1}{n} \sum_{d=1}^n y_i(t - d \cdot T)$$

Dự báo dựa trên trung bình cùng thời điểm các ngày trước.

(b) **ARIMA**: Mô hình thống kê truyền thống cho time series.

(c) **LSTM**: LSTM thuần không có graph structure.

(d) **Standard T-GCN**: T-GCN với static adjacency matrix.

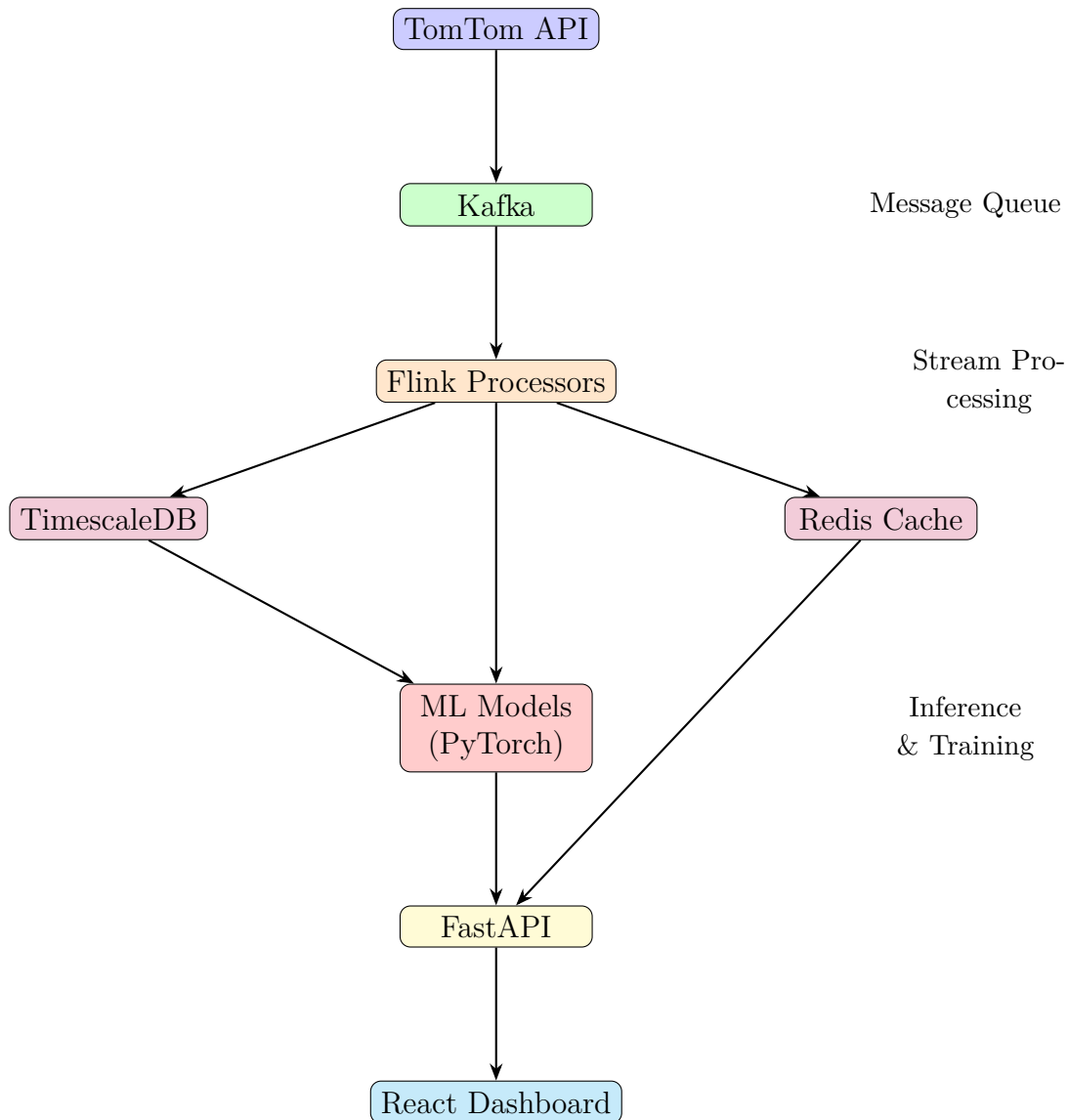
(e) **DCRNN**: Diffusion Convolutional Recurrent Neural Network.

(f) **Graph WaveNet**: State-of-the-art model với adaptive adjacency matrix.

4.9 Triển khai và tích hợp hệ thống

4.9.1 Kiến trúc hệ thống thực tế

4.9.1.a System Components



Hình 5: Kiến trúc hệ thống triển khai phân tích và dự báo giao thông

4.9.1.b Data Flow

Real-time Pipeline:

- (a) TomTom API → Kafka topic `traffic.raw`
- (b) Flink job: Data validation → Kafka topic `traffic.validated`
- (c) Flink job: Feature extraction → Kafka topic `traffic.features`

- (d) Flink job: Model inference → Kafka topic `traffic.predictions`
- (e) Alert service consumes predictions → Generate alerts
- (f) WebSocket pushes updates → Dashboard

Batch Pipeline:

- (a) Scheduled job retrieves historical data from TimescaleDB
- (b) Compute correlation matrices and causal graphs
- (c) Retrain models weekly with new data
- (d) Update model artifacts in production

4.9.2 Containerization và Deployment

4.9.2.a Docker Compose Setup

Services definition:

- **zookeeper**: Coordination service
- **kafka**: Message broker (3 brokers for HA)
- **postgres**: Database with PostGIS and TimescaleDB extensions
- **redis**: Cache and pub/sub
- **flink-jobmanager**: Flink cluster manager
- **flink-taskmanager**: Flink workers (scalable)
- **ml-service**: ML inference service (GPU-enabled)
- **api**: FastAPI backend
- **frontend**: React SPA (Nginx)
- **prometheus**: Metrics collection
- **grafana**: Monitoring dashboard

4.9.2.b Kubernetes Deployment

Cho production với high availability:

- **StatefulSets**: Kafka, PostgreSQL, Redis
- **Deployments**: API, ML service, Frontend
- **DaemonSets**: Log collectors, monitoring agents

- **HorizontalPodAutoscaler:** Auto-scaling based on CPU/memory
- **Ingress:** Load balancing và SSL termination
- **PersistentVolumes:** Data persistence

4.9.3 Monitoring và Maintenance

4.9.3.a Monitoring Stack

1. System Metrics:

- CPU, memory, disk, network usage
- Container health checks
- Service uptime and restarts

2. Application Metrics:

- API request rate, latency, error rate
- Kafka consumer lag
- Flink job throughput, backpressure
- Database query performance

3. ML Metrics:

- Model inference latency
- Prediction accuracy (online evaluation)
- Data drift detection
- Model version tracking

4. Business Metrics:

- Number of alerts generated
- Alert accuracy rate
- User engagement (dashboard views, API calls)

4.9.3.b Alerting Rules

Prometheus alerting rules:

- **High Error Rate:** API error rate $> 5\%$ for 5 minutes

- **High Latency:** 95th percentile latency $> 1s$ for 5 minutes
- **Consumer Lag:** Kafka consumer lag > 10000 messages
- **Model Degradation:** MAE increases by $> 20\%$ compared to baseline
- **Service Down:** Any critical service down for > 1 minute

4.9.3.c Maintenance Procedures

1. Model Retraining:

- Weekly retraining with latest data
- A/B testing new models against production
- Gradual rollout with canary deployment

2. Database Maintenance:

- Daily backups with point-in-time recovery
- Weekly vacuum and analyze for PostgreSQL
- Compression of old data in TimescaleDB
- Retention policy: 1 year of detailed data, then aggregated

3. Log Management:

- Centralized logging với ELK stack
- Log rotation và archival
- Retention: 30 days online, 1 year archived

4.10 Kết chương

Chương này đã trình bày chi tiết phương pháp luận toàn diện để giải quyết bài toán phân tích mối liên hệ giao thông tại TP.HCM. Các đóng góp chính của phương pháp đề xuất bao gồm:

4.10.1 Về mặt dữ liệu

- Sử dụng hiệu quả dữ liệu lịch sử từ TomTom API với quy trình thu thập và tiền xử lý chuẩn hóa
- Chiến lược xử lý missing values và outliers phù hợp với đặc thù dữ liệu giao thông
- Trích xuất đặc trưng phong phú ở nhiều khía cạnh: không gian, thời gian, động học

4.10.2 Về mặt phân tích

- Áp dụng nhiều phương pháp phân tích tương quan từ cơ bản đến nâng cao (Pearson, DTW, Wavelet Coherence)
- Sử dụng Granger Causality Test để phân biệt tương quan và quan hệ nhân quả
- Xây dựng causal graph thể hiện mối quan hệ lan truyền giữa các nút
- Xác định propagation delay và direction of influence

4.10.3 Về mặt mô hình hóa

- Đề xuất Dynamic GCN với ma trận kề học được và thay đổi theo thời gian, khắc phục hạn chế của T-GCN truyền thống
- Tích hợp multi-scale temporal modeling để nắm bắt patterns ở nhiều time scales
- Kết hợp cơ chế attention để học được importance của các nút láng giềng
- Training strategy với regularization, ensemble, và uncertainty quantification

4.10.4 Về mặt ứng dụng

- Hệ thống cảnh báo đa cấp với khả năng dự đoán lan truyền tắc nghẽn
- Cascade prediction model mô phỏng sự lan rộng của congestion
- Metrics toàn diện để đánh giá cả prediction accuracy và causal discovery
- Kiến trúc hệ thống production-ready với Kappa architecture, containerization, và monitoring

4.10.5 Tính mới và đóng góp

So với các nghiên cứu hiện có, phương pháp đề xuất có những điểm nổi bật:

- (a) **Dynamic graph learning:** Không giả định ma trận kề cố định mà học từ dữ liệu và cập nhật theo thời gian
- (b) **Causal analysis integration:** Kết hợp statistical causality testing với deep learning để không chỉ dự báo mà còn giải thích được cơ chế lan truyền
- (c) **Multi-method approach:** Tích hợp nhiều kỹ thuật từ các lĩnh vực khác nhau (graph theory, time series analysis, deep learning, statistical inference) trong một framework thống nhất
- (d) **Practical system design:** Không chỉ đề xuất mô hình lý thuyết mà còn thiết kế hệ thống hoàn chỉnh có thể triển khai thực tế với data pipeline, monitoring, và alerting
- (e) **Focus on interpretability:** Nhấn mạnh khả năng giải thích kết quả (explainability) thông qua causal graph và propagation analysis, phục vụ nhu cầu ra quyết định của nhà quản lý

Phương pháp này tạo nền tảng vững chắc cho các chương tiếp theo, nơi sẽ trình bày chi tiết về hiện thực hóa, kết quả thực nghiệm, và đánh giá hiệu quả của hệ thống.

5 Hệ thống đề xuất

5.1 Mô tả hệ thống

Hệ thống được thiết kế nhằm hỗ trợ phân tích và đánh giá tình trạng giao thông (TTGT) trong môi trường đô thị thông qua việc xử lý, tổ chức và phân tích dữ liệu giao thông được cung cấp. Hệ thống cho phép mô hình hóa mạng lưới giao thông, nhận diện các mối quan hệ giữa các đoạn đường và khai thác dữ liệu theo cả hai khía cạnh không gian và thời gian để hỗ trợ đánh giá và hiểu rõ hơn về tình trạng lưu thông.

5.1.1 Mục đích của hệ thống

Mục tiêu chính của hệ thống bao gồm:

- Quản lý dữ liệu giao thông từ nhiều nguồn (dữ liệu bản đồ số, dữ liệu cảm biến, dữ liệu GPS, dữ liệu từ các nền tảng giao thông của bên thứ ba, v.v.).
- Tiền xử lý, chuyển đổi và chuẩn hóa dữ liệu để phục vụ phân tích.
- Mô hình hóa mối liên hệ giữa các đoạn đường trong mạng lưới giao thông nhằm hiểu rõ cấu trúc và dòng chảy giao thông.
- Phân tích dữ liệu để hỗ trợ nhận diện tình trạng ùn tắc, đánh giá xu hướng và hỗ trợ ra quyết định.
- Trực quan hóa kết quả phân tích bằng biểu đồ, bản đồ hoặc báo cáo nhằm giúp người dùng theo dõi và hiểu kết quả.
- Cung cấp nền tảng linh hoạt để thử nghiệm, so sánh hoặc tích hợp các phương pháp phân tích trong tương lai.

5.1.2 Nhóm người dùng mục tiêu

Hệ thống hướng đến ba nhóm người dùng chính:

- **Nhà nghiên cứu và sinh viên:** sử dụng hệ thống để phân tích dữ liệu giao thông, thử nghiệm các phương pháp mô hình hóa khác nhau và đánh giá các yếu tố ảnh hưởng đến TTGT.
- **Cơ quan quản lý giao thông hoặc nhà quy hoạch đô thị:** khai thác hệ thống để theo dõi tình trạng giao thông, phát hiện điểm nóng ùn tắc và đánh giá tác động của các giải pháp tổ chức giao thông.
- **Kỹ sư dữ liệu và nhà phát triển hệ thống:** sử dụng hệ thống như nền tảng để xử lý dữ liệu giao thông, mở rộng mô hình, tích hợp thêm tính năng hoặc triển khai vào môi trường thực tế.

5.1.3 Tổng quan

Tổng thể, hệ thống được thiết kế theo hướng mở, linh hoạt. Điều này cho phép hệ thống dễ dàng mở rộng, thay đổi hoặc kết hợp các phương pháp phân tích khi cần thiết, đồng thời đảm bảo phù hợp cho cả mục tiêu nghiên cứu lẫn ứng dụng thực tiễn trong phân tích tình trạng giao thông đô thị.

5.2 Yêu cầu

5.2.1 Yêu cầu chức năng

5.2.1.1 Thu thập dữ liệu giao thông

- **FR1.1:** Hệ thống cho phép tải lên dữ liệu lịch sử giao thông.
- **FR1.2:** Hỗ trợ nhiều định dạng dữ liệu đầu vào: JSON, GeoJSON, CSV.
- **FR1.3:** Kiểm tra cấu trúc và tính hợp lệ của dữ liệu (schema validation).

5.2.1.2 Tiền xử lý dữ liệu

- **FR2.1:** Loại bỏ dữ liệu thiếu, sai định dạng hoặc bất thường.
- **FR2.2:** Chuẩn hoá các thuộc tính như tốc độ, thời gian di chuyển, timestamp.
- **FR2.3:** Ghép nối dữ liệu với mạng lưới đường (map-matching) khi cần.

5.2.1.3 Phân tích mạng lưới giao thông

- **FR3.1:** Xây dựng mô hình đồ thị gồm các nút (node) và cạnh (segment).
- **FR3.2:** Tính toán các thông số của mạng đường: độ dài, phân cấp đường (FRC), các chỉ số trung tâm.
- **FR3.3:** Xác định “điểm nóng” (hotspot) tắc nghẽn theo thời gian.

5.2.1.4 Phân tích mối liên hệ giữa các đoạn đường

- **FR4.1:** Tính toán mức độ tương quan giữa các đoạn đường theo thời gian.
- **FR4.2:** Đo độ tương đồng dạng thời gian bằng kỹ thuật Dynamic Time Warping (DTW).
- **FR4.3:** Phát hiện hiện tượng lan truyền tắc nghẽn trong mạng lưới.

5.2.1.5 Dự báo tình trạng giao thông

- **FR5.1:** Xây dựng tập dữ liệu theo chuỗi thời gian (time-series).
- **FR5.2:** Huấn luyện mô hình dự báo như LSTM, GCN, T-GCN,...
- **FR5.3:** Đánh giá độ chính xác mô hình bằng các thước đo MAE, RMSE.

5.2.1.6 Trực quan hoá dữ liệu

- **FR6.1:** Hiển thị bản đồ mạng giao thông bằng dữ liệu OSM hoặc TomTom.
- **FR6.2:** Trực quan tốc độ trung bình theo thời gian của từng segment.
- **FR6.3:** Hiển thị biểu đồ chuỗi thời gian.
- **FR6.4:** Hiển thị ma trận tương quan giữa các đoạn đường.
- **FR6.5:** Cho phép thao tác xem dữ liệu trên dashboard (ví dụ: Streamlit).

5.2.1.7 Xuất báo cáo

- **FR7.1:** Xuất kết quả phân tích ra các định dạng CSV, JSON hoặc PDF.
- **FR7.2:** Cho phép tải xuống kết quả.

5.2.2 Yêu cầu phi chức năng

5.2.2.1 Khả năng mở rộng

- Hệ thống phải có khả năng xử lý dữ liệu lớn lên đến hàng triệu bản ghi, đồng thời có thể mở rộng mà không suy giảm hiệu năng.

5.2.2.2 Hiệu năng

- Thời gian xử lý dữ liệu phải đảm bảo hiệu quả (ví dụ: xử lý 1 MB dữ liệu trong vòng dưới 5 giây).
- Thời gian tải và hiển thị biểu đồ phải dưới 5 giây.

5.2.2.3 Độ tin cậy

- Hệ thống phải hoạt động ổn định và cho kết quả nhất quán khi sử dụng cùng một bộ dữ liệu.

5.2.2.4 Tính bảo trì

- Mã nguồn phải được tổ chức thành các module độc lập gồm: thu thập dữ liệu, tiền xử lý, phân tích, mô hình hoá và trực quan hoá.

5.2.2.5 Tính tương thích

- Hệ thống phải hoạt động tốt với Python 3.10+, các thư viện phân tích (Pandas, GeoPandas, NetworkX), TomTom MOVE và dữ liệu OSM.

5.2.2.6 Tính dễ sử dụng

- Giao diện và quy trình thao tác phải đơn giản, giúp người dùng dễ dàng tải dữ liệu, xem biểu đồ và tạo báo cáo.

5.2.2.7 An toàn dữ liệu

- Không thu thập hoặc xử lý dữ liệu định danh cá nhân. Dữ liệu giao thông chỉ được xử lý ở mức tổng hợp.

5.2.3 Yêu cầu dữ liệu

5.2.3.a Loại dữ liệu cần thu thập

Hệ thống cần thu thập dữ liệu mô tả tình trạng giao thông đô thị từ một hoặc nhiều nguồn khác nhau (API giao thông, cảm biến, dữ liệu mở, dữ liệu bản đồ số, v.v.). Các nhóm dữ liệu chính bao gồm:

- **Dữ liệu cấu trúc mạng lưới giao thông:**
 - Thông tin các đoạn đường (segment).
 - Mã định danh đoạn đường.
 - Tên đường, khu vực, vùng hoặc zone.
 - Thông tin bản đồ nền và phiên bản bản đồ.
- **Dữ liệu thời gian:**
 - Thời điểm thu thập dữ liệu.
 - Khoảng ngày quan sát (*date ranges*).
 - Khoảng thời gian trong ngày (*time sets*).
- **Dữ liệu tình trạng giao thông:**
 - Tốc độ trung bình, tốc độ tự do.

- Mức độ chậm trễ, chỉ số ùn tắc.
- Điều kiện lưu thông trên từng đoạn đường.
- Nguồn thu thập (GPS, cảm biến, v.v.) nếu có.

- **Thông tin cấu hình và siêu dữ liệu:**

- Tên phiên thu thập hoặc mã job.
- Đơn vị đo vận tốc.
- Các lựa chọn cấu hình do người dùng đặt.
- Các tham số được sử dụng để tạo bộ dữ liệu.

5.2.3.b Yêu cầu về định dạng và lưu trữ dữ liệu

- Dữ liệu đầu vào có thể ở dạng JSON, CSV hoặc các định dạng bán cấu trúc khác, tùy theo nguồn thu thập.
- Hệ thống cần hỗ trợ chuyển đổi sang dạng bảng (tabular) để phân tích.
- Dữ liệu được lưu trữ ở các định dạng thuận tiện cho xử lý như CSV, Parquet...
- Hỗ trợ nhập dữ liệu từ nhiều nguồn: file tải lên, API hoặc kho lưu trữ trên nền tảng đám mây.

5.2.3.c Yêu cầu về chất lượng dữ liệu

- **Đầy đủ:** có đủ thông tin tối thiểu để phân tích tình trạng giao thông.
- **Nhất quán:** đồng nhất về đơn vị đo, định dạng thời gian và mã định danh.
- **Đúng cấu trúc:** các trường phải đúng kiểu dữ liệu.
- **Giảm thiểu dữ liệu lỗi hoặc thiếu:** hạn chế phần tử bị trống hoặc sai lệch.
- **Có thể truy vết:** biết rõ nguồn và thời điểm thu thập.

5.2.3.d Yêu cầu về cập nhật dữ liệu

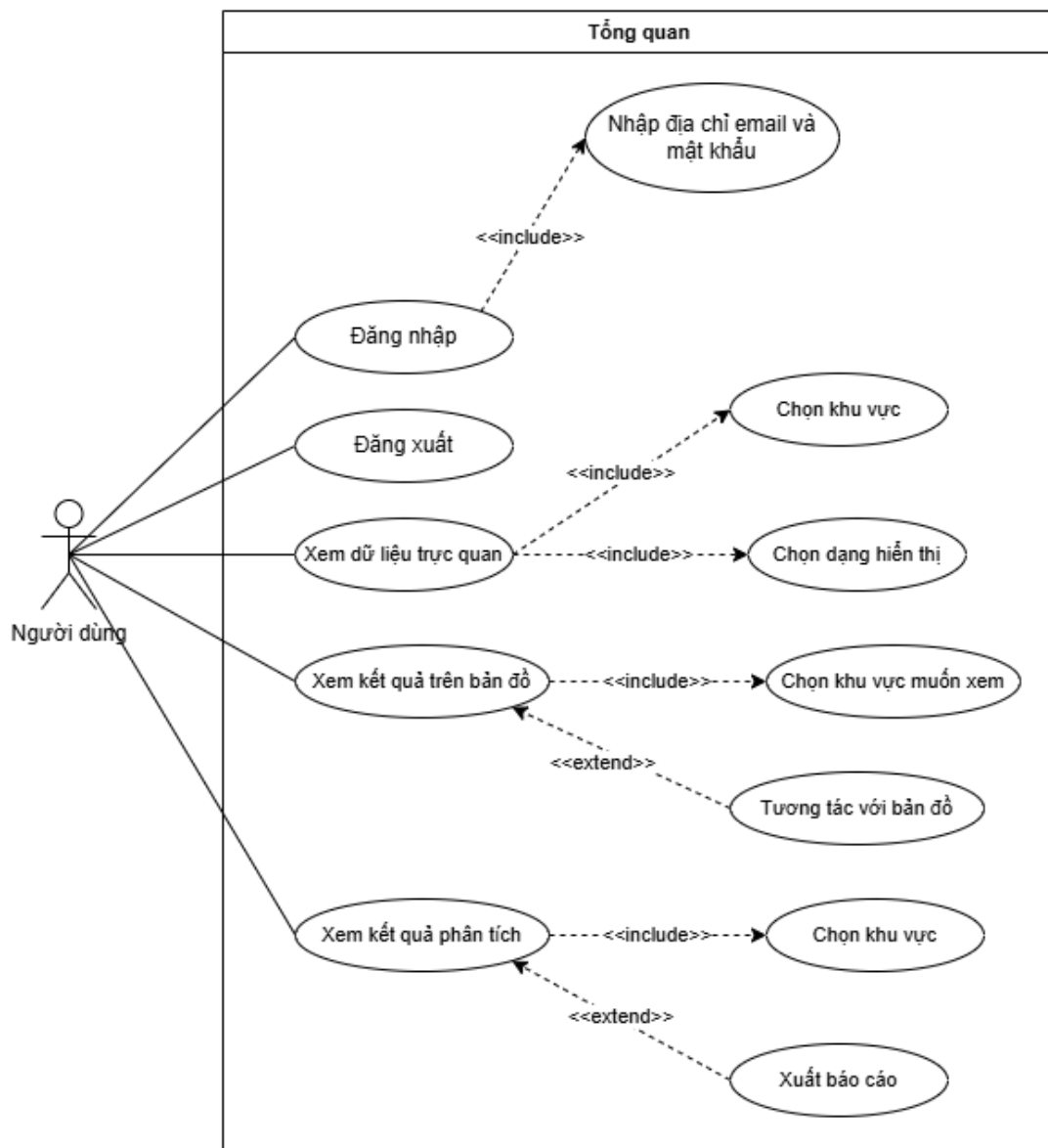
- Hệ thống cần hỗ trợ nhập và cập nhật dữ liệu từ nhiều nguồn khác nhau trong tương lai.
- Cho phép cập nhật định kỳ khi có dữ liệu mới.

6 Phân tích và Thiết kế

6.1 Phân tích

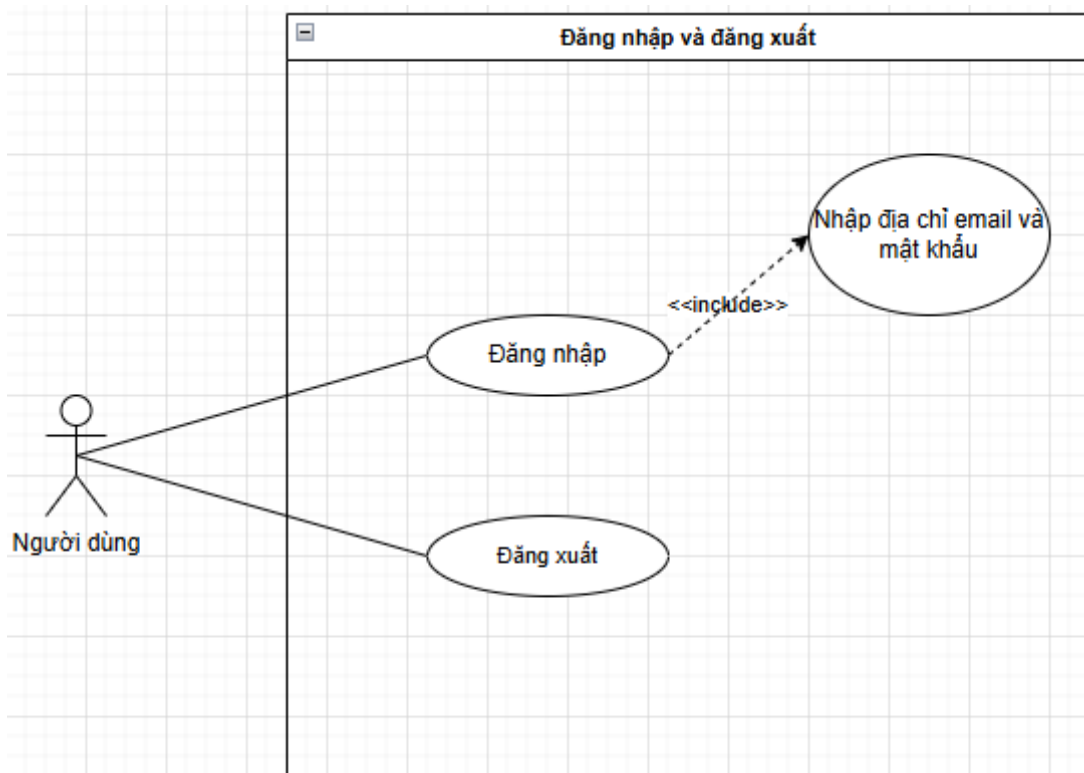
6.1.1 Mô hình hóa tương tác

6.1.1.a Tổng quan



Hình 6: Sơ đồ Use Case tổng quan

6.1.1.b Đăng nhập và đăng xuất

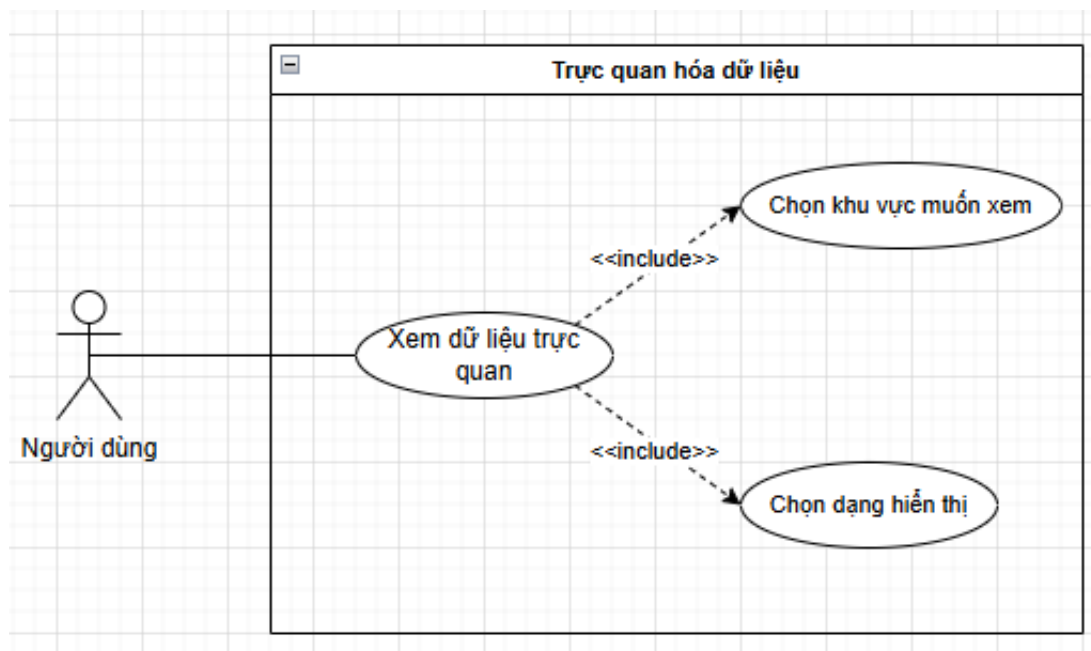


Hình 7: Sơ đồ Use Case đăng nhập và đăng xuất

Tên use case	Đăng nhập
Mô tả	Cho phép người dùng (đã có tài khoản) truy cập vào hệ thống bằng thông tin xác thực của họ.
Tác nhân	Người dùng (đã có tài khoản).
Điều kiện kích hoạt	Người dùng ở trạng thái chưa đăng nhập nhấn vào nút “Đăng nhập”.
Điều kiện tiên quyết	Người dùng đang ở trạng thái chưa đăng nhập nhưng đã có tài khoản.
Điều kiện sau hành động	Nếu thành công, người dùng được xác thực và chuyển đến trang tài khoản. Nếu thất bại, hệ thống hiển thị lỗi.
Luồng chính	1. Người dùng nhấp vào giao diện “Đăng nhập”. 2. Nhập email và mật khẩu. 3. Nhấn “Đăng nhập”. 4. Hệ thống kiểm tra và xác thực thông tin. 5. Hệ thống tạo phiên đăng nhập và chuyển người dùng đến trang chủ.
Luồng rẽ nhánh	5a. Email hoặc mật khẩu không chính xác dẫn đến hệ thống báo lỗi và yêu cầu nhập lại.
Kết quả	Phiên đăng nhập được tạo hoặc hiển thị lỗi.

Tên use case	Đăng xuất
Mô tả	Cho phép người dùng (đã đăng nhập) thoát ra khỏi tài khoản đang đăng nhập.
Tác nhân	Người dùng
Điều kiện kích hoạt	Người dùng nhấn vào nút “Đăng xuất”.
Điều kiện tiên quyết	Người dùng đang trong trạng thái đăng nhập.
Điều kiện sau hành động	Phiên đăng nhập bị hủy và chuyển hướng về trang đăng nhập (trạng thái chưa đăng nhập).
Luồng chính	1. Người dùng nhấn nút “Đăng xuất”. 2. Hệ thống hủy phiên đăng nhập. 3. Hệ thống xóa thông tin đăng nhập (cookies, local storage nếu có). 4. Chuyển hướng người dùng về trang đăng nhập.
Luồng rẽ nhánh	Không có.
Kết quả	Người dùng trở về trạng thái chưa đăng nhập.

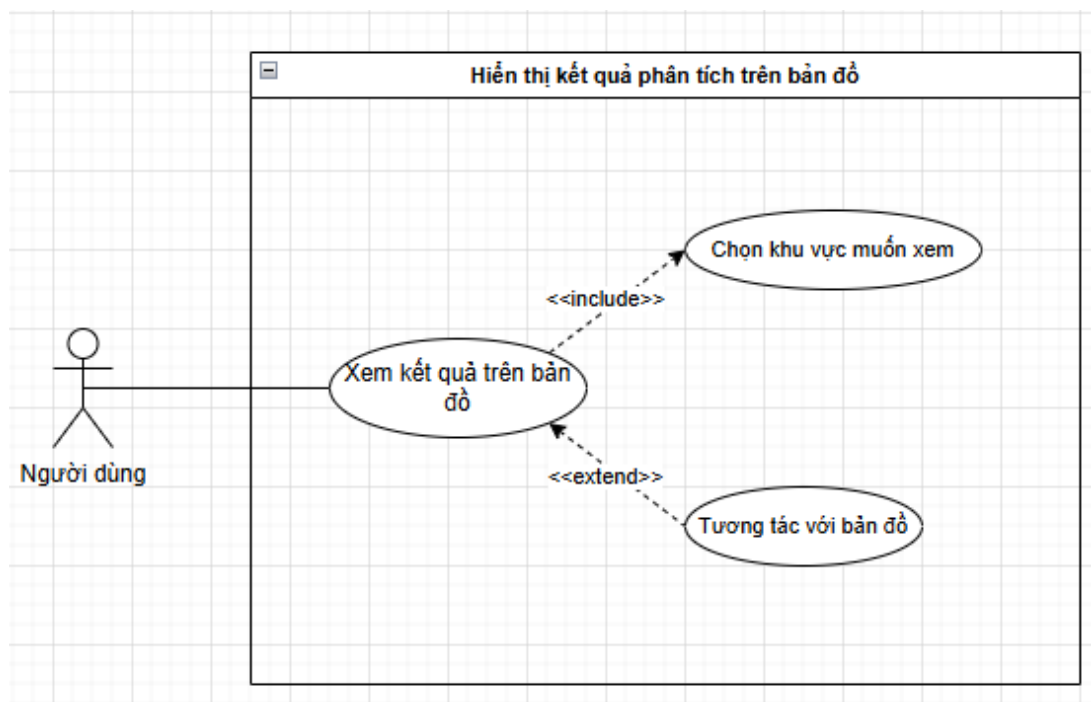
6.1.1.c Trực quan hóa dữ liệu



Hình 8: Sơ đồ Use Case trực quan hóa dữ liệu

Tên use case	Trực quan hóa dữ liệu
Mô tả	Cho phép nhà phân tích quan sát dữ liệu giao thông dưới dạng biểu đồ như: biểu đồ thời gian, biểu đồ phân phối, heatmap, hoặc biểu đồ so sánh nhằm hỗ trợ việc phân tích.
Tác nhân	Người dùng.
Điều kiện kích hoạt	Người dùng yêu cầu xem dữ liệu hoặc xem kết quả xử lý dữ liệu ở dạng biểu đồ.
Điều kiện tiên quyết	Dữ liệu giao thông đã được tiền xử lý và sẵn sàng để hiển thị.
Điều kiện sau hành động	Biểu đồ hoặc hình ảnh trực quan được tạo và hiển thị để phục vụ phân tích.
Luồng chính	1. Người dùng chọn khu vực muốn xem (có thể là đoạn đường, phường hoặc quận). 2. Chọn loại biểu đồ muốn hiển thị. 3. Hệ thống xử lý và tạo biểu đồ tương ứng. 4. Biểu đồ được hiển thị cho nhà phân tích xem hoặc lưu lại.
Luồng rẽ nhánh	3a. Nếu dữ liệu thiếu hoặc không hợp lệ, hệ thống sẽ hiển thị thông báo lỗi.
Kết quả	Biểu đồ hiển thị thành công hoặc thông báo lỗi nếu dữ liệu không hợp lệ.

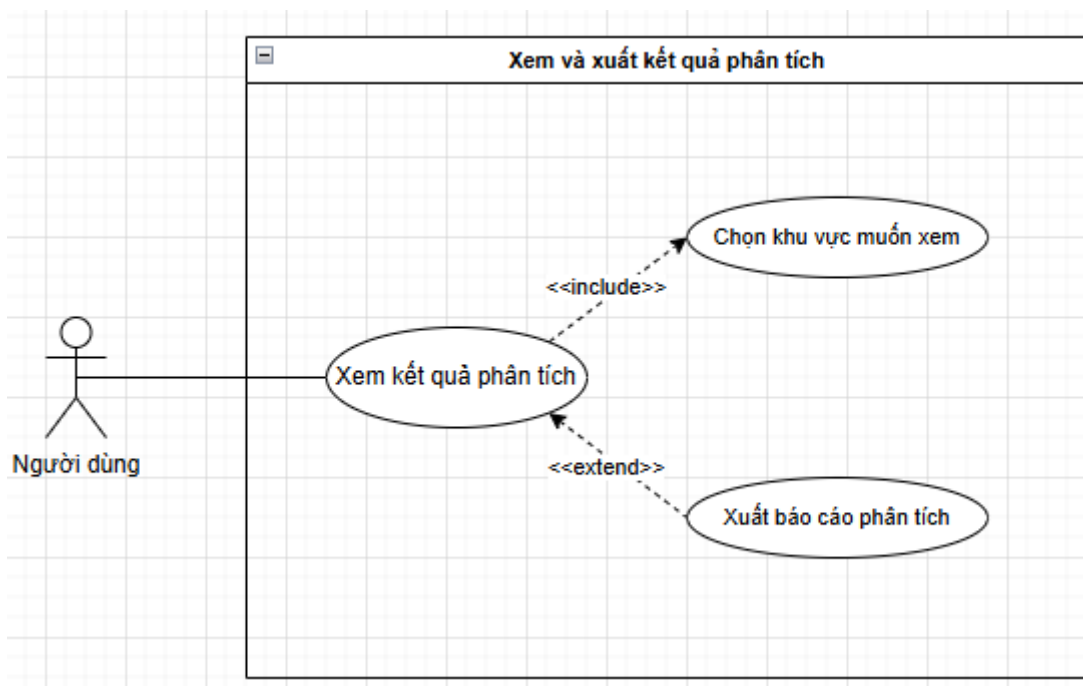
6.1.1.d Hiển thị kết quả phân tích trên bản đồ



Hình 9: Sơ đồ Use Case hiển thị kết quả phân tích trên bản đồ

Tên use case	Hiển thị kết quả phân tích trên bản đồ
Mô tả	Cho phép người dùng quan sát kết quả phân tích giao thông trực tiếp trên bản đồ, bao gồm tô màu đoạn đường theo tốc độ, mức độ ùn tắc hoặc các thông số phân tích khác.
Tác nhân	Người dùng.
Điều kiện kích hoạt	Người dùng yêu cầu xem kết quả phân tích trên nền bản đồ.
Điều kiện tiên quyết	Dữ liệu có chứa thông tin không gian (đoạn đường, tọa độ, hoặc dạng network).
Điều kiện sau hành động	Hiển thị bản đồ với các tuyến đường được tô màu theo giá trị tương ứng; nhà phân tích có thể quan sát và ghi nhận kết quả.
Luồng chính	1. Người dùng chọn khu vực cần hiển thị. 2. Hệ thống ánh xạ dữ liệu lên bản đồ. 3. Bản đồ được tạo và tô màu theo tốc độ hoặc mức độ ùn tắc. 4. Người dùng tương tác, xem chi tiết hoặc lưu hình ảnh bản đồ.
Luồng rẽ nhánh	2a. Thiếu dữ liệu vị trí, hệ thống không thể tạo bản đồ và hiển thị thông báo lỗi.
Kết quả	Bản đồ được hiển thị và sẵn sàng phục vụ phân tích.

6.1.1.e Xem và xuất báo cáo kết quả phân tích

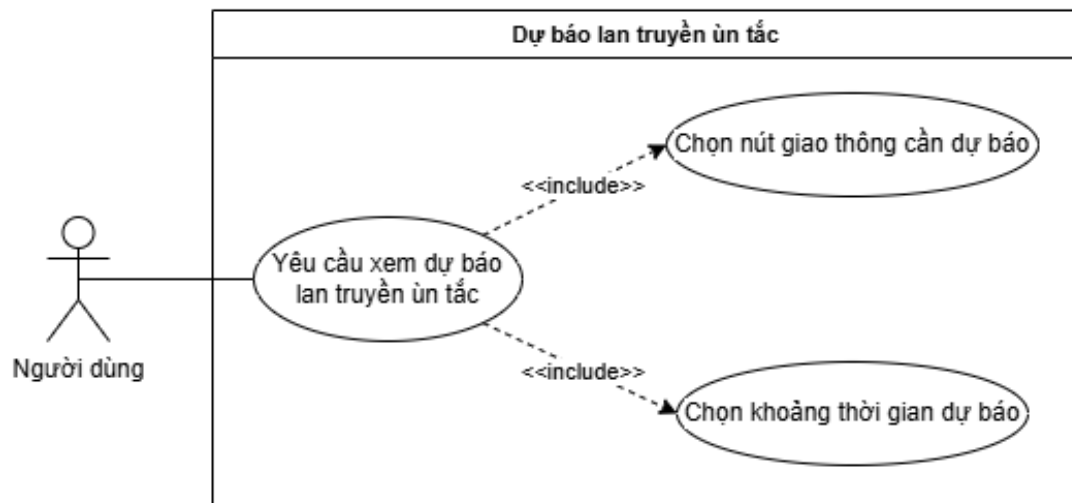


Hình 10: Sơ đồ Use Case xem và xuất kết quả phân tích

Tên use case	Xem kết quả phân tích
Mô tả	Cho phép người dùng xem các kết quả phân tích tình trạng giao thông, bao gồm các biểu đồ, bản đồ trực quan, thống kê hoặc chỉ số được hệ thống tạo ra từ dữ liệu giao thông đã xử lý.
Tác nhân	Người dùng.
Điều kiện kích hoạt	Người dùng truy cập vào mục xem kết quả phân tích hoặc chọn một khu vực để hiển thị kết quả.
Điều kiện tiên quyết	Hệ thống đã xử lý dữ liệu và tạo ra kết quả phân tích tương ứng.
Điều kiện sau hành động	Kết quả phân tích được hiển thị dưới dạng biểu đồ, bản đồ hoặc số liệu thống kê.
Luồng chính	1. Người dùng truy cập chức năng xem kết quả phân tích. 2. Người dùng chọn khu vực muốn xem. 3. Hệ thống truy xuất kết quả phân tích tương ứng từ cơ sở dữ liệu hoặc mô-đun xử lý. 4. Hệ thống hiển thị kết quả dưới dạng biểu đồ, bản đồ hoặc bảng số liệu.
Luồng rẽ nhánh	3a. Nếu chưa có kết quả phân tích cho khu vực đã chọn, hệ thống thông báo "Chưa có dữ liệu phân tích".
Kết quả	Người dùng xem được kết quả phân tích hoặc nhận thông báo nếu dữ liệu không có sẵn.

Tên use case	Xuất báo cáo phân tích
Mô tả	Cho phép nhà phân tích xuất các kết quả phân tích (biểu đồ, bảng số liệu, bản đồ, thống kê) thành tệp tin như PDF, CSV hoặc hình ảnh phục vụ trình bày hoặc báo cáo nghiên cứu.
Tác nhân	Người dùng.
Điều kiện kích hoạt	Người dùng yêu cầu lưu lại kết quả phân tích dưới dạng tệp.
Điều kiện tiên quyết	Kết quả phân tích đã được tạo và sẵn sàng để xuất file.
Điều kiện sau hành động	Hệ thống tạo file báo cáo hoặc hình ảnh và cung cấp cho người dùng để tải xuống.
Luồng chính	1. Người dùng chọn loại file báo cáo muốn xuất (PDF, CSV, hình ảnh...). 2. Hệ thống tổng hợp dữ liệu và biểu đồ liên quan. 3. Hệ thống tạo tệp báo cáo. 4. Người dùng tải xuống tệp.
Luồng rẽ nhánh	2a. Thiếu dữ liệu đầu vào, hệ thống không thể tạo báo cáo và thông báo lỗi.
Kết quả	Tệp báo cáo được tạo thành công hoặc hiển thị lỗi nếu thất bại.

6.1.1.f Dự báo lan truyền ứ tắc giao thông

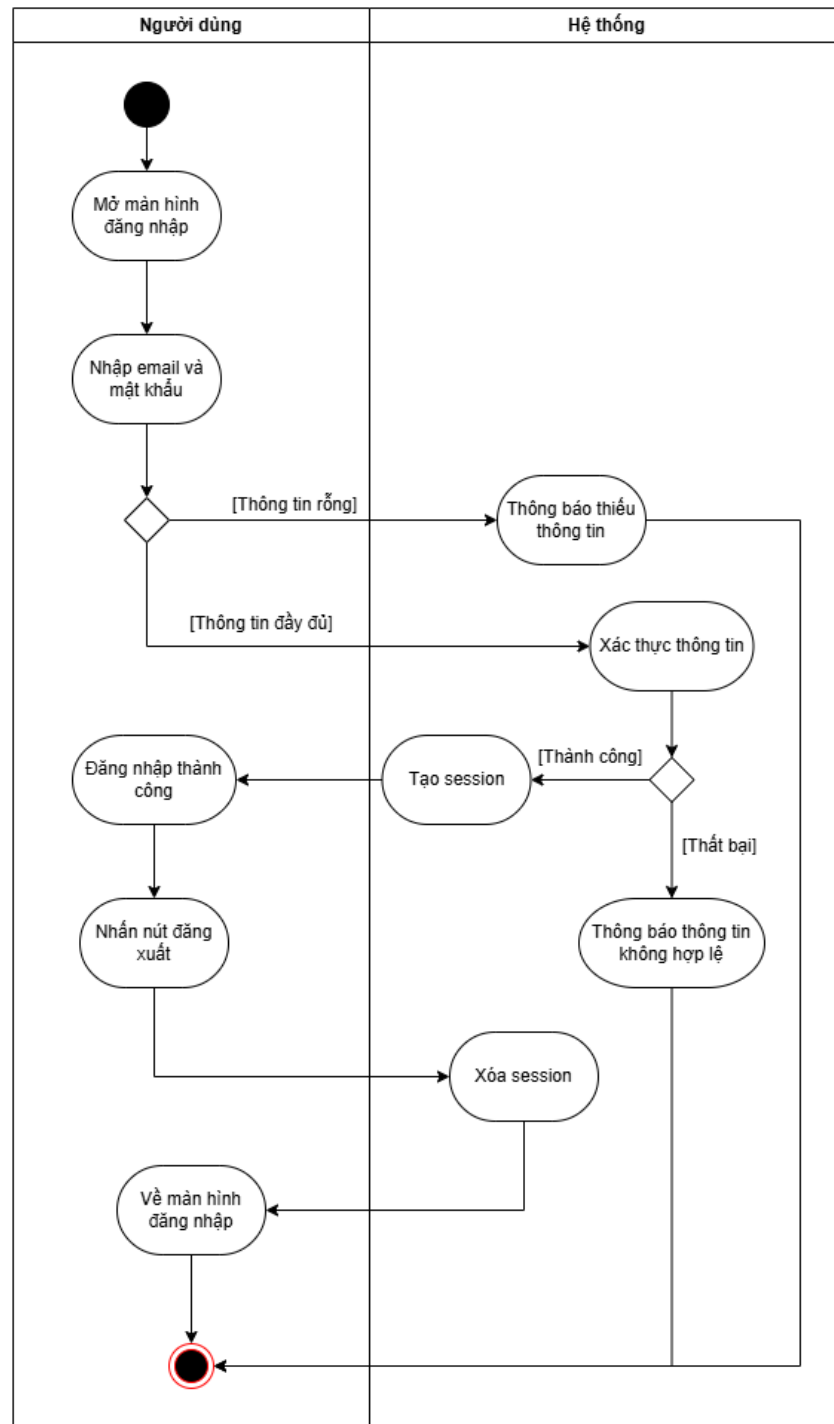


Hình 11: Sơ đồ Use Case dự báo ứ tắc giao thông

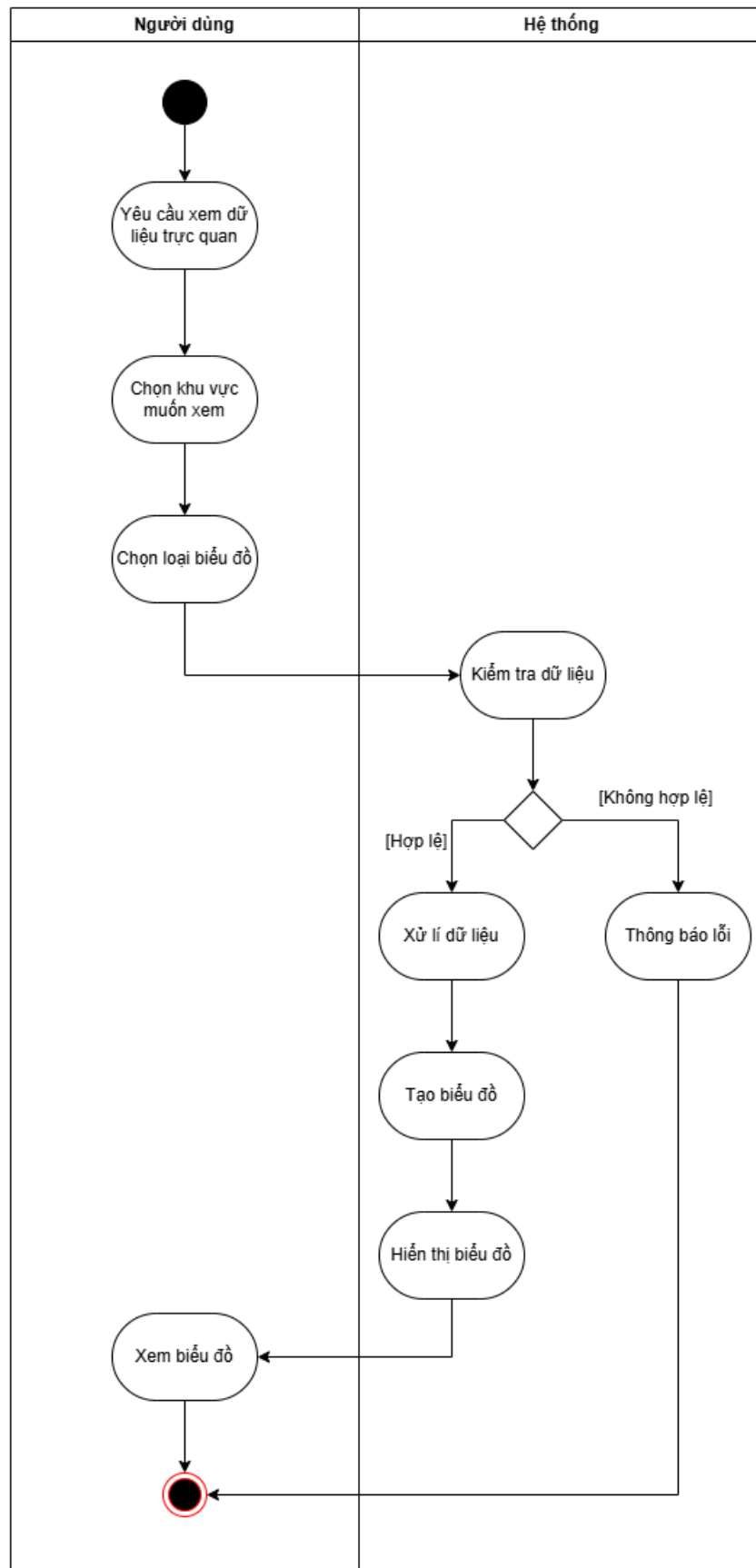
Use case id	7
Tên use case	Dự báo lan truyền ùn tắc giao thông
Mô tả	Cho phép người dùng xem dự báo việc một nút giao thông bị tắc nghẽn sẽ ảnh hưởng và lan truyền đến các nút giao thông lân cận nào trong mạng lưới giao thông, đồng thời ước lượng khoảng thời gian lan truyền.
Tác nhân	Người dùng.
Điều kiện kích hoạt	Người dùng chọn một nút giao thông đang hoặc có nguy cơ bị tắc nghẽn và yêu cầu hệ thống thực hiện dự báo lan truyền.
Điều kiện tiên quyết	Dữ liệu giao thông đã được thu thập và tiền xử lý. Mạng lưới giao thông đã được xây dựng. Hệ thống có mô hình hoặc thuật toán dự báo lan truyền ùn tắc.
Điều kiện sau hành động	Kết quả dự báo được tạo ra, bao gồm danh sách các nút bị ảnh hưởng và thời gian lan truyền tương ứng.
Luồng chính	1. Người dùng chọn nút giao thông cần dự báo. 2. Người dùng chọn khoảng thời gian dự báo. 3. Hệ thống truy xuất dữ liệu giao thông và cấu trúc mạng lưới liên quan. 4. Hệ thống thực hiện mô phỏng hoặc mô hình dự báo lan truyền ùn tắc. 5. Hệ thống xác định các nút giao thông bị ảnh hưởng và thời gian lan truyền. 6. Kết quả dự báo được hiển thị dưới dạng bản đồ, biểu đồ hoặc bảng số liệu.
Luồng rẽ nhánh	4a. Nếu dữ liệu không đủ hoặc mô hình không hội tụ, hệ thống hiển thị thông báo lỗi hoặc cảnh báo độ tin cậy thấp.
Kết quả	Người dùng quan sát được phạm vi lan truyền ùn tắc và thời gian ảnh hưởng, hoặc nhận thông báo nếu dự báo không khả thi.

6.1.2 Mô hình hóa quy trình nghiệp vụ

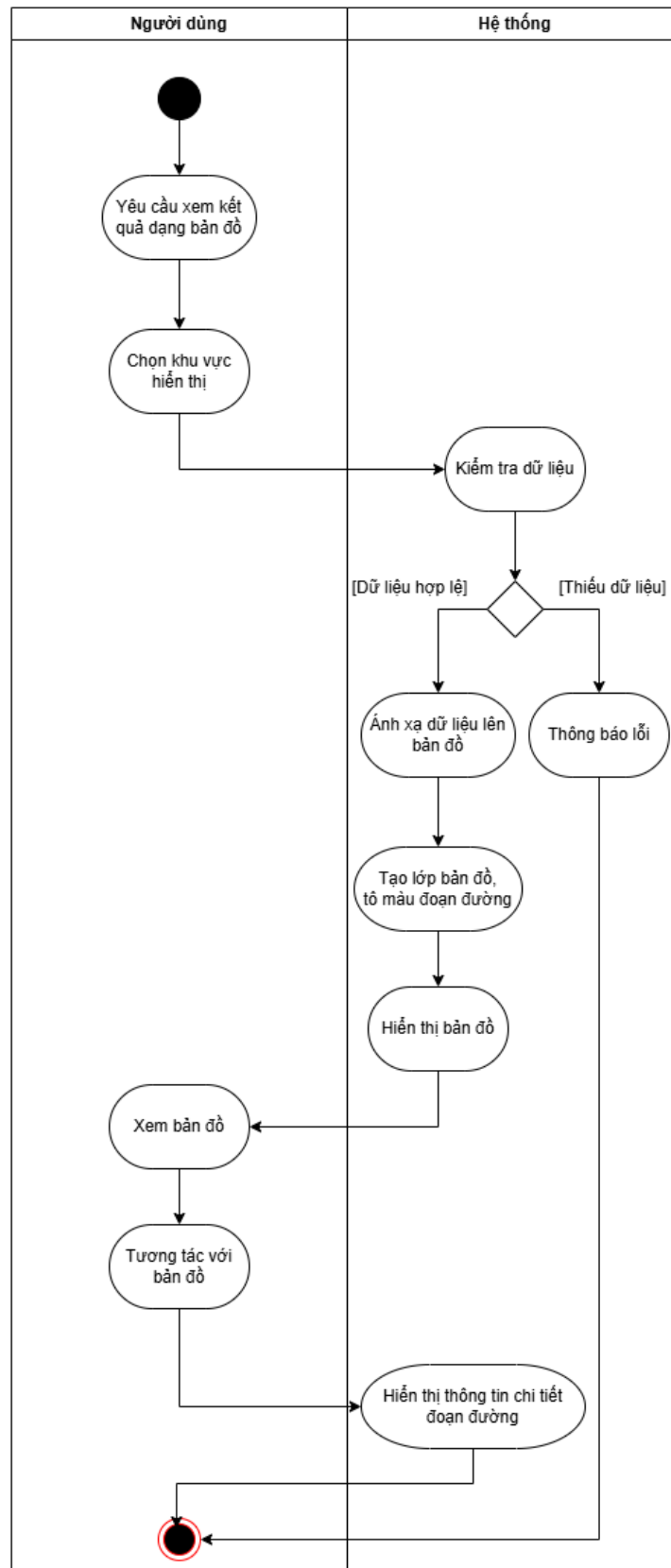
6.1.2.a Activity diagram



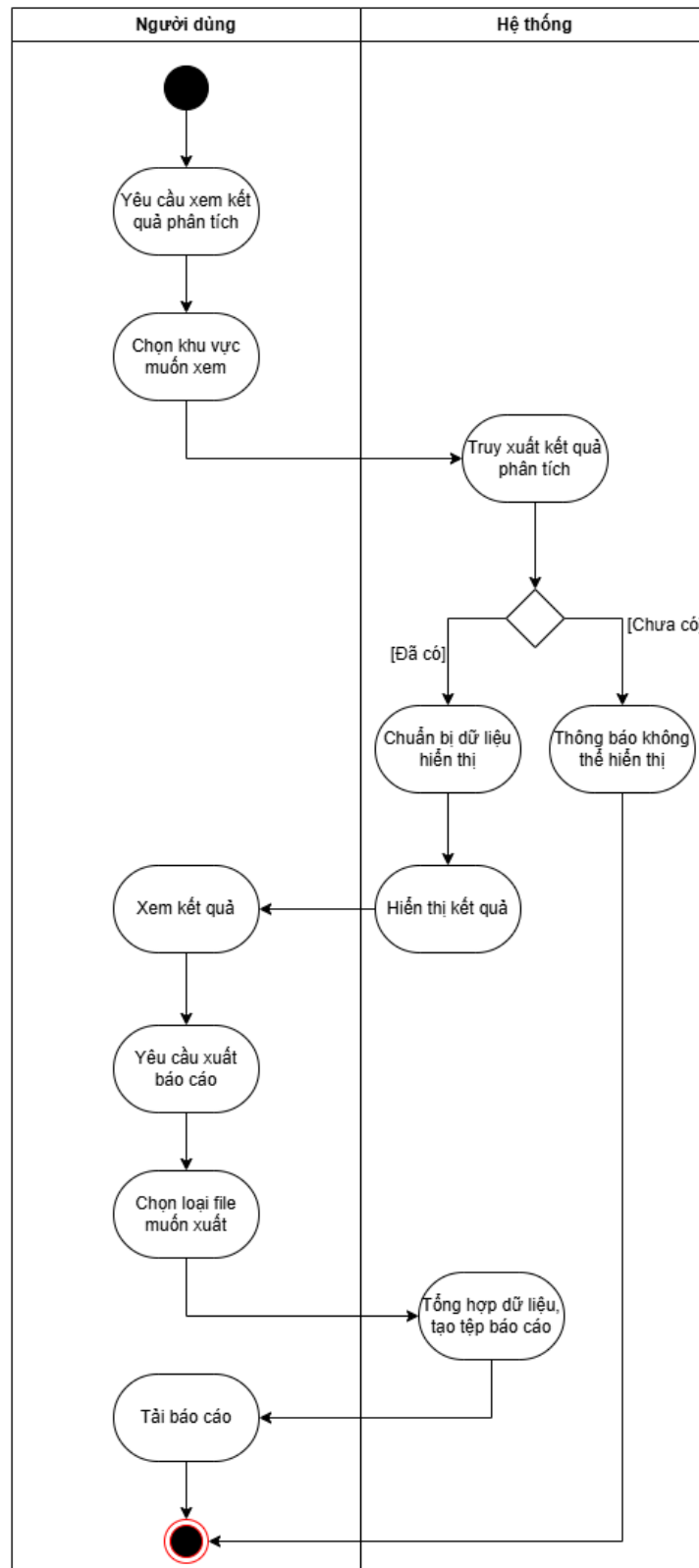
Hình 12: Sơ đồ Activity đăng nhập, đăng xuất



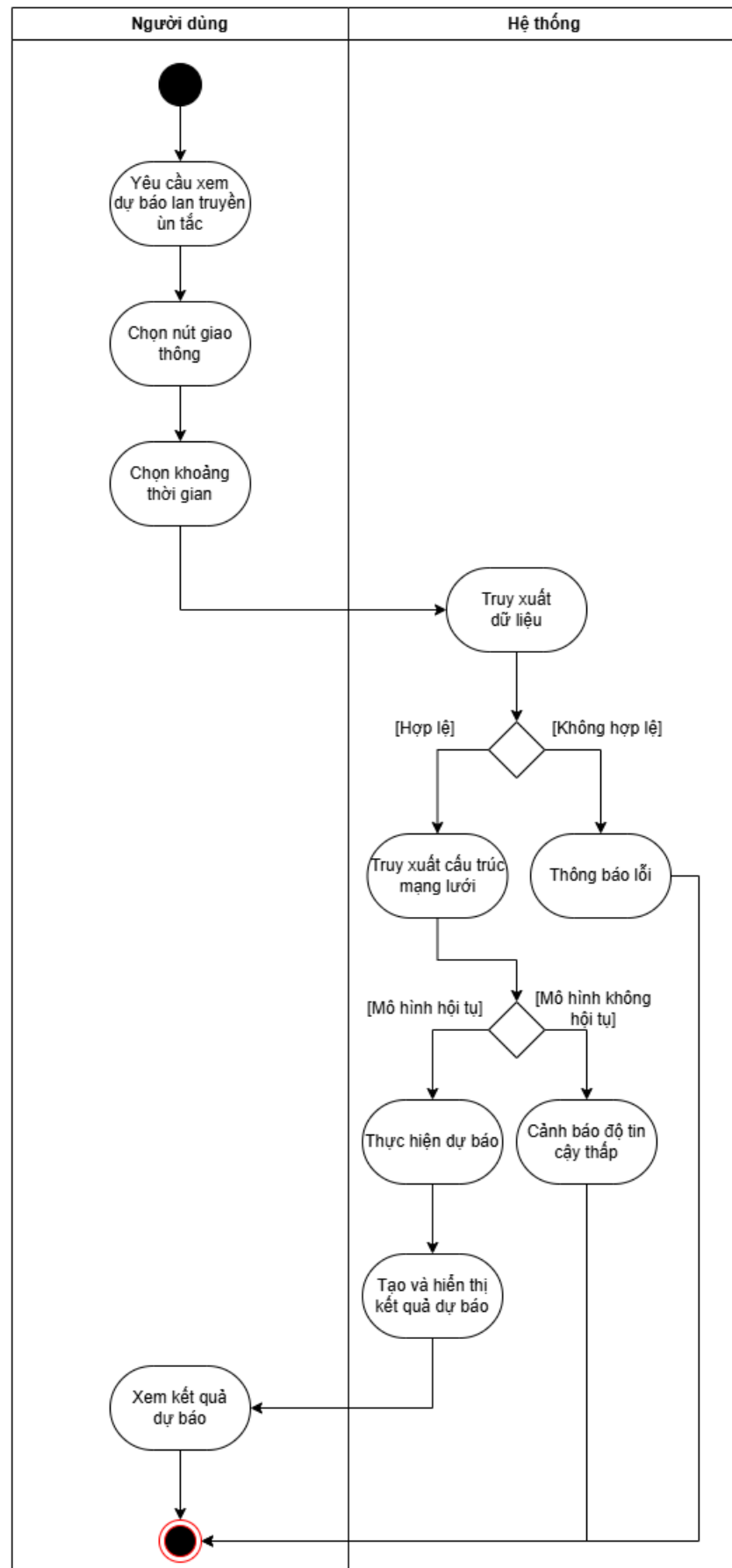
Hình 13: Sơ đồ Activity trực quan hóa dữ liệu



Hình 14: Sơ đồ Activity xem kết quả dạng bản đồ

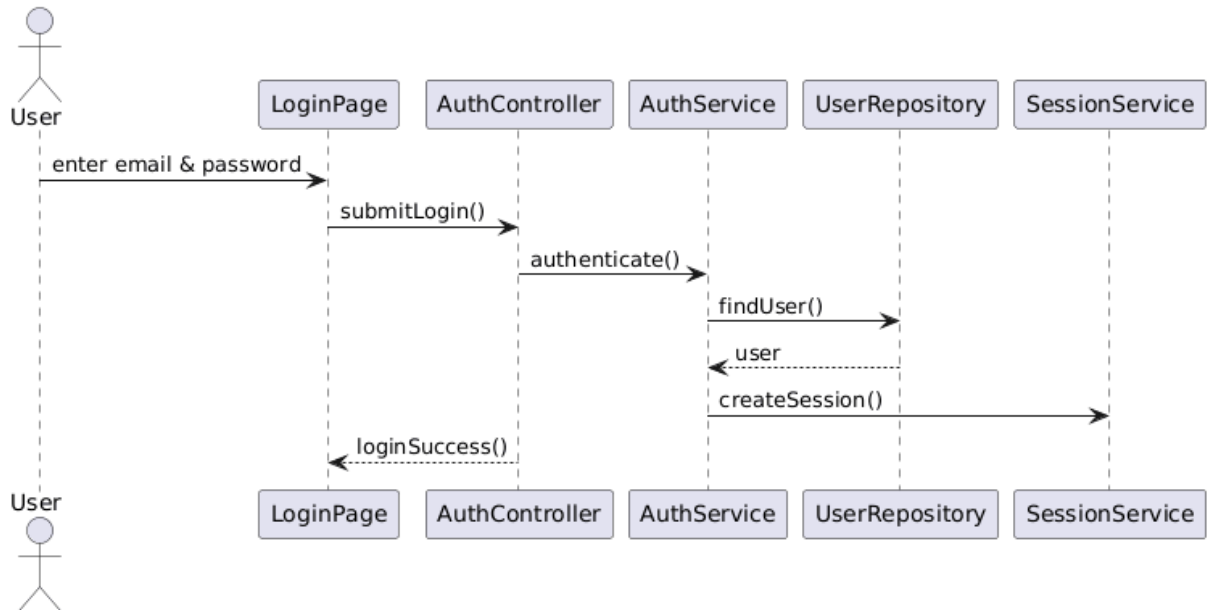


Hình 15: Sơ đồ Activity xem và xuất kết quả phân tích

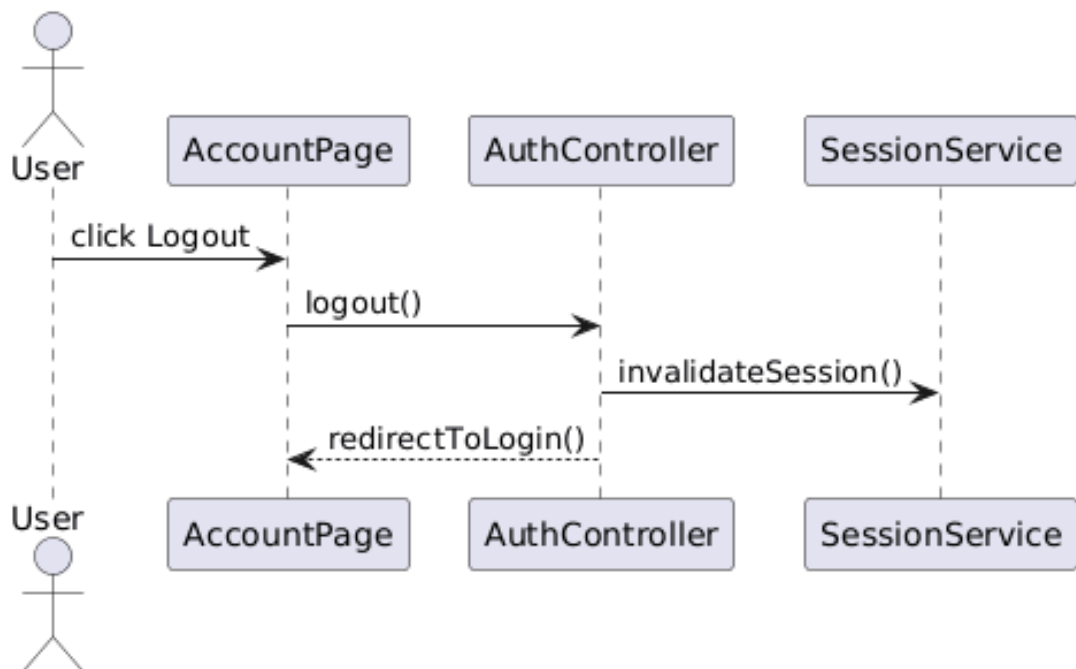


Hình 16: Sơ đồ Activity dự báo lan truyền ừn tắc giao thông

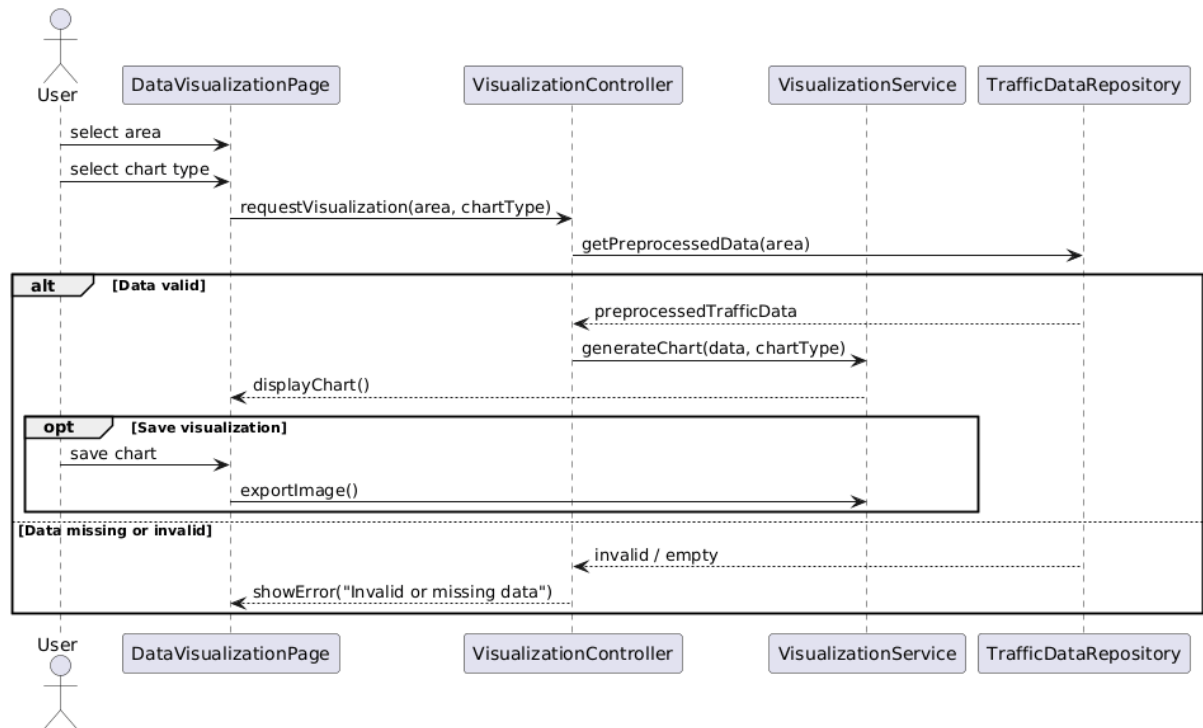
6.1.2.b Sequence diagram



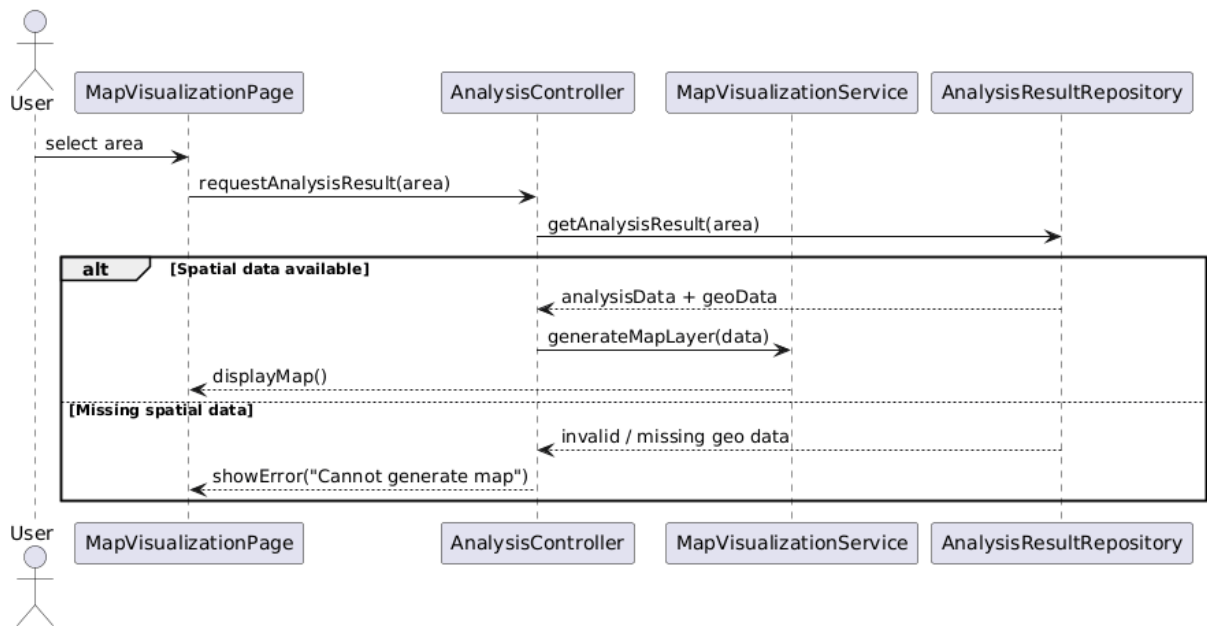
Hình 17: Sơ đồ Sequence đăng nhập



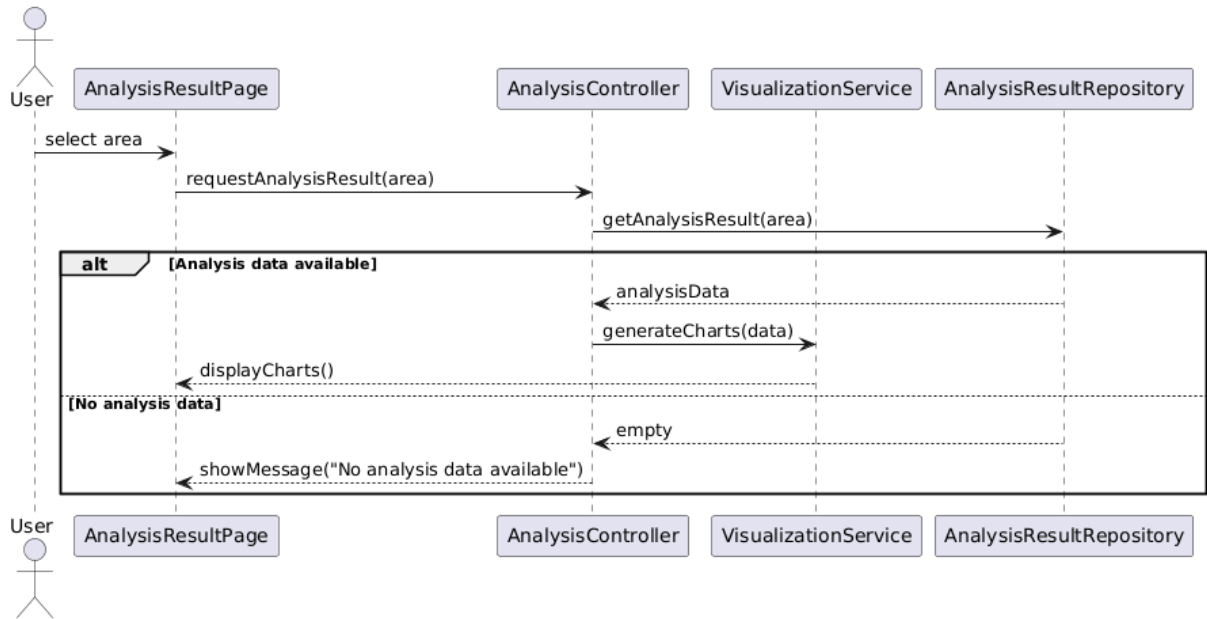
Hình 18: Sơ đồ Sequence đăng xuất



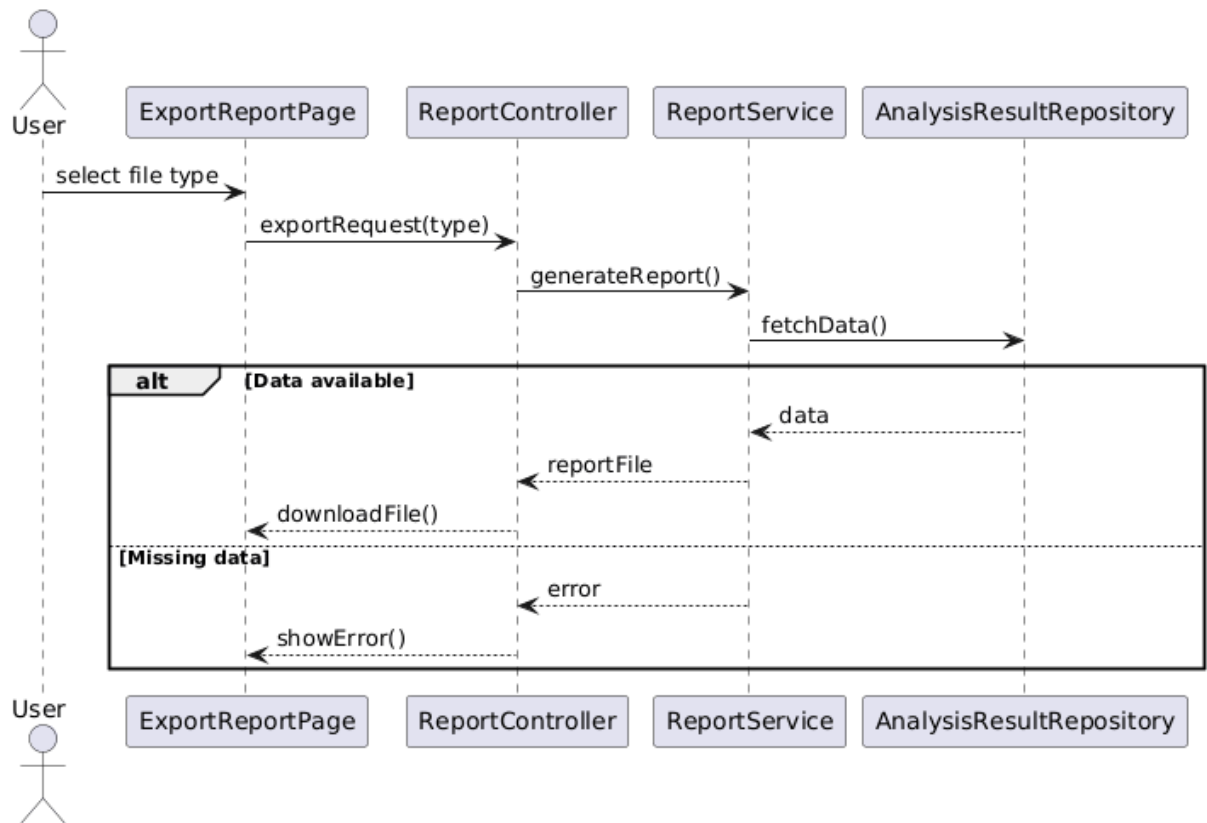
Hình 19: Sơ đồ Sequence trực quan hóa dữ liệu



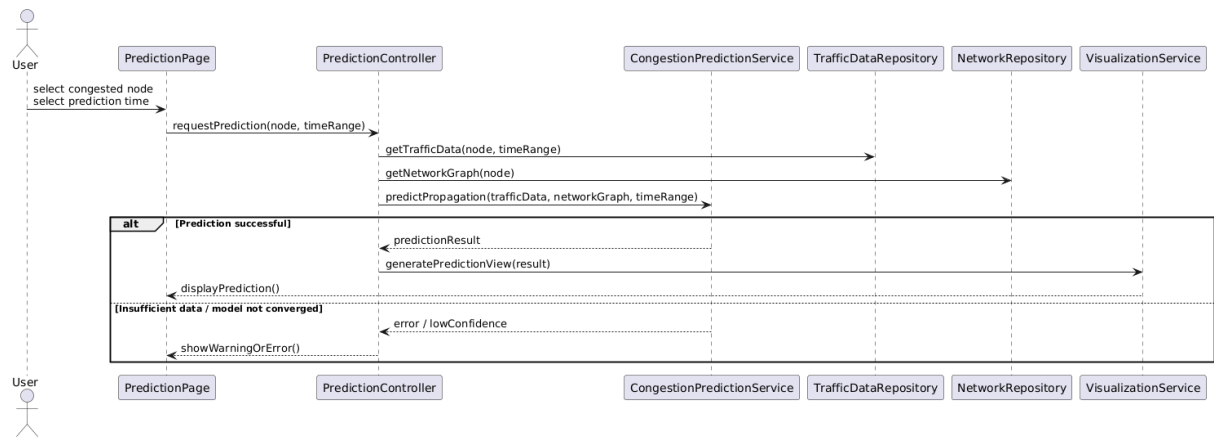
Hình 20: Sơ đồ Sequence xem kết quả dạng bản đồ



Hình 21: Sơ đồ Sequence xem kết quả phân tích



Hình 22: Sơ đồ Sequence xuất kết quả phân tích



Hình 23: Sơ đồ Sequence dự báo lan truyền ùn tắc giao thông

7 Hiện thực và kiểm thử

7.1 Hiện thực

7.2 Kiểm thử hệ thống

7.3 Kết chương



8 Đánh giá hệ thống

9 Tổng kết

9.1 Kết quả đạt được

9.2 Hướng phát triển



Kế hoạch thực hiện Đồ án tốt nghiệp

Tài liệu

- [1] M. Saberi, H. Hamedmoghadam, M. Ashfaq *et al.*, “A simple contagion process describes spreading of traffic jams in urban networks,” *Nature Communications*, vol. 11, no. 1, pp. 15353, 2020.
- [2] L. Zhao, Y. Song, C. Zhang, Y. Liu, P. Wang, T. Lin, M. Deng, H. Li, “T-GCN: A temporal graph convolutional network for traffic prediction,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 21, no. 9, pp. 3848–3858, 2020.
- [3] A. M. Nagy, V. Simon, “Improving traffic prediction using congestion propagation patterns in smart cities,” *Advanced Engineering Informatics*, vol. 50, pp. 101343, 2021.
- [4] C. Li, W. Yue, G. Mao, Z. Xu, “Congestion propagation based bottleneck identification in urban road networks,” *IEEE Transactions on Vehicular Technology*, vol. 69, no. 5, pp. 4827–4837, 2020.
- [5] Y. Zhang, Y. Li, X. Zhou, X. Kong, J. Luo, “Curb-GAN: Conditional urban traffic estimation through spatio-temporal generative adversarial networks,” in *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 842–852, 2020.
- [6] A. Ermagun, S. Chatterjee, D. Levinson, “Using temporal detrending to observe the spatial correlation of traffic,” *PLOS ONE*, vol. 12, no. 5, e0176853, 2017.
- [7] Y. Xu, X. Cai, E. Wang, W. Liu, Y. Yang, F. Yang, “Dynamic traffic correlations based spatio-temporal graph convolutional network for urban traffic prediction,” *Information Sciences*, vol. 621, pp. 580–595, 2023.
- [8] A. Lampo, J. Borge-Holthoefer, S. Gómez, A. Solé-Ribalta, “Emergence of spatial transitions in urban congestion dynamics,” *Applied Network Science*, vol. 6, no. 1, pp. 1–19, 2021.
- [9] J. Chen, Y. Wu, “The propagation of congestion on transportation networks analyzed by the percolation process,” *Mathematics*, vol. 12, no. 20, pp. 3247, 2024.

PHỤ LỤC

A Mô tả chi tiết dữ liệu TomTom

Phụ lục này trình bày cấu trúc dữ liệu lịch sử thu thập từ TomTom Statistics API, bao gồm vận tốc trung bình, mức độ tắc nghẽn và chỉ số độ tin cậy theo từng đoạn đường.

B Chi tiết thuật toán mô phỏng lan truyền tắc nghẽn

Thuật toán ?? mô phỏng quá trình lan truyền tắc nghẽn trên đồ thị nhân quả, dựa trên độ trễ lan truyền và xác suất ảnh hưởng giữa các nút.

C Kiến trúc hệ thống triển khai

Hình ?? mô tả kiến trúc tổng thể của hệ thống, bao gồm tầng thu thập dữ liệu, xử lý luồng, mô hình học máy và giao diện người dùng.